



Image Compression and Watermarking

But life is short and information endless ... Abbreviation is a necessary evil and the abbreviator's business is to make the best of a job which, although bad, is still better than nothing.

Aldous Huxley

The Titanic will protect itself.

Robert Ballard

Preview

Image compression, the art and science of reducing the amount of data required to represent an image, is one of the most useful and commercially successful technologies in the field of digital image processing. The number of images that are compressed and decompressed daily is staggering, and the compressions and decompressions themselves are virtually invisible to the user. Everyone who owns a digital camera, surfs the web, or streams the latest Hollywood movies over the Internet benefits from the algorithms and standards that will be discussed in this chapter. The material, which is largely introductory in nature, is applicable to both still-image and video applications. We will introduce both theory and practice, examining the most frequently used compression techniques, and describing the industry standards that make them useful. The chapter concludes with an introduction to *digital image watermarking*, the process of inserting visible and invisible data (such as copyright information) into images.

Upon completion of this chapter, students should:

- Be able to measure the amount of information in a digital image.
- Understand the main sources of data redundancy in digital images.
- Know the difference between lossy and error-free compression, and the amount of compression that is possible with each.
- Be familiar with the popular image compression standards, such as JPEG and JPEG-2000, that are in use today.
- Understand the principal image compression methods, and how and why they work.
- Be able to compress and decompress grayscale, color, and video imagery.
- Know the difference between visible, invisible, robust, fragile, public, private, restricted-key, and unrestricted-key watermarks.
- Understand the basics of watermark insertion and extraction in both the spatial and transform domain.

8.1 FUNDAMENTALS

The term *data compression* refers to the process of reducing the amount of data required to represent a given quantity of information. In this definition, *data* and *information* are not the same; data are the means by which information is conveyed. Because various amounts of data can be used to represent the same amount of information, representations that contain irrelevant or repeated information are said to contain *redundant data*. If we let b and b' denote the number of bits (or information-carrying units) in two representations of the same information, the *relative data redundancy*, R , of the representation with b bits is

$$R = 1 - \frac{1}{C} \quad (8-1)$$

where C , commonly called the *compression ratio*, is defined as

$$C = \frac{b}{b'} \quad (8-2)$$

If $C = 10$ (sometimes written 10:1), for instance, the larger representation has 10 bits of data for every 1 bit of data in the smaller representation. The corresponding relative data redundancy of the larger representation is 0.9 ($R = 0.9$), indicating that 90% of its data is redundant.

In the context of digital image compression, b in Eq. (8-2) usually is the number of bits needed to represent an image as a 2-D array of intensity values. The 2-D intensity arrays introduced in Section 2.4 are the preferred formats for human viewing and interpretation—and the standard by which all other representations are judged. When it comes to compact image representation, however, these formats are far from optimal. Two-dimensional intensity arrays suffer from three principal types of data redundancies that can be identified and exploited:

1. *Coding redundancy.* A code is a system of symbols (letters, numbers, bits, and the like) used to represent a body of information or set of events. Each piece of information or event is assigned a sequence of *code symbols*, called a *code word*. The number of symbols in each code word is its *length*. The 8-bit codes that are used to represent the intensities in most 2-D intensity arrays contain more bits than are needed to represent the intensities.
2. *Spatial and temporal redundancy.* Because the pixels of most 2-D intensity arrays are correlated spatially (i.e., each pixel is similar to or dependent upon neighboring pixels), information is unnecessarily replicated in the representations of the correlated pixels. In a video sequence, temporally correlated pixels (i.e., those similar to or dependent upon pixels in nearby frames) also duplicate information.
3. *Irrelevant information.* Most 2-D intensity arrays contain information that is ignored by the human visual system and/or extraneous to the intended use of the image. It is redundant in the sense that it is not used.



FIGURE 8.1 Computer generated $256 \times 256 \times 8$ bit images with (a) coding redundancy, (b) spatial redundancy, and (c) irrelevant information. (Each was designed to demonstrate one principal redundancy, but may exhibit others as well.)

The computer-generated images in Figs. 8.1(a) through (c) exhibit each of these fundamental redundancies. As will be seen in the next three sections, compression is achieved when one or more redundancy is reduced or eliminated.

CODING REDUNDANCY

In Chapter 3, we developed techniques for image enhancement by histogram processing, assuming that the intensity values of an image are random quantities. In this section, we will use a similar formulation to introduce optimal information coding.

Assume that a discrete random variable r_k in the interval $[0, L-1]$ is used to represent the intensities of an $M \times N$ image, and that each r_k occurs with probability $p_r(r_k)$. As in Section 3.3,

$$p_r(r_k) = \frac{n_k}{MN} \quad k = 0, 1, 2, \dots, L-1 \quad (8-3)$$

where L is the number of intensity values, and n_k is the number of times that the k th intensity appears in the image. If the number of bits used to represent each value of r_k is $l(r_k)$, then the average number of bits required to represent each pixel is

$$L_{\text{avg}} = \sum_{k=0}^{L-1} l(r_k) p_r(r_k) \quad (8-4)$$

That is, the average length of the code words assigned to the various intensity values is found by summing the products of the number of bits used to represent each intensity and the probability that the intensity occurs. The total number of bits required to represent an $M \times N$ image is MNL_{avg} . If the intensities are represented

using a *natural* m -bit fixed-length code,[†] the right-hand side of Eq. (8-4) reduces to m bits. That is, $L_{\text{avg}} = m$ when m is substituted for $l(r_k)$. The constant m can be taken outside the summation, leaving only the sum of the $p_r(r_k)$ for $0 \leq k \leq L-1$, which, of course, equals 1.

EXAMPLE 8.1: A simple illustration of variable-length coding.

The computer-generated image in Fig. 8.1(a) has the intensity distribution shown in the second column of Table 8.1. If a natural 8-bit binary code (denoted as code 1 in Table 8.1) is used to represent its four possible intensities, L_{avg} (the average number of bits for code 1) is 8 bits, because $l_1(r_k) = 8$ bits for all r_k . On the other hand, if the scheme designated as code 2 in Table 8.1 is used, the average length of the encoded pixels is, in accordance with Eq. (8-4),

$$L_{\text{avg}} = 0.25(2) + 0.47(1) + 0.03(3) = 1.81 \text{ bits}$$

The total number of bits needed to represent the entire image is $MNL_{\text{avg}} = 256 \times 56 \times 1.81$, or 118,621. From Eqs. (8-2) and (8-1), the resulting compression and corresponding relative redundancy are

$$C = \frac{256 \times 256 \times 8}{118,621} = \frac{8}{1.81} \approx 4.42$$

and

$$R = 1 - \frac{1}{4.42} = 0.774$$

respectively. Thus, 77.4% of the data in the original 8-bit 2-D intensity array is redundant.

The compression achieved by code 2 results from assigning fewer bits to the more probable intensity values than to the less probable ones. In the resulting *variable-length code*, r_{128} (the image's most probable intensity) is assigned the 1-bit code word 1 [of length $l_2(128) = 1$], while r_{255} (its least probable occurring intensity) is assigned the 3-bit code word 001 [of length $l_2(255) = 3$]. Note that the best *fixed-length code* that can be assigned to the intensities of the image in Fig. 8.1(a) is the natural 2-bit counting sequence $\{00, 01, 10, 11\}$, but the resulting compression is only 8/2 or 4:1—about 10% less than the 4.42:1 compression of the variable-length code.

As the preceding example shows, *coding redundancy* is present when the codes assigned to a set of events (such as intensity values) do not take full advantage of the probabilities of the events. Coding redundancy is almost always present when the intensities of an image are represented using a natural binary code. The reason is that most images are composed of objects that have a regular and somewhat predictable morphology (shape) and reflectance, and are sampled so the objects being depicted are much larger than the picture elements. The natural consequence is that,

[†]A *natural* binary code is one in which each event or piece of information to be encoded (such as intensity value) is assigned one of 2^m codes from an m -bit binary counting sequence.

TABLE 8.1

Example of variable-length coding.

r_k	$p_r(r_k)$	Code 1	$l_1(r_k)$	Code 2	$l_2(r_k)$
$r_{87} = 87$	0.25	01010111	8	01	2
$r_{128} = 128$	0.47	01010111	8	1	1
$r_{186} = 186$	0.25	01010111	8	000	3
$r_{255} = 255$	0.03	01010111	8	001	3
r_k for $k = 87, 128, 186, 255$	0	—	8	—	0

for most images, certain intensities are more probable than others (that is, the histograms of most images are not uniform). A natural binary encoding assigns the same number of bits to both the most and least probable values, failing to minimize Eq. (8-4), and resulting in coding redundancy.

SPATIAL AND TEMPORAL REDUNDANCY

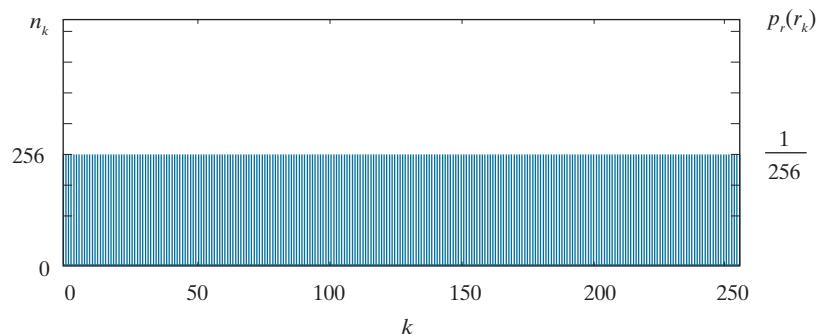
Consider the computer-generated collection of constant intensity lines in Fig. 8.1(b). In the corresponding 2-D intensity array:

1. All 256 intensities are equally probable. As Fig. 8.2 shows, the histogram of the image is uniform.
2. Because the intensity of each line was selected randomly, its pixels are independent of one another in the vertical direction.
3. Because the pixels along each line are identical, they are maximally correlated (completely dependent on one another) in the horizontal direction.

The first observation tells us that the image in Fig. 8.1(b) (when represented as a conventional 8-bit intensity array) cannot be compressed by variable-length coding alone. Unlike the image of Fig. 8.1(a) and Example 8.1, whose histogram was *not* uniform, a fixed-length 8-bit code in this case minimizes Eq. (8-4). Observations 2 and 3 reveal a significant spatial redundancy that can be eliminated by representing the image in Fig. 8.1(b) as a sequence of *run-length pairs*, where each run-length pair specifies the start of a new intensity and the number of consecutive pixels that have that intensity. A run-length based representation compresses the original 2-D, 8-bit

FIGURE 8.2

The intensity histogram of the image in Fig. 8.1(b).



intensity array by $(256 \times 256 \times 8) / [(256 + 256) \times 8]$ or 128:1. Each 256-pixel line of the original representation is replaced by a single 8-bit intensity value and length 256 in the run-length representation.

In most images, pixels are correlated spatially (in both x and y) and in time (when the image is part of a video sequence). Because most pixel intensities can be predicted reasonably well from neighboring intensities, the information carried by a single pixel is small. Much of its visual contribution is redundant in the sense that it can be inferred from its neighbors. To reduce the redundancy associated with spatially and temporally correlated pixels, a 2-D intensity array must be transformed into a more efficient but usually “non-visual” representation. For example, run-lengths or the differences between adjacent pixels can be used. Transformations of this type are called *mappings*. A mapping is said to be *reversible* if the pixels of the original 2-D intensity array can be reconstructed without error from the transformed data set; otherwise, the mapping is said to be *irreversible*.

IRRELEVANT INFORMATION

One of the simplest ways to compress a set of data is to remove superfluous data from the set. In the context of digital image compression, information that is ignored by the human visual system, or is extraneous to the intended use of an image, are obvious candidates for omission. Thus, the computer-generated image in Fig. 8.1(c), because it appears to be a homogeneous field of gray, can be represented by its average intensity alone—a single 8-bit value. The original $256 \times 256 \times 8$ bit intensity array is reduced to a single byte, and the resulting compression is $(256 \times 256 \times 8) / 8$ or 65,536:1. Of course, the original $256 \times 256 \times 8$ bit image must be recreated to view and/or analyze it, but there would be little or no perceived decrease in reconstructed image quality.

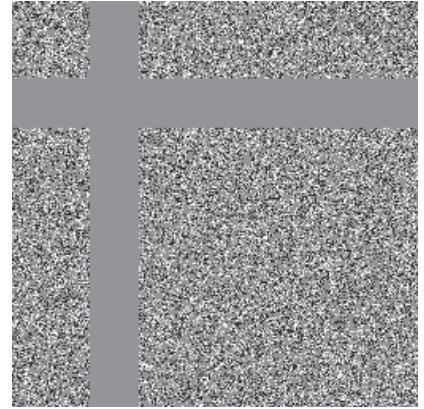
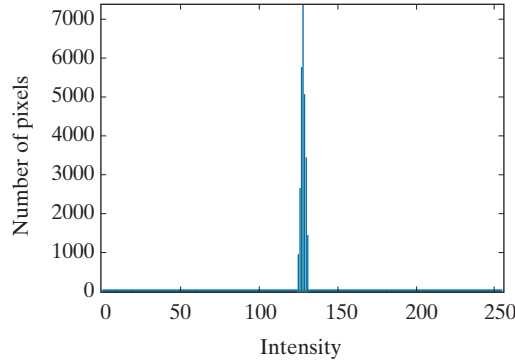
Figure 8.3(a) shows the histogram of the image in Fig. 8.1(c). Note that there are several intensity values (125 through 131) actually present. The human visual system averages these intensities, perceives only the average value, then ignores the small changes in intensity that are present in this case. Figure 8.3(b), a histogram-equalized version of the image in Fig. 8.1(c), makes the intensity changes visible *and* reveals two previously undetected regions of constant intensity—one oriented vertically, and the other horizontally. If the image in Fig. 8.1(c) is represented by its average value alone, this “invisible” structure (i.e., the constant intensity regions) and the random intensity variations surrounding them (real information) is lost. Whether or not this information should be preserved is application dependent. If the information is important, as it might be in a medical application like digital X-ray archival, it should not be omitted; otherwise, the information is redundant and can be excluded for the sake of compression performance.

We conclude this section by noting that the redundancy examined here is fundamentally different from the redundancies discussed in the previous two sections. Its elimination is possible because the information itself is not essential for normal visual processing and/or the intended use of the image. Because its omission results in a loss of quantitative information, its removal is commonly referred to as

a b

FIGURE 8.3

(a) Histogram of the image in Fig. 8.1(c) and (b) a histogram equalized version of the image.



quantization. This terminology is consistent with normal use of the word, which generally means the mapping of a broad range of input values to a limited number of output values (see Section 2.4). Because information is lost, quantization is an irreversible operation.

MEASURING IMAGE INFORMATION

In the previous sections, we introduced several ways to reduce the amount of data used to represent an image. The question that naturally arises is: How few bits are actually needed to represent the information in an image? That is, is there a minimum amount of data that is sufficient to describe an image without losing information? *Information theory* provides the mathematical framework to answer this and related questions. Its fundamental premise is that the generation of information can be modeled as a probabilistic process which can be measured in a manner that agrees with intuition. In accordance with this supposition, a random event E with probability $P(E)$ is said to contain

$$I(E) = \log \frac{1}{P(E)} = -\log P(E) \quad (8-5)$$

units of information. If $P(E) = 1$ (that is, the event always occurs), $I(E) = 0$ and no information is attributed to it. Because no uncertainty is associated with the event, no information would be transferred by communicating that the event has occurred [it *always* occurs if $P(E) = 1$].

The base of the logarithm in Eq. (8-5) determines the unit used to measure information. If the base m logarithm is used, the measurement is said to be in m -ary units. If the base 2 is selected, the unit of information is the *bit*. Note that if $P(E) = \frac{1}{2}$, $I(E) = -\log_2 \frac{1}{2}$ or 1 bit. That is, 1 bit is the amount of information conveyed when one of two possible equally likely events occurs. A simple example is flipping a coin and communicating the result.

Consult the book website for a brief review of information and probability theory.

Given a source of statistically independent random events from a discrete set of possible events $\{a_1, a_1, \dots, a_J\}$ with associated probabilities $\{P(a_1), P(a_1), \dots, P(a_J)\}$, the average information per source output, called the *entropy* of the source, is

$$H = -\sum_{j=1}^J P(a_j) \log P(a_j) \quad (8-6)$$

The a_j in this equation are called *source symbols*. Because they are statistically independent, the source itself is called a *zero-memory source*.

If an image is considered to be the output of an imaginary zero-memory “intensity source,” we can use the histogram of the observed image to estimate the symbol probabilities of the source. Then, the intensity source’s entropy becomes

$$\tilde{H} = -\sum_{k=0}^{L-1} p_r(r_k) \log_2 p_r(r_k) \quad (8-7)$$

where variables L , r_k , and $p_r(r_k)$ are as defined earlier and in Section 3.3. Because the base 2 logarithm is used, Eq. (8-7) is the average information per intensity output of the imaginary intensity source in bits. It is not possible to code the *intensity values* of the imaginary source (and thus the sample image) with fewer than $\tilde{H}$ bits/pixel.

Equation (8-6) is for zero-memory sources with J source symbols. Equation (8-7) uses probability estimates for the $L-1$ intensity values in an image.

EXAMPLE 8.2: Image entropy estimates.

The entropy of the image in Fig. 8.1(a) can be estimated by substituting the intensity probabilities from Table 8.1 into Eq. (8-7):

$$\begin{aligned} \tilde{H} &= -[0.25 \log_2 0.25 + 0.47 \log_2 0.47 + 0.25 \log_2 0.25 + 0.03 \log_2 0.03] \\ &= -[0.25(-2) + 0.47(-1.09) + 0.25(-2) + 0.03(-5.06)] \\ &\approx 1.6614 \text{ bits/pixel} \end{aligned}$$

In a similar manner, the entropies of the images in Fig. 8.1(b) and (c) can be shown to be 8 bits/pixel and 1.566 bits/pixel, respectively. Note that the image in Fig. 8.1(a) appears to have the most visual information, but has almost the lowest computed entropy—1.66 bits/pixel. The image in Fig. 8.1(b) has almost five times the entropy of the image in (a), but appears to have about the same (or less) visual information. The image in Fig. 8.1(c), which seems to have little or no information, has almost the same entropy as the image in (a). The obvious conclusion is that the amount of entropy, and thus information in an image, is far from intuitive.

Shannon’s First Theorem

Recall that the variable-length code in Example 8.1 was able to represent the intensities of the image in Fig. 8.1(a) using only 1.81 bits/pixel. Although this is higher than the 1.6614 bits/pixel entropy estimate from Example 8.2, Shannon’s first theorem, also called the *noiseless coding theorem* (Shannon [1948]), assures us that the

image in Fig. 8.1(a) can be represented with as few as 1.6614 bits/pixel. To prove it in a general way, Shannon looked at representing groups of consecutive source symbols with a single code word (rather than one code word per source symbol), and showed that

$$\lim_{n \rightarrow \infty} \left[\frac{L_{\text{avg},n}}{n} \right] = H \quad (8-8)$$

where $L_{\text{avg},n}$ is the average number of code symbols required to represent all n -symbol groups. In the proof, he defined the n th extension of a zero-memory source to be the hypothetical source that produces n -symbol blocks[†] using the symbols of the original source, and computed $L_{\text{avg},n}$ by applying Eq. (8-4) to the code words used to represent the n -symbol blocks. Equation (8-8) tells us that $L_{\text{avg},n}/n$ can be made arbitrarily close to H by encoding infinitely long extensions of the single-symbol source. That is, it is possible to represent the output of a zero-memory source with an average of H information units per source symbol.

If we now return to the idea that an image is a “sample” of the intensity source that produced it, a block of n source symbols corresponds to a group of n adjacent pixels. To construct a variable-length code for n -pixel blocks, the relative frequencies of the blocks must be computed. But the n th extension of a hypothetical intensity source with 256 intensity values has 256^n possible n -pixel blocks. Even in the simple case of $n = 2$, a 65,536 element histogram and up to 65,536 variable-length code words must be generated. For $n = 3$, as many as 16,777,216 code words are needed. So even for small values of n , computational complexity limits the usefulness of the extension coding approach in practice.

Finally, we note that although Eq. (8-7) provides a lower bound on the compression that can be achieved when directly coding statistically independent pixels, it breaks down when the pixels of an image are correlated. Blocks of correlated pixels can be coded with fewer average bits per pixel than the equation predicts. Rather than using source extensions, less correlated descriptors (such as intensity run-lengths) are normally selected and coded without extension. This was the approach used to compress Fig. 8.1(b) in the section on spatial and temporal redundancy. When the output of a source of information depends on a finite number of preceding outputs, the source is called a *Markov source* or *finite memory source*.

FIDELITY CRITERIA

It was noted earlier that the removal of “irrelevant visual” information involves a loss of real or quantitative image information. Because information is lost, a means of quantifying the nature of the loss is needed. Two types of criteria can be used for such an assessment: (1) objective fidelity criteria, and (2) subjective fidelity criteria.

[†]The output of the n th extension is an n -tuple of symbols from the underlying *single-symbol source*. It was considered a *block random variable* in which the probability of each n -tuple is the product of the probabilities of its individual symbols. The entropy of the n th extension is then n times the entropy of the single-symbol source from which it is derived.

When information loss can be expressed as a mathematical function of the input and output of a compression process, it is said to be based on an *objective fidelity criterion*. An example is the root-mean-squared (rms) error between two images. Let $f(x, y)$ be an input image, and $\hat{f}(x, y)$ be an approximation of $f(x, y)$ that results from compressing and subsequently decompressing the input. For any value of x and y , the error $e(x, y)$ between $f(x, y)$ and $\hat{f}(x, y)$ is

$$e(x, y) = \hat{f}(x, y) - f(x, y) \quad (8-9)$$

so that the total error between the two images is

$$\sum_{x=0}^{M-1} \sum_{y=0}^{N-1} [\hat{f}(x, y) - f(x, y)]$$

where the images are of size $M \times N$. The *root-mean-squared error*, e_{rms} , between $f(x, y)$ and $\hat{f}(x, y)$ is then the square root of the squared error averaged over the $M \times N$ array, or

$$e_{\text{rms}} = \left[\frac{1}{MN} \sum_{x=0}^{M-1} \sum_{y=0}^{N-1} [\hat{f}(x, y) - f(x, y)]^2 \right]^{1/2} \quad (8-10)$$

If $\hat{f}(x, y)$ is considered [by a simple rearrangement of the terms in Eq. (8-9)] to be the sum of the original image $f(x, y)$ and an error or “noise” signal $e(x, y)$, the *mean-squared signal-to-noise ratio* of the output image, denoted SNR_{ms} , can be defined as in Section 5.8:

$$\text{SNR}_{\text{ms}} = \frac{\sum_{x=0}^{M-1} \sum_{y=0}^{N-1} \hat{f}(x, y)^2}{\sum_{x=0}^{M-1} \sum_{y=0}^{N-1} [\hat{f}(x, y) - f(x, y)]^2} \quad (8-11)$$

The rms value of the signal-to-noise ratio, denoted SNR_{rms} , is obtained by taking the square root of Eq. (8-11).

While objective fidelity criteria offer a simple and convenient way to evaluate information loss, decompressed images are often ultimately viewed by humans. So, measuring image quality by the subjective evaluations of people is often more appropriate. This can be done by presenting a decompressed image to a cross section of viewers and averaging their evaluations. The evaluations may be made using an absolute rating scale, or by means of side-by-side comparisons of $f(x, y)$ and $\hat{f}(x, y)$. Table 8.2 shows one possible absolute rating scale. Side-by-side comparisons can be done with a scale such as $\{-3, -2, -1, 0, 1, 2, 3\}$ to represent the subjective evaluations *{much worse, worse, slightly worse, the same, slightly better, better, much better}*, respectively. In either case, the evaluations are based on *subjective fidelity criteria*.

TABLE 8.2
Rating scale of
the Television
Allocations Study
Organization.
(Frendendall and
Behrend.)

Value	Rating	Description
1	Excellent	An image of extremely high quality, as good as you could desire.
2	Fine	An image of high quality, providing enjoyable viewing. Interference is not objectionable.
3	Passable	An image of acceptable quality. Interference is not objectionable.
4	Marginal	An image of poor quality; you wish you could improve it. Interference is somewhat objectionable.
5	Inferior	A very poor image, but you could watch it. Objectionable interference is definitely present.
6	Unusable	An image so bad that you could not watch it.

EXAMPLE 8.3: Image quality comparisons.

Figure 8.4 shows three different approximations of the image in Fig. 8.1(a). Using Eq. (8-10) with Fig. 8.1(a) as $f(x,y)$ and Figs. 8.4(a) through (c) as $\hat{f}(x,y)$, the computed rms errors are 5.17, 15.67, and 14.17 intensity levels, respectively. In terms of rms error (an objective fidelity criterion), the images are ranked in order of decreasing quality as $\{(a), (c), (b)\}$. A subjective evaluation of the images using Table 8.2, however, might yield an *excellent* rating for (a), a *marginal* rating for (b), and an *inferior* or *unusable* rating for (c). Thus, using a subjective fidelity criteria, (b) is ranked ahead of (c).

IMAGE COMPRESSION MODELS

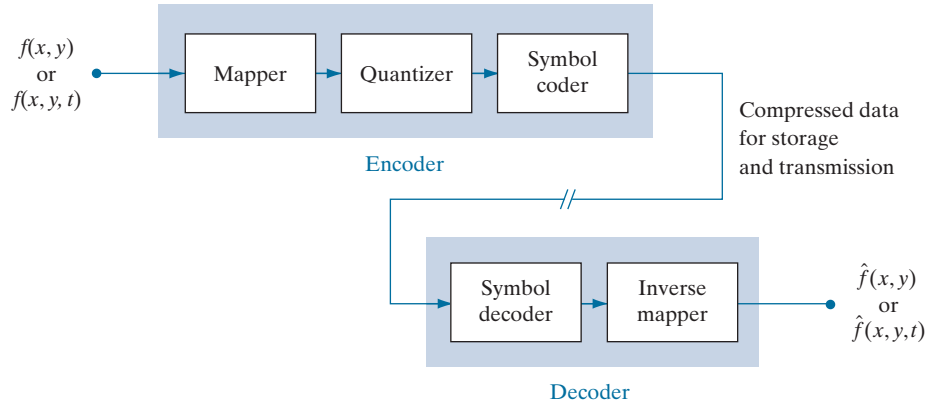
As Fig. 8.5 shows, an image compression system is composed of two distinct functional components: an *encoder* and a *decoder*. The encoder performs compression, and the decoder performs the complementary operation of decompression. Both operations can be performed in software, as is the case in Web browsers and many commercial image-editing applications, or in a combination of hardware and firmware, as in commercial DVD players. A *codec* is a device or program that is capable of both encoding and decoding.



FIGURE 8.4 Three approximations of the image in Fig. 8.1(a).

FIGURE 8.5

Functional block diagram of a general image compression system.



Here, the notation $f(x, \dots)$ is used to denote both $f(x, y)$ and $f(x, y, t)$.

Input image $f(x, \dots)$ is fed into the encoder, which creates a compressed representation of the input. This representation is stored for later use, or transmitted for storage and use at a remote location. When the compressed representation is presented to its complementary decoder, a reconstructed output image $\hat{f}(x, \dots)$ is generated. In still-image applications, the encoded input and decoder output are $f(x, y)$ and $\hat{f}(x, y)$, respectively. In video applications, they are $f(x, y, t)$ and $\hat{f}(x, y, t)$, where the discrete parameter t specifies time. In general, $\hat{f}(x, \dots)$ may or may not be an exact replica of $f(x, \dots)$. If it is, the compression system is called *error free*, *lossless*, or *information preserving*. If not, the reconstructed output image is distorted, and the compression system is referred to as *lossy*.

The Encoding or Compression Process

The encoder of Fig. 8.5 is designed to remove the redundancies described in the previous sections through a series of three independent operations. In the first stage of the encoding process, a *mapper* transforms $f(x, \dots)$ into a (usually nonvisual) format designed to reduce spatial and temporal redundancy. This operation generally is reversible, and may or may not directly reduce the amount of data required to represent the image. Run-length coding is an example of a mapping that normally yields compression in the first step of the encoding process. The mapping of an image into a set of less correlated transform coefficients (see Section 8.9) is an example of the opposite case (the coefficients must be further processed to achieve compression). In video applications, the mapper uses previous (and, in some cases, future) video frames to facilitate the removal of temporal redundancy.

The *quantizer* in Fig. 8.5 reduces the accuracy of the mapper's output in accordance with a pre-established fidelity criterion. The goal is to keep irrelevant information out of the compressed representation. As noted earlier, this operation is irreversible. It must be omitted when error-free compression is desired. In video applications, the *bit rate* of the encoded output is often measured (in bits/second), and is used to adjust the operation of the quantizer so a predetermined average output rate is maintained. Thus, the visual quality of the output can vary from frame to frame as a function of image content.

In the third and final stage of the encoding process, the *symbol coder* of Fig. 8.5 generates a fixed-length or variable-length code to represent the quantizer output, and maps the output in accordance with the code. In many cases, a variable-length code is used. The shortest code words are assigned to the most frequently occurring quantizer output values, thus minimizing coding redundancy. This operation is reversible. Upon its completion, the input image has been processed for the removal of each of the three redundancies described in the previous sections.

The Decoding or Decompression Process

The decoder of Fig. 8.5 contains only two components: a *symbol decoder* and an *inverse mapper*. They perform, in reverse order, the inverse operations of the encoder's symbol encoder and mapper. Because quantization results in irreversible information loss, an inverse quantizer block is not included in the general decoder model. In video applications, decoded output frames are maintained in an internal frame store (not shown) and used to reinsert the temporal redundancy that was removed at the encoder.

IMAGE FORMATS, CONTAINERS, AND COMPRESSION STANDARDS

In the context of digital imaging, an *image file format* is a standard way to organize and store image data. It defines how the data is arranged and the type of compression (if any) that is used. An *image container* is similar to a file format, but handles multiple types of image data. Image *compression standards*, on the other hand, define procedures for compressing and decompressing images—that is, for reducing the amount of data needed to represent an image. These standards are the underpinning of the widespread acceptance of image compression technology.

Figure 8.6 lists the most important image compression standards, file formats, and containers in use today, grouped by the type of image handled. The entries in blue are international standards sanctioned by the *International Standards Organization* (ISO), the *International Electrotechnical Commission* (IEC), and/or the *International Telecommunications Union* (ITU-T)—a *United Nations* (UN) organization that was once called the *Consultative Committee of the International Telephone and Telegraph* (CCITT). Two video compression standards, VC-1 by the *Society of Motion Pictures and Television Engineers* (SMPTE) and AVS by the *Chinese Ministry of Information Industry* (MII), are also included. Note that they are shown in black, which is used in Fig. 8.6 to denote entries that are not sanctioned by an international standards organization.

Tables 8.3 through 8.5 summarize the standards, formats, and containers listed in Fig. 8.6. Responsible organizations, targeted applications, and key compression methods are identified. The compression methods themselves are the subject of Sections 8.2 through 8.11, where we will describe the principal lossy and error-free compression methods in use today. The focus of these sections is on methods that have proven useful in mainstream binary, continuous-tone still-image, and video compression standards. The standards themselves are used to demonstrate the methods presented. In Tables 8.3 through 8.5, forward references to the relevant sections in which the compression methods are described are enclosed in square brackets.

FIGURE 8.6
Some popular image compression standards, file formats, and containers. Internationally sanctioned entries are shown in blue; all others are in black.

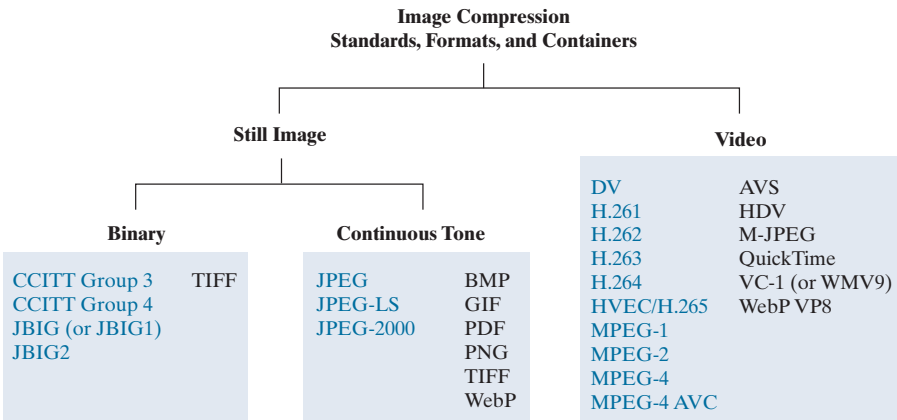


TABLE 8.3
Internationally sanctioned image compression standards. The numbers in brackets refer to sections in this chapter.

Name	Organization	Description
<i>Bi-Level Still Images</i>		
CCITT Group 3	ITU-T	Designed as a facsimile (FAX) method for transmitting binary documents over telephone lines. Supports 1-D and 2-D run-length [8.6] and Huffman [8.2] coding.
CCITT Group 4	ITU-T	A simplified and streamlined version of the CCITT Group 3 standard supporting 2-D run-length coding only.
JBIG or JBIG1	ISO/IEC/ ITU-T	A <i>Joint Bi-level Image Experts Group</i> standard for progressive, lossless compression of bi-level images. Continuous-tone images of up to 6 bits/pixel can be coded on a bit-plane basis [8.8]. Context-sensitive arithmetic coding [8.4] is used and an initial low-resolution version of the image can be gradually enhanced with additional compressed data.
JBIG2	ISO/IEC/ ITU-T	A follow-on to JBIG1 for bi-level images in desktop, Internet, and FAX applications. The compression method used is content based, with dictionary-based methods [8.7] for text and halftone regions, and Huffman [8.2] or arithmetic coding [8.4] for other image content. It can be lossy or lossless.
<i>Continuous-Tone Still Images</i>		
JPEG	ISO/IEC/ ITU-T	A <i>Joint Photographic Experts Group</i> standard for images of photographic quality. Its lossy baseline coding system (most commonly implemented) uses quantized discrete cosine transforms (DCT) on image blocks [8.9], Huffman [8.2], and run-length [8.6] coding. It is one of the most popular methods for compressing images on the Internet.
JPEG-LS	ISO/IEC/ ITU-T	A lossless to near-lossless standard for continuous-tone images based on adaptive prediction [8.10], context modeling [8.4], and Golomb coding [8.3].
JPEG-2000	ISO/IEC/ ITU-T	A follow-on to JPEG for increased compression of photographic quality images. Arithmetic coding [8.4] and quantized discrete wavelet transforms (DWT) [8.11] are used. The compression can be lossy or lossless.

TABLE 8.4

Internationally sanctioned video compression standards. The numbers in brackets refer to sections in this chapter.

Name	Organization	Description
DV	IEC	<i>Digital Video</i> . A video standard tailored to home and semiprofessional video production applications and equipment, such as electronic news gathering and camcorders. Frames are compressed independently for uncomplicated editing using a DCT-based approach [8.9] similar to JPEG.
H.261	ITU-T	A two-way videoconferencing standard for ISDN (<i>integrated services digital network</i>) lines. It supports non-interlaced 352×288 and 176×144 resolution images, called CIF (<i>Common Intermediate Format</i>) and QCIF (<i>Quarter CIF</i>), respectively. A DCT-based compression approach [8.9] similar to JPEG is used, with frame-to-frame prediction differencing [8.10] to reduce temporal redundancy. A block-based technique is used to compensate for motion between frames.
H.262	ITU-T	See MPEG-2 below.
H.263	ITU-T	An enhanced version of H.261 designed for ordinary telephone modems (i.e., 28.8 Kb/s) with additional resolutions: SQCIF (<i>Sub-Quarter CIF</i> 128×96), 4CIF (704×576) and 16CIF (1408×512).
H.264	ITU-T	An extension of H.261–H.263 for videoconferencing, streaming, and television. It supports prediction differences within frames [8.10], variable block size integer transforms (rather than the DCT), and context adaptive arithmetic coding [8.4].
H.265 MPEG-H HEVC	ISO/IEC ITU-T	<i>High Efficiency Video Coding</i> (HVEC). An extension of H.264 that includes support for macroblock sizes up to 64×64 and additional intraframe prediction modes, both useful in 4K video applications.
MPEG-1	ISO/IEC	A <i>Motion Pictures Expert Group</i> standard for CD-ROM applications with non-interlaced video at up to 1.5 Mb/s. It is similar to H.261 but frame predictions can be based on the previous frame, next frame, or an interpolation of both. It is supported by almost all computers and DVD players.
MPEG-2	ISO/IEC	An extension of MPEG-1 designed for DVDs with transfer rates at up to 15 Mb/s. Supports interlaced video and HDTV. It is the most successful video standard to date.
MPEG-4	ISO/IEC	An extension of MPEG-2 that supports variable block sizes and prediction differencing [8.10] within frames.
MPEG-4 AVC	ISO/IEC	MPEG-4 Part 10 <i>Advanced Video Coding</i> (AVC). Identical to H.264.

8.2 HUFFMAN CODING

With reference to Tables 8.3–8.5, Huffman codes are used in

- CCITT
- JBIG2
- JPEG
- MPEG-1, 2, 4
- H.261, H.262,
- H.263, H.264

and other compression standards.

One of the most popular techniques for removing coding redundancy is due to Huffman (Huffman [1952]). When coding the symbols of an information source individually, *Huffman coding* yields the smallest possible number of code symbols per source symbol. In terms of Shannon's first theorem (see Section 8.1), the resulting code is optimal for a fixed value of n , subject to the constraint that the source symbols be coded *one at a time*. In practice, the source symbols may be either the intensities of an image or the output of an intensity mapping operation (pixel differences, run lengths, and so on).

TABLE 8.5

Popular image and video compression standards, file formats, and containers not included in Tables 8.3 and 8.4. The numbers in brackets refer to sections in this chapter.

Name	Organization	Description
<i>Continuous-Tone Still Images</i>		
BMP	Microsoft	<i>Windows Bitmap</i> . A file format used mainly for simple uncompressed images.
GIF	CompuServe	<i>Graphic Interchange Format</i> . A file format that uses lossless LZW coding [8.5] for 1- through 8-bit images. It is frequently used to make small animations and short low-resolution films for the Internet.
PDF	Adobe Systems	<i>Portable Document Format</i> . A format for representing 2-D documents in a device and resolution independent way. It can function as a container for JPEG, JPEG-2000, CCITT, and other compressed images. Some PDF versions have become ISO standards.
PNG	World Wide Web Consortium (W3C)	<i>Portable Network Graphics</i> . A file format that losslessly compresses full color images with transparency (up to 48 bits/pixel) by coding the difference between each pixel's value and a predicted value based on past pixels [8.10].
TIFF	Aldus	<i>Tagged Image File Format</i> . A flexible file format supporting a variety of image compression standards, including JPEG, JPEG-LS, JPEG-2000, JBIG2, and others.
WebP	Google	<i>WebP</i> supports lossy compression via WebP VP8 intraframe video compression (see below) and lossless compression using spatial prediction [8.10] and a variant of LZW backward referencing [8.5] and Huffman entropy coding [8.2]. Transparency is also supported.
<i>Video</i>		
AVS	MII	<i>Audio-Video Standard</i> . Similar to H.264 but uses exponential Golomb coding [8.3]. Developed in China.
HDV	Company consortium	<i>High Definition Video</i> . An extension of DV for HD television that uses compression similar to MPEG-2, including temporal redundancy removal by prediction differencing [8.10].
M-JPEG	Various companies	<i>Motion JPEG</i> . A compression format in which each frame is compressed independently using JPEG.
Quick-Time	Apple Computer	A media container supporting DV, H.261, H.262, H.264, MPEG-1, MPEG-2, MPEG-4, and other video compression formats.
VC-1 WMV9	SMPTE Microsoft	The most used video format on the Internet. Adopted for HD and <i>Blu-ray</i> high-definition DVDs. It is similar to H.264/AVC, using an integer DCT with varying block sizes [8.9 and 8.10] and context-dependent variable-length code tables [8.2], but no predictions within frames.
WebP VP8	Google	A file format based on block transform coding [8.9] prediction differences within frames and between frames [8.10]. The differences are entropy encoded using an adaptive arithmetic coder [8.4].

The first step in Huffman's approach is to create a series of source reductions by ordering the probabilities of the symbols under consideration, then combining the lowest probability symbols into a single symbol that replaces them in the next source reduction. Figure 8.7 illustrates this process for binary coding (K -ary Huffman codes also can be constructed). At the far left, a hypothetical set of source symbols and their probabilities are ordered from top to bottom in terms of decreasing probability values. To form the first source reduction, the bottom two probabilities, 0.06 and 0.04, are combined to form a "compound symbol" with probability 0.1. This compound symbol and its associated probability are placed in the first source reduction column so that the probabilities of the reduced source also are ordered from the most to the least probable. This process is then repeated until a reduced source with two symbols (at the far right) is reached.

The second step in Huffman's procedure is to code each reduced source, starting with the smallest source and working back to the original source. The minimal length binary code for a two-symbol source, of course, are the symbols 0 and 1. As Fig. 8.8 shows, these symbols are assigned to the two symbols on the right. (The assignment is arbitrary; reversing the order of the 0 and 1 would work just as well.) As the reduced source symbol with probability 0.6 was generated by combining two symbols in the reduced source to its left, the 0 used to code it is now assigned to both of these symbols, and a 0 and 1 are arbitrarily appended to each to distinguish them from each other. This operation is then repeated for each reduced source until the original source is reached. The final code appears at the far left in Fig. 8.8. The average length of this code is

$$\begin{aligned} L_{\text{avg}} &= (0.4)(1) + (0.3)(2) + (0.1)(3) + (0.1)(4) + (0.06)(5) + (0.04)(5) \\ &= 2.2 \text{ bits/pixel} \end{aligned}$$

and the entropy of the source is 2.14 bits/symbol.

Huffman's procedure creates the optimal code for a set of symbols and probabilities *subject to the constraint* that the symbols be coded one at a time. After the code has been created, coding and/or error-free decoding is accomplished in a simple lookup table manner. The code itself is an instantaneous uniquely decodable block code. It is called a *block code* because each source symbol is mapped into a fixed sequence of code symbols. It is *instantaneous* because each code word in a string of

FIGURE 8.7
Huffman source reductions.

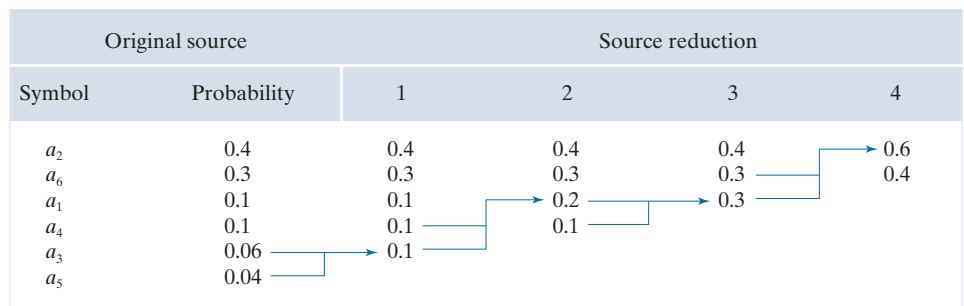


FIGURE 8.8
Huffman code
assignment
procedure.

Original source			Source reduction							
Symbol	Probability	Code	1		2		3		4	
a_2	0.4	1	0.4	1	0.4	1	0.4	1	0.6	0
a_6	0.3	00	0.3	00	0.3	00	0.3	00	0.4	1
a_1	0.1	011	0.1	011	0.2	010	0.3	01		
a_4	0.1	0100	0.1	0100	0.1	011				
a_3	0.06	01010	0.1	0101						
a_5	0.04	01011								

code symbols can be decoded without referencing succeeding symbols. It is *uniquely decodable* because any string of code symbols can be decoded in only one way. Thus, any string of Huffman encoded symbols can be decoded by examining the individual symbols of the string in a left-to-right manner. For the binary code of Fig. 8.8, a left-to-right scan of the encoded string 010100111100 reveals that the first valid code word is 01010, which is the code for symbol a_3 . The next valid code is 011, which corresponds to symbol a_1 . Continuing in this manner reveals the completely decoded message to be $a_3 a_1 a_2 a_2 a_6$.

EXAMPLE 8.4: Huffman Coding.

The 512×512 8-bit monochrome image in Fig. 8.9(a) has the intensity histogram shown in Fig. 8.9(b). Because the intensities are not equally probable, a MATLAB implementation of Huffman’s procedure was used to encode them with 7.428 bits/pixel, including the Huffman code table that is required to reconstruct the original 8-bit image intensities. The compressed representation exceeds the estimated entropy of the image [7.3838 bits/pixel from Eq. (8-7)] by $512^2 \times (7.428 - 7.3838)$ or 11,587 bits—about 0.6%. The resulting compression ratio and corresponding relative redundancy are $C = 8/7.428 = 1.077$, and $R = 1 - (1/1.077) = 0.0715$, respectively. Thus 7.15% of the original 8-bit fixed-length intensity representation was removed as coding redundancy.

When a large number of symbols is to be coded, the construction of an optimal Huffman code is a nontrivial task. For the general case of J source symbols, J symbol probabilities, $J - 2$ source reductions, and $J - 2$ code assignments are required. When source symbol probabilities can be estimated in advance, “near optimal” coding can be achieved with pre-computed Huffman codes. Several popular image compression standards, including the JPEG and MPEG standards discussed in Sections 8.9 and 8.10, specify default Huffman coding tables that have been pre-computed based on experimental data.

8.3 GOLOMB CODING

In this section, we consider the coding of nonnegative integer inputs with exponentially decaying probability distributions. Inputs of this type can be optimally encoded (in the sense of Shannon’s first theorem) using a family of codes that are computationally simpler than Huffman codes. The codes themselves were first proposed for the representation of nonnegative run lengths (Golomb [1966]). In the discussion

With reference to
Tables 8.3–8.5, Golomb
codes are used in

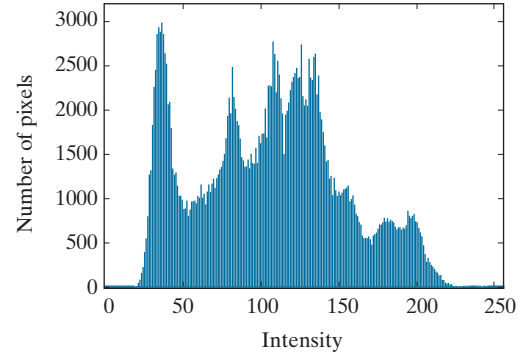
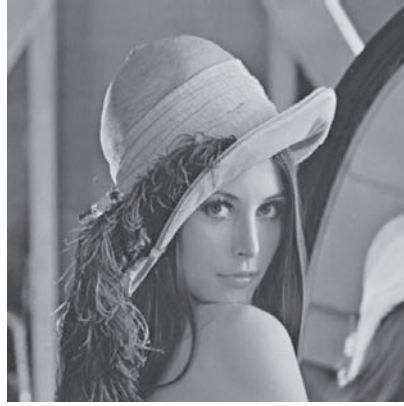
- JPEG-LS
- AVS

compression.

a b

FIGURE 8.9

(a) A 512×512 8-bit image and
(b) its histogram.



that follows, the notation $\lfloor x \rfloor$ denotes the largest integer less than or equal to x , $\lceil x \rceil$ means the smallest integer greater than or equal to x , and $x \bmod y$ is the remainder of x divided by y .

Given a nonnegative integer n and a positive integer *divisor* $m > 0$, the *Golomb code* of n with respect to m , denoted $G_m(n)$, is a combination of the unary code of *quotient* $\lfloor n/m \rfloor$ and the binary representation of *remainder* $n \bmod m$. $G_m(n)$ is constructed as follows:

1. Form the unary code of quotient $\lfloor n/m \rfloor$. (The *unary code* of an integer q is defined as q 1's followed by a 0.)
2. Let $k = \lceil \log_2 m \rceil$, $c = 2^k - m$, $r = n \bmod m$, and compute truncated remainder r' such that

$$r' = \begin{cases} r \text{ truncated to } k-1 \text{ bits} & 0 \leq r < c \\ r + c \text{ truncated to } k \text{ bits} & \text{otherwise} \end{cases} \quad (8-12)$$

3. Concatenate the results of Steps 1 and 2.

To compute $G_4(9)$, for example, begin by determining the unary code of the quotient $\lfloor 9/4 \rfloor = \lfloor 2.25 \rfloor = 2$, which is 110 (the result of Step 1). Then let $k = \lceil \log_2 4 \rceil = 2$, $c = 2^2 - 4 = 0$, and $r = 9 \bmod 4$, which in binary is $1001 \bmod 0100$ or 0001. In accordance with Eq. (8-12), r' is then r (i.e., 0001) truncated to 2 bits, which is 01 (the result of Step 2). Finally, concatenate 110 from Step 1 and 01 from Step 2 to get 11001, which is $G_4(9)$.

For the special case of $m = 2^k$, $c = 0$ and $r' = r = n \bmod m$ truncated to k bits in Eq. (8-12) for all n . The divisions required to generate the resulting Golomb codes become binary shift operations, and the computationally simpler codes are called *Golomb-Rice* or *Rice codes* (Rice [1975]). Columns 2, 3, and 4 of Table 8.6 list the G_1 , G_2 , and G_4 codes of the first ten nonnegative integers. Because m is a power of 2 in each case (i.e., $1 = 2^0$, $2 = 2^1$, and $4 = 2^2$), they are the first three Golomb-Rice codes as well. Moreover, G_1 is the unary code of the nonnegative integers because $\lfloor n/1 \rfloor = n$ and $n \bmod 1 = 0$ for all n .

TABLE 8.6
Several Golomb
codes for the
integers 0–9.

n	$G_1(n)$	$G_2(n)$	$G_4(n)$	$G_{\text{exp}}^0(n)$
0	0	00	000	0
1	10	01	001	100
2	110	100	010	101
3	1110	101	011	11000
4	11110	1100	1000	11001
5	111110	1101	1001	11010
6	1111110	11100	1010	11011
7	11111110	11101	1011	1110000
8	111111110	111100	11000	1110001
9	1111111110	111101	11001	1110010

Keeping in mind that Golomb codes only can be used to represent nonnegative integers, and that there are many Golomb codes to choose from, a key step in their effective application is the selection of divisor m . When the integers to be represented are *geometrically* distributed with a *probability mass function* (PMF)[†]

$$P(n) = (1 - \rho)\rho^n \quad (8-13)$$

for some $0 < \rho < 1$, Golomb codes can be shown to be optimal in the sense that $G_m(n)$ provides the shortest average code length of all uniquely decipherable codes (Gallager and Voorhis [1975]) when

$$m = \left\lceil \frac{\log_2(1 + \rho)}{\log_2(1/\rho)} \right\rceil \quad (8-14)$$

Figure 8.10(a) plots Eq. (8-13) for three values of ρ and graphically illustrates the symbol probabilities that Golomb codes handle well (that is, code efficiently). As is shown in the figure, small integers are much more probable than large ones.

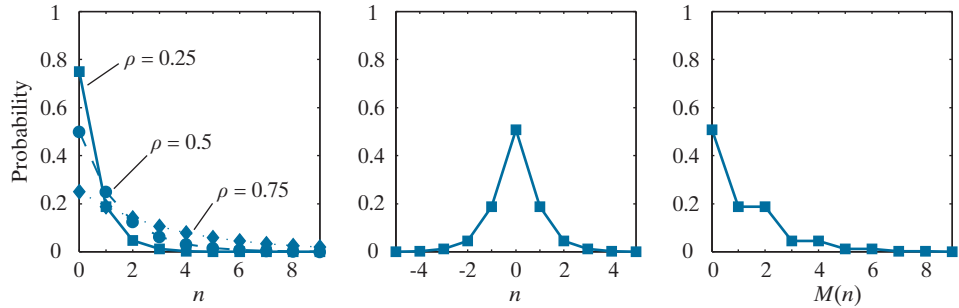
Because the probabilities of the intensities in an image [see, for example, the histogram of Fig. 8.9(b)] are unlikely to match the probabilities specified in Eq. (8-13) and shown in Fig. 8.10(a), Golomb codes are seldom used for the coding of intensities. When intensity differences are to be coded, however, the probabilities of the resulting “difference values” (see Section 8.10) (with the notable exception of the negative differences) often resemble those of Eq. (8-13) and Fig. 8.10(a). To handle negative differences in Golomb coding, which can only represent nonnegative integers, a mapping like

[†]A *probability mass function* (PMF) is a function that defines the probability that a discrete random variable is exactly equal to some value. A PMF differs from a PDF in that a PDF’s values are not probabilities; rather, the integral of a PDF over a specified interval is a probability.

a b c

FIGURE 8.10

(a) Three one-sided geometric distributions from Eq. (8-13); (b) a two-sided exponentially decaying distribution; and (c) a reordered version of (b) using Eq. (8-15).



$$M(n) = \begin{cases} 2n & n \geq 0 \\ 2|n| - 1 & n < 0 \end{cases} \quad (8-15)$$

is typically used. Using this mapping, for example, the two-sided PMF shown in Fig. 8.10(b) can be transformed into the one-sided PMF in Fig. 8.10(c). Its integers are reordered, alternating the negative and positive integers so the negative integers are mapped into the odd positive integer positions. If $P(n)$ is two-sided and centered at zero, $P(M(n))$ will be one-sided. The mapped integers, $M(n)$, can then be efficiently encoded using an appropriate Golomb-Rice code (Weinberger et al. [1996]).

EXAMPLE 8.5: Golomb-Rice coding.

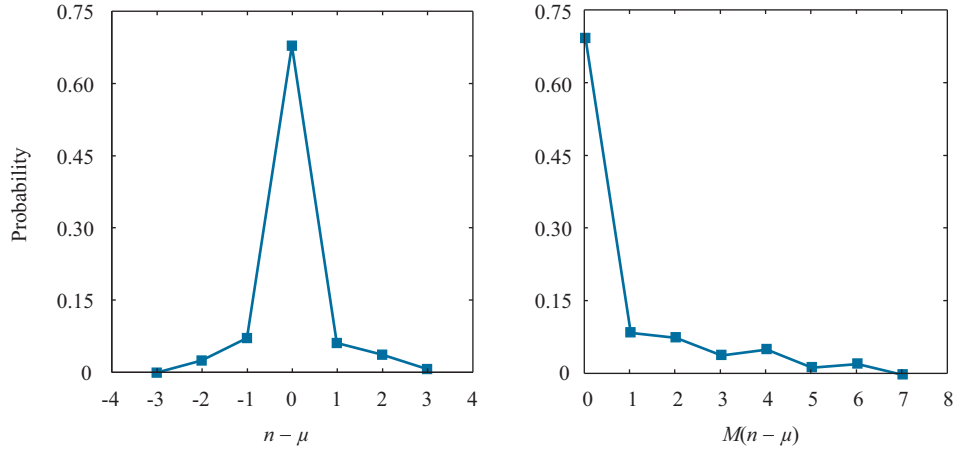
Consider again the image from Fig. 8.1(c) and note that its histogram [see Fig. 8.3(a)] is similar to the two-sided distribution in Fig. 8.10(b) above. If we let n be some nonnegative integer intensity in the image, where $0 \leq n \leq 225$, and μ be the mean intensity, $P(n - \mu)$ is the two-sided distribution shown in Fig. 8.11(a). This plot was generated by normalizing the histogram in Fig. 8.3(a) by the total number of pixels in the image and shifting the normalized values to the left by 128 (which in effect subtracts the mean intensity from the image). In accordance with Eq. (8-15), $P(M(n - \mu))$ is then the one-sided distribution shown in Fig. 8.11(b). If the reordered intensity values are Golomb coded using a MATLAB implementation of code G_1 in column 2 of Table 8.6, the encoded representation is 4.5 times smaller than the original image (i.e., $C = 4.5$). The G_1 code realizes 4.5/5.1, or 88% of the theoretical compression possible with variable-length coding. [Based on the entropy calculated in Example (8-2), the maximum possible compression ratio through variable-length coding is $C = 8/1.566 \approx 5.1$.] Moreover, Golomb coding achieves 96% of the compression provided by a MATLAB implementation of Huffman's approach, and doesn't require the computation of a custom Huffman coding table.

Now consider the image in Fig. 8.9(a). If its intensities are Golomb coded using the same G_1 code as above, $C = 0.0922$. That is, there is *data expansion*. This is due to the fact that the probabilities of the intensities of the image in Fig. 8.9(a) are much different than the probabilities defined in Eq. (8-13). In a similar manner, Huffman codes can produce data expansion when used to encode symbols whose probabilities are different from those for which the code was computed. In practice, the further you depart from the input probability assumptions for which a code is designed, the greater the risk of poor compression performance and data expansion.

a b

FIGURE 8.11

(a) The probability distribution of the image in Fig. 8.1(c) after subtracting the mean intensity from each pixel.
 (b) A mapped version of (a) using Eq. (8-15).



To conclude our coverage of Golomb codes, we note that Column 5 of Table 8.6 contains the first 10 codes of the zeroth-order *exponential Golomb code*, denoted $G_{\text{exp}}^0(n)$. Exponential-Golomb codes are useful for the encoding of run lengths, because both short and long runs are encoded efficiently. An order- k exponential-Golomb code $G_{\text{exp}}^k(n)$ is computed as follows:

1. Find an integer $i \geq 0$ such that

$$\sum_{j=0}^{i-1} 2^{j+k} \leq n < \sum_{j=0}^i 2^{j+k} \quad (8-16)$$

and form the unary code of i . If $k = 0$, $i = \lfloor \log_2(n+1) \rfloor$ and the code is also known as the *Elias gamma code*.

2. Truncate the binary representation of

$$n - \sum_{j=0}^{i-1} 2^{j+k} \quad (8-17)$$

to $k + i$ least significant bits.

3. Concatenate the results of Steps 1 and 2.

To find $G_{\text{exp}}^0(8)$, for example, we let $i = \lfloor \log_2 9 \rfloor$ or 3 in Step 1 because $k = 0$. Equation (8-16) is then satisfied because

$$\begin{aligned} \sum_{j=0}^{3-1} 2^{j+0} &\leq 8 < \sum_{j=0}^3 2^{j+0} \\ \sum_{j=0}^2 2^j &\leq 8 < \sum_{j=0}^3 2^j \end{aligned}$$

$$2^0 + 2^1 + 2^2 \leq 8 < 2^0 + 2^1 + 2^2 + 2^3$$

$$7 \leq 8 < 15$$

The unary code of 3 is 1110 and Eq. (8-17) of Step 2 yields

$$8 - \sum_{j=0}^{3-1} 2^{j+0} = 8 - \sum_{j=0}^2 2^j = 8 - (2^0 + 2^1 + 2^2) = 8 - 7 = 1 = 0001$$

which when truncated to its 3 + 0 least significant bits becomes 001. The concatenation of the results from Steps 1 and 2 then yields 1110001. Note that this is the entry in column 4 of Table 8.6 for $n = 8$. Finally, we note that like the Huffman codes of the last section, the Golomb codes of Table 8.6 are variable-length, instantaneous, and uniquely decodable block codes.

8.4 ARITHMETIC CODING

With reference to Tables 8.3–8.5, arithmetic coding is used in

- JBIG1
- JBIG2
- JPEG-2000
- H.264
- MPEG-4 AVC

and other compression standards.

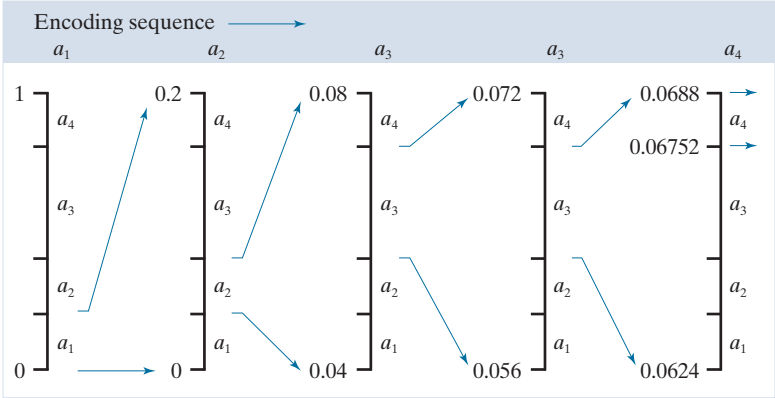
Unlike the variable-length codes of the previous two sections, *arithmetic coding* generates nonblock codes. In arithmetic coding, which can be traced to the work of Elias (Abramson [1963]), a one-to-one correspondence between source symbols and code words does not exist. Instead, an entire sequence of source symbols (or message) is assigned a single arithmetic code word. The code word itself defines an interval of real numbers between 0 and 1. As the number of symbols in the message increases, the interval used to represent it becomes smaller, and the number of information units (say, bits) required to represent the interval becomes larger. Each symbol of the message reduces the size of the interval in accordance with its probability of occurrence. Because the technique does not require, as does Huffman's approach, that each source symbol translate into an integral number of code symbols (that is, that the symbols be coded one at a time), it achieves (but only in theory) the bound established by Shannon's first theorem of Section 8.1.

Figure 8.12 illustrates the basic arithmetic coding process. Here, a five-symbol sequence or message, $a_1a_2a_3a_4$, from a four-symbol source is coded. At the start of the coding process, the message is assumed to occupy the entire half-open interval $[0, 1)$. As Table 8.7 shows, this interval is subdivided initially into four regions based on the probabilities of each source symbol. Symbol a_1 , for example, is associated with subinterval $[0, 0.2)$. Because it is the first symbol of the message being coded, the message interval is initially narrowed to $[0, 0.2)$. Thus, in Fig. 8.12, $[0, 0.2)$ is expanded to the full height of the figure, and its end points labeled by the values of the narrowed range. The narrowed range is then subdivided in accordance with the original

TABLE 8.7
Arithmetic coding example.

Source Symbol	Probability	Initial Subinterval
a_1	0.2	$[0.0, 0.2)$
a_2	0.2	$[0.2, 0.4)$
a_3	0.4	$[0.4, 0.8)$
a_4	0.2	$[0.8, 1.0)$

FIGURE 8.12
Arithmetic coding
procedure.



source symbol probabilities, and the process continues with the next message symbol. In this manner, symbol a_2 narrows the subinterval to $[0.04, 0.08)$, a_3 further narrows it to $[0.056, 0.072)$, and so on. The final message symbol, which must be reserved as a special end-of-message indicator, narrows the range to $[0.06752, 0.0688)$. Of course, any number within this subinterval, for example, 0.068, can be used to represent the message. In the arithmetically coded message of Fig. 8.12, three decimal digits are used to represent the five-symbol message. This translates into 0.6 decimal digits per source symbol and compares favorably with the entropy of the source, which, from Eq. 8.6, is 0.58 decimal digits per source symbol. As the length of the sequence being coded increases, the resulting arithmetic code approaches the bound established by Shannon's first theorem. In practice, two factors cause coding performance to fall short of the bound: (1) the addition of the end-of-message indicator that is needed to separate one message from another, and (2) the use of finite precision arithmetic. Practical implementations of arithmetic coding address the latter problem by introducing a scaling strategy and a rounding strategy (Langdon and Rissanen [1981]). The scaling strategy renormalizes each subinterval to the $[0, 1)$ range before subdividing it in accordance with the symbol probabilities. The rounding strategy guarantees that the truncations associated with finite precision arithmetic do not prevent the coding subintervals from being accurately represented.

ADAPTIVE CONTEXT DEPENDENT PROBABILITY ESTIMATES

With accurate input symbol *probability models*, that is, models that provide the true probabilities of the symbols being coded, arithmetic coders are near optimal in the sense of minimizing the average number of code symbols required to represent the symbols being coded. As in both Huffman and Golomb coding, however, inaccurate probability models can lead to non-optimal results. A simple way to improve the accuracy of the probabilities employed is to use an adaptive, context dependent probability model. *Adaptive* probability models update symbol probabilities as symbols are coded or become known. Thus, the probabilities adapt to the local statistics of the symbols being coded. *Context-dependent* models provide probabilities

that are based on a predefined neighborhood of pixels, called the *context*, around the symbols being coded. Normally, a *causal context* (one limited to symbols that have already been coded) is used. Both the Q-coder (Pennebaker et al. [1988]) and MQ-coder (ISO/IEC [2000]), two well-known arithmetic coding techniques that have been incorporated into the JBIG, JPEG-2000, and other important image compression standards, use probability models that are both adaptive and context dependent. The Q-coder dynamically updates symbol probabilities during the interval renormalizations that are part of the arithmetic coding process. Adaptive context dependent models also have been used in Golomb coding, for example, in the JPEG-LS compression standard.

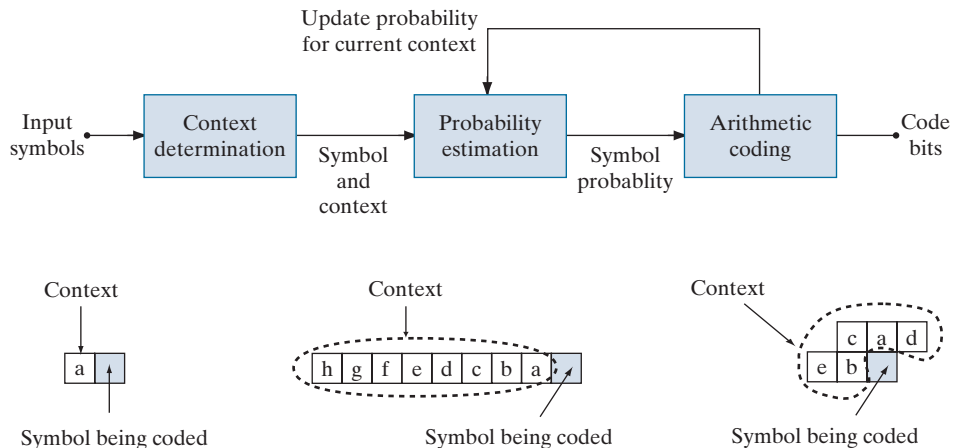
Figure 8.13(a) diagrams the steps involved in adaptive, context-dependent arithmetic coding of *binary* source symbols. Arithmetic coding often is used when binary symbols are to be coded. As each symbol (or bit) begins the coding process, its context is formed in the *Context determination* block of Fig. 8.13(a). Figures 8.13(b) through (d) show three possible contexts that can be used: (1) the immediately preceding symbol, (2) a group of preceding symbols, and (3) some number of preceding symbols plus symbols on the previous scan line. For the three cases shown, the *Probability estimation* block must manage 2^1 (or 2), 2^8 (or 256), and 2^5 (or 32) contexts and their associated probabilities. For instance, if the context in Fig. 8.13(b) is used, conditional probabilities $P(0|a=0)$ (the probability that the symbol being coded is a 0 given that the preceding symbol is a 0), $P(1|a=0)$, $P(0|a=1)$, and $P(1|a=1)$ must be tracked. The appropriate probabilities are then passed to the *Arithmetic coding* block as a function of the current context, and drive the generation of the arithmetically coded output sequence in accordance with the process illustrated in Fig. 8.12. The probabilities associated with the context involved in the current coding step are then updated to reflect the fact that another symbol within that context has been processed.

Finally, we note that a variety of arithmetic coding techniques are protected by United States patents (and may be protected in other jurisdictions as well). Because



FIGURE 8.13

(a) An adaptive, context-based arithmetic coding approach (often used for binary source symbols). (b)–(d) Three possible context models.



of these patents, and the possibility of unfavorable monetary judgments for their infringement, most implementations of the JPEG compression standard, which contains options for both Huffman and arithmetic coding, typically support Huffman coding alone.

8.5 LZW CODING

With reference to Tables 8.3–8.5, LZW coding is used in the

- GIF
- TIFF
- PDF

formats, but not in any of the internationally sanctioned compression standards.

The techniques covered in Sections 8.2 through 8.4 are focused on the removal of coding redundancy. In this section, we consider an error-free compression approach that also addresses spatial redundancies in an image. The technique, called *Lempel-Ziv-Welch (LZW) coding*, assigns fixed-length code words to variable length sequences of source symbols. Recall from the earlier section on measuring image information that Shannon used the idea of coding sequences of source symbols, rather than individual source symbols, in the proof of his first theorem. A key feature of LZW coding is that it requires no a priori knowledge of the probability of occurrence of the symbols to be encoded. Despite the fact that until recently it was protected under a United States patent, LZW compression has been integrated into a variety of mainstream imaging file formats, including GIF, TIFF, and PDF. The PNG format was created to get around LZW licensing requirements.

EXAMPLE 8.6: LZW coding Fig. 8.9(a).

Consider again the 512×512 , 8-bit image from Fig. 8.9(a). Using Adobe Photoshop, an uncompressed TIFF version of this image requires 286,740 bytes of disk space—262,144 bytes for the 512×512 8-bit pixels plus 24,596 bytes of overhead. Using TIFF's LZW compression option, however, the resulting file is 224,420 bytes. The compression ratio is $C = 1.28$. Recall that for the Huffman encoded representation of Fig. 8.9(a) in Example 8.4, $C = 1.077$. The additional compression realized by the LZW approach is due the removal of some of the image's spatial redundancy.

LZW coding is conceptually very simple (Welch [1984]). At the onset of the coding process, a codebook or *dictionary* containing the source symbols to be coded is constructed. For 8-bit monochrome images, the first 256 words of the dictionary are assigned to intensities 0, 1, 2, ..., 255. As the encoder sequentially examines image pixels, intensity sequences that are not in the dictionary are placed in algorithmically determined (e.g., the next unused) locations. If the first two pixels of the image are white, for instance, sequence "255–255" might be assigned to location 256, the address following the locations reserved for intensity levels 0 through 255. The next time two consecutive white pixels are encountered, code word 256, the address of the location containing sequence 255–255, is used to represent them. If a 9-bit, 512-word dictionary is employed in the coding process, the original $(8 + 8)$ bits that were used to represent the two pixels are replaced by a single 9-bit code word. Clearly, the size of the dictionary is an important system parameter. If it is too small, the detection of matching intensity-level sequences will be less likely; if it is too large, the size of the code words will adversely affect compression performance.

EXAMPLE 8.7: LZW coding.

Consider the following 4×4 , 8-bit image of a vertical edge:

```

39 39 126 126
39 39 126 126
39 39 126 126
39 39 126 126

```

Table 8.8 details the steps involved in coding its 16 pixels. A 512-word dictionary with the following starting content is assumed:

Dictionary Location	Entry
0	0
1	1
⋮	⋮
255	255
256	—
⋮	⋮
511	—

Locations 256 through 511 initially are unused.

The image is encoded by processing its pixels in a left-to-right, top-to-bottom manner. Each successive intensity value is concatenated with a variable, column 1 of Table 8.8, called the “currently recognized sequence.” As can be seen, this variable is initially null or empty. The dictionary is searched for each concatenated sequence and if found, as was the case in the first row of the table, is replaced by the newly concatenated and recognized (i.e., located in the dictionary) sequence. This was done in column 1 of row 2. No output codes are generated, nor is the dictionary altered. If the concatenated sequence is not found, however, the address of the currently recognized sequence is output as the next encoded value, the concatenated but unrecognized sequence is added to the dictionary, and the currently recognized sequence is initialized to the current pixel value. This occurred in row 2 of the table. The last two columns detail the intensity sequences that are added to the dictionary when scanning the entire 128-bit image. Nine additional code words are defined. At the conclusion of coding, the dictionary contains 265 code words and the LZW algorithm has successfully identified several repeating intensity sequences—leveraging them to reduce the original 128-bit image to 90 bits (i.e., 10 9-bit codes). The encoded output is obtained by reading the third column from top to bottom. The resulting compression ratio is 1.42:1.

A unique feature of the LZW coding just demonstrated is that the coding dictionary or code book is created while the data are being encoded. Remarkably, an LZW decoder builds an identical decompression dictionary as it simultaneously decodes the encoded data stream. It is left as an exercise to the reader (see Problem 8.20) to decode the output of the preceding example and reconstruct the code book. Although not needed in this example, most practical applications require a

TABLE 8.8
LZW Coding
example.

Currently Recognized Sequence	Pixel Being Processed	Encoded Output	Dictionary Location (Code Word)	Dictionary Entry
	39			
39	39	39	256	39–39
39	126	39	257	39–126
126	126	126	258	126–126
126	39	126	259	126–39
39	39			
39–39	126	256	260	39–39–126
126	126			
126–126	39	258	261	126–126–39
39	39			
39–39	126			
39–39–126	126	260	262	39–39–126–126
126	39			
126–39	39	259	263	126–39–39
39	126			
39–126	126	257	264	39–126–126
126		126		

strategy for handling dictionary overflow. A simple solution is to flush or reinitialize the dictionary when it becomes full and continue coding with a new initialized dictionary. A more complex option is to monitor compression performance and flush the dictionary when it becomes poor or unacceptable. Alternatively, the least used dictionary entries can be tracked and replaced when necessary.

8.6 RUN-LENGTH CODING

As was noted earlier, images with repeating intensities along their rows (or columns) can often be compressed by representing runs of identical intensities as *run-length pairs*, where each run-length pair specifies the start of a new intensity and the number of consecutive pixels that have that intensity. The technique, referred to as *run-length encoding* (RLE), was developed in the 1950s and became, along with its 2-D extensions, the standard compression approach in facsimile (FAX) coding. Compression is achieved by eliminating a simple form of spatial redundancy—groups of identical intensities. When there are few (or no) runs of identical pixels, run-length encoding results in data expansion.

With reference to
Tables 8.3–8.5, the coding
of run-lengths is used in

- CCITT
- JBIG2
- JPEG
- M-JPEG
- MPEG-1,2,4
- BMP

and other compres-
sion standards and file
formats.

EXAMPLE 8.8: RLE in the BMP file format.

The BMP file format uses a form of run-length encoding in which image data is represented in two different modes: encoded and absolute. Either mode can occur anywhere in the image. In *encoded* mode, a

two byte RLE representation is used. The first byte specifies the number of consecutive pixels that have the color index contained in the second byte. The 8-bit color index selects the run's intensity (color or gray value) from a table of 256 possible intensities.

In *absolute* mode, the first byte is 0, and the second byte signals one of four possible conditions, as shown in Table 8.9. When the second byte is 0 or 1, the end of a line or the end of the image has been reached. If it is 2, the next two bytes contain unsigned horizontal and vertical offsets to a new spatial position (and pixel) in the image. If the second byte is between 3 and 255, it specifies the number of uncompressed pixels that follow with each subsequent byte containing the color index of one pixel. The total number of bytes must be aligned on a 16-bit word boundary.

An uncompressed BMP file (saved using Photoshop) of the $512 \times 512 \times 8$ bit image shown in Fig. 8.9(a) requires 263,244 bytes of memory. Compressed using BMP's RLE option, the file expands to 267,706 bytes, and the compression ratio is $C = 0.98$. There are not enough equal intensity runs to make run-length compression effective; a small amount of expansion occurs. For the image in Fig. 8.1(c), however, the BMP RLE option results in a compression ratio $C = 1.35$. (Note that due to differences in overhead, the uncompressed BMP file is smaller than the uncompressed TIFF file in Example 8.6.)

Run-length encoding is particularly effective when compressing binary images. Because there are only two possible intensities (black and white), adjacent pixels are more likely to be identical. In addition, each image row can be represented by a sequence of lengths only, rather than length-intensity pairs as was used in Example 8.8. The basic idea is to code each contiguous group (i.e., run) of 0's or 1's encountered in a left-to-right scan of a row by its length *and* to establish a convention for determining the value of the run. The most common conventions are (1) to specify the value of the first run of each row, or (2) to assume that each row begins with a white run, whose run length may in fact be zero.

Although run-length encoding is in itself an effective method of compressing binary images, additional compression can be achieved by variable-length coding the run lengths themselves. The black and white run lengths can be coded separately using variable-length codes that are specifically tailored to their own statistics. For example, letting symbol a_j represent a black run of length j , we can estimate the probability that symbol a_j was emitted by an imaginary black run-length source by dividing the number of black run lengths of length j in the entire image by the total number of black runs. An estimate of the entropy of this black run-length source, denoted as H_0 , follows by substituting these probabilities into Eq. (8-6). A similar argument holds for the entropy of the white runs, denoted as H_1 . The approximate run-length entropy of the image is then

$$H_{\text{RL}} = \frac{H_0 + H_1}{L_0 + L_1} \quad (8-18)$$

where the variables L_0 and L_1 denote the average values of black and white run lengths, respectively. Equation (8-18) provides an estimate of the average number of bits per pixel required to code the run lengths in a binary image using a variable-length code.

TABLE 8.9
BMP absolute coding mode options. In this mode, the first byte of the BMP pair is 0.

Second Byte Value	Condition
0	End of line
1	End of image
2	Move to a new position
3-255	Specify pixels individually

Two of the oldest and most widely used image compression standards are the CCITT Group 3 and 4 standards for binary image compression. Although they have been used in a variety of computer applications, they were originally designed as facsimile (FAX) coding methods for transmitting documents over telephone networks. The Group 3 standard uses a 1-D run-length coding technique in which the last $K - 1$ lines of each group of K lines (for $K = 2$ or 4) can be optionally coded in a 2-D manner. The Group 4 standard is a simplified or streamlined version of the Group 3 standard in which only 2-D coding is allowed. Both standards use the same 2-D coding approach, which is two-dimensional in the sense that information from the previous line is used to encode the current line. Both 1-D and 2-D coding will be discussed next.

ONE-DIMENSIONAL CCITT COMPRESSION

In the 1-D CCITT Group 3 compression standard, each line of an image[†] is encoded as a series of variable-length Huffman code words that represent the run lengths of alternating white and black runs in a left-to-right scan of the line. The compression method employed is commonly referred to as *Modified Huffman* (MH) coding. The code words themselves are of two types, which the standard refers to as *terminating codes* and *makeup codes*. If run length r is less than or equal to 63, a terminating code is used to represent it. The standard specifies different terminating codes for black and white runs. If $r > 63$, two codes are used; a makeup code for quotient $\lfloor r/64 \rfloor \times 64$, and terminating code for remainder $r \bmod 64$. Makeup codes may or may not depend on the intensity (black or white) of the run being coded. If $\lfloor r/64 \rfloor \times 64 \leq 1728$, separate black and white run makeup codes are specified; otherwise, makeup codes are independent of run intensity. The standard requires that each line begin with a white run-length code word, which may in fact be 00110101, the code for a white run of length zero. Finally, a unique end-of-line (EOL) code word 000000000001 is used to terminate each line, as well as to signal the first line of each new image. The end of a sequence of images is indicated by six consecutive EOLs.

TWO-DIMENSIONAL CCITT COMPRESSION

The 2-D compression approach adopted for both the CCITT Group 3 and 4 standards is a line-by-line method in which the position of each black-to-white or white-to-black run transition is coded with respect to the position of a *reference element* a_0 that is situated on the current *coding line*. The previously coded line is called the *reference line*; the reference line for the first line of each new image is an

Consult the book web-site for tables of the MH terminating and makeup codes.

Recall that the notation $\lfloor x \rfloor$ denotes the largest integer less than or equal to x .

[†]In the standard, images are referred to as *pages* and sequences of images are called *documents*.

imaginary white line. The 2-D coding technique that is used is called *Relative Element Address Designate* (READ) coding. In the Group 3 standard, one or three READ coded lines are allowed between successive MH coded lines; this technique is called *Modified READ* (MR) coding. In the Group 4 standard, a greater number of READ coded lines are allowed, and the method is called *Modified Modified READ* (MMR) coding. As was previously noted, the coding is two-dimensional in the sense that information from the previous line is used to encode the current line. Two-dimensional transforms are not involved.

Figure 8.14 shows the basic 2-D coding process for a single scan line. Note that the initial steps of the procedure are directed at locating several key *changing elements*: a_0 , a_1 , a_2 , b_1 , and b_2 . A changing element is defined by the standard as a pixel whose value is different from that of the previous pixel on the same line. The most important changing element is a_0 (the reference element), which is either set to the location of an imaginary white changing element to the left of the first pixel of each new coding line, or determined from the previous coding mode. Coding modes will be discussed in the following paragraph. After a_0 is located, a_1 is identified as the location of the next changing element to the right of a_0 on the current coding line, a_2 as the next changing element to the right of a_1 on the coding line, b_1 as the changing element of the opposite value (of a_0) and to the right of a_0 on the reference (or previous) line, and b_2 as the next changing element to the right of b_1 on the reference line. If any of these changing elements are not detected, they are set to the location of an imaginary pixel to the right of the last pixel on the appropriate line. Figure 8.15 provides two illustrations of the general relationships between the various changing elements.

After identification of the current reference element and associated changing elements, two simple tests are performed to select one of three possible coding modes: *pass mode*, *vertical mode*, or *horizontal mode*. The initial test, which corresponds to the first branch point in the flowchart in Fig. 8.14, compares the location of b_2 to that of a_1 . The second test, which corresponds to the second branch point in Fig. 8.14, computes the distance (in pixels) between the locations of a_1 and b_1 and compares it against 3. Depending on the outcome of these tests, one of the three outlined coding blocks of Fig. 8.14 is entered and the appropriate coding procedure is executed. A new reference element is then established, as per the flowchart, in preparation for the next coding iteration.

Table 8.10 defines the specific codes utilized for each of the three possible coding modes. In pass mode, which specifically excludes the case in which b_2 is directly above a_1 , only the pass mode code word 0001 is needed. As Fig. 8.15(a) shows, this mode identifies white or black reference line runs that do not overlap the current white or black coding line runs. In horizontal coding mode, the distances from a_0 to a_1 and a_1 to a_2 must be coded in accordance with the termination and makeup codes of 1-D CCITT Group 3 compression, then appended to the horizontal mode code word 001. This is indicated in Table 8.10 by the notation $001 + M(a_0a_1) + M(a_1a_2)$, where a_0a_1 and a_1a_2 denote the distances from a_0 to a_1 and a_1 to a_2 , respectively. Finally, in vertical coding mode, one of six special variable-length codes is assigned to the distance between a_1 and b_1 . Figure 8.15(b) illustrates the parameters involved

FIGURE 8.14

CCITT 2-D

READ coding

procedure. The

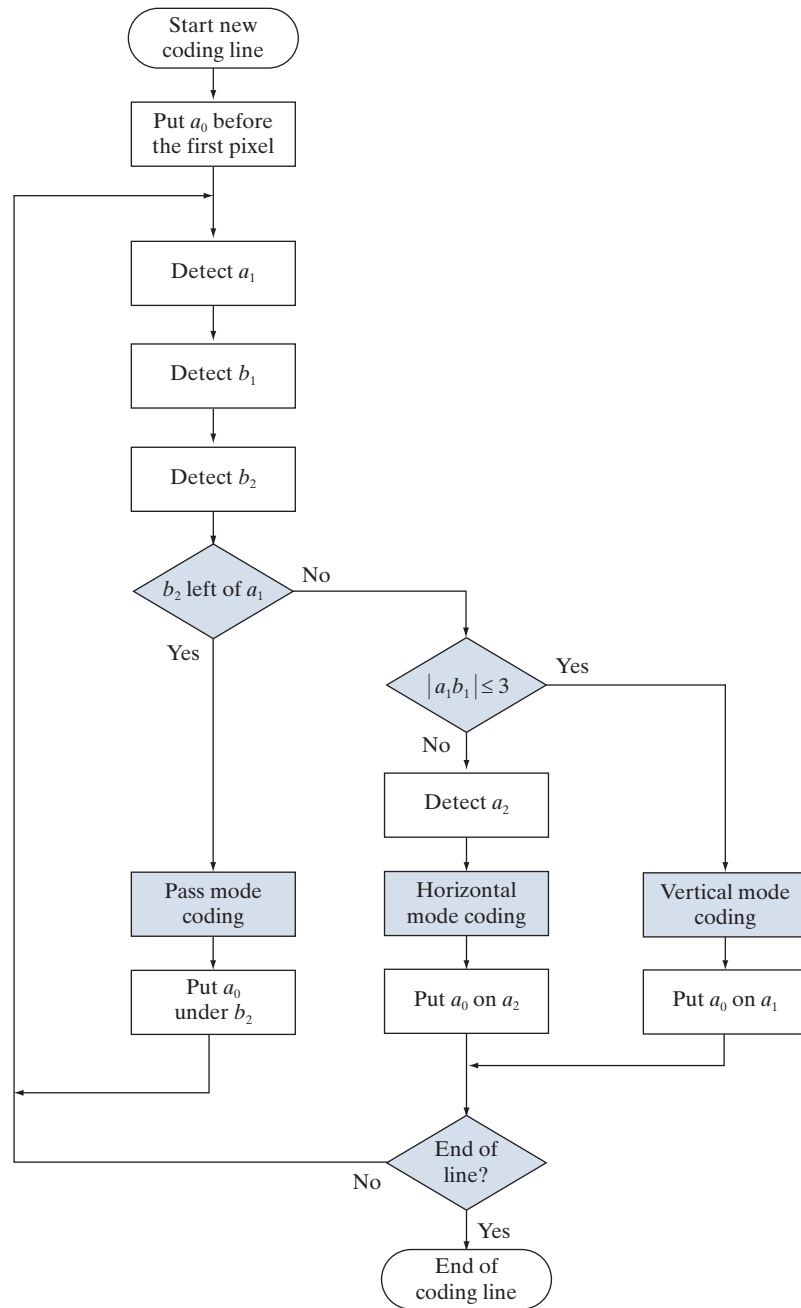
notation $|a_1 b_1|$

denotes the abso-

lute value of the

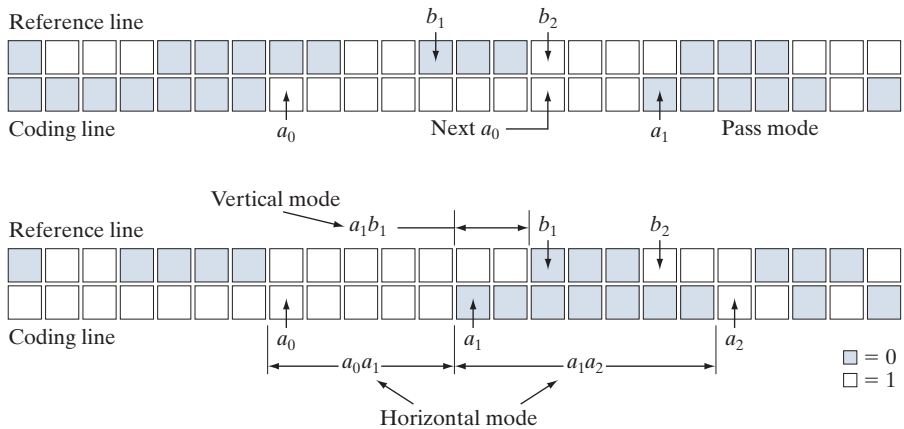
distance between

changing

elements a_1 and b_1 .

a
b

FIGURE 8.15
CCITT (a) pass
mode and
(b) horizontal
and vertical
mode
coding
parameters.



in both horizontal and vertical mode coding. The extension mode code word at the bottom of Table 8.10 is used to enter an optional facsimile coding mode. For example, the 0000001111 code is used to initiate an uncompressed mode of transmission.

EXAMPLE 8.9: CCITT vertical mode coding example.

Although Fig. 8.15(b) is annotated with the parameters for both horizontal and vertical mode coding (to facilitate the discussion above), the depicted pattern of black and white pixels is a case for vertical mode coding. That is, because b_2 is to the right of a_1 , the first (or pass mode) test in Fig. 8.14 fails. The second test, which determines whether the vertical or horizontal coding mode is entered, indicates that vertical mode coding should be used, because the distance from a_1 to b_1 is less than 3. In accordance with Table 8.10, the appropriate code word is 000010, implying that a_1 is two pixels left of b_1 . In preparation for the next coding iteration, a_0 is moved to the location of a_1 .

TABLE 8.10
CCITT
two-dimensional
code table.

Mode	Code Word
Pass	0001
Horizontal	$001 + M(a_0a_1) + M(a_1a_2)$
Vertical	
a_1 below b_1	1
a_1 one to the right of b_1	011
a_1 two to the right of b_1	000011
a_1 three to the right of b_1	0000011
a_1 one to the left of b_1	010
a_1 two to the left of b_1	000010
a_1 three to the left of b_1	0000010
Extension	0000001xxx

EXAMPLE 8.10: CCITT compression example.

Figure 8.16(a) is a 300 dpi scan of a 7 × 9.25 inch book page displayed at about 1/3 scale. Note that about half of the page contains text, around 9% is occupied by a halftone image, and the rest is white space. A section of the page is enlarged in Fig. 8.16(b). Keep in mind that we are dealing with a binary image; the illusion of gray tones is created, as was described in Section 4.5, by the halftoning process used in printing. If the binary pixels of the image in Fig. 8.16(a) are stored in groups of 8 pixels per byte, the 1952 × 2697 bit scanned image, commonly called a document, requires 658,068 bytes. An uncompressed PDF file of the document (created in Photoshop) requires 663,445 bytes. CCITT Group 3 compression reduces the file to 123,497 bytes, resulting in a compression ratio $C = 5.37$. CCITT Group 4 compression reduces the file to 110,456 bytes, increasing the compression ratio to about 6.

With reference to Tables 8.3–8.5, symbol-based coding is used in

- JBIG2 compression.

8.7 SYMBOL-BASED CODING

In *symbol-* or *token-based* coding, an image is represented as a collection of frequently occurring subimages, called *symbols*. Each such symbol is stored in a *symbol dictionary* and the image is coded as a set of triplets $\{(x_1, y_1, t_1), (x_2, y_2, t_2), \dots\}$, where each (x_i, y_i) pair specifies the location of a symbol in the image and *token* t_i is the address of the symbol or subimage in the dictionary. That is, each triplet represents an instance of a dictionary symbol in the image. Storing repeated symbols only once can compress images significantly, particularly in document storage and retrieval applications where the symbols are often character bitmaps that are repeated many times.

a b

FIGURE 8.16
A binary scan of a book page: (a) scaled to show the general page content; (b) scaled to show the binary pixels used in dithering.

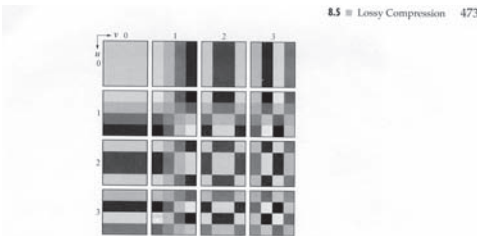


FIGURE 8.30 Discrete-cosine basis functions for $N = 4$. The origin of each block is at its top left.

where

$$\alpha(u) = \begin{cases} \sqrt{\frac{1}{N}} & \text{for } u = 0 \\ \sqrt{\frac{2}{N}} & \text{for } u = 1, 2, \dots, N - 1 \end{cases} \quad (8.5-33)$$

and similarly for $\alpha(v)$. Figure 8.30 shows $g(x, y, u, v)$ for the case $N = 4$. The computation follows the same format as explained for Fig. 8.29, with the difference that the values of g are not integers. In Fig. 8.30, the lighter gray levels correspond to larger values of g .

Figures 8.31(a), (c), and (e) show three approximations of the 512 × 512 monochrome image in Fig. 8.23. These pictures were obtained by dividing the original image into subimages of size 8 × 8, representing each subimage using one of the transforms just described (i.e., the DFT, WHT, or DCT transform), truncating 50% of the resulting coefficients, and taking the inverse transform of the truncated coefficient arrays.

In each case, the 32 retained coefficients were selected on the basis of maximum magnitude. When we disregard any quantization or coding issues, this process amounts to compressing the original image by a factor of 2. Note that in all cases, the 32 discarded coefficients had little visual impact on reconstructed image quality. Their elimination, however, was accompanied by some mean-square error, which can be seen in the scaled error images of Figs. 8.31(b), (d), and (f). The actual rms errors were 1.28, 0.86, and 0.68 gray levels, respectively.

EXAMPLE 8.19: Transform coding with the DFT, WHT, and DCT.



$$r N = 4. Tl$$

Consider the simple bilevel image in Fig. 8.17(a). It contains the single word, *banana*, which is composed of three unique symbols: a *b*, three *a*'s, and two *n*'s. Assuming that the *b* is the first symbol identified in the coding process, its 9×7 bitmap is stored in location 0 of the symbol dictionary. As Fig. 8.17(b) shows, the token identifying the *b* bitmap is 0. Thus, the first triplet in the encoded image's representation [see Fig. 8.17(c)] is (0, 2, 0), indicating that the upper-left corner (an arbitrary convention) of the rectangular bitmap representing the *b* symbol is to be placed at location (0, 2) in the decoded image. After the bitmaps for the *a* and *n* symbols have been identified and added to the dictionary, the remainder of the image can be encoded with five additional triplets. As long as the six triplets required to locate the symbols in the image, together with the three bitmaps required to define them, are smaller than the original image, compression occurs. In this case, the starting image has $9 \times 51 \times 1$ or 459 bits and, assuming that each triplet is composed of three bytes, the compressed representation has $(6 \times 3 \times 8) + [(9 \times 7) + (6 \times 7) + (6 \times 6)]$ or 285 bits; the resulting compression ratio $C = 1.61$. To decode the symbol-based representation in Fig. 8.17(c), you simply read the bitmaps of the symbols specified in the triplets from the symbol dictionary and place them at the spatial coordinates specified in each triplet.

Symbol-based compression was proposed in the early 1970s (Ascher and Nagy [1974]), but has become practical only recently. Advances in symbol-matching algorithms (see Chapter 12) and increased CPU computer processing speeds have made it possible to both select dictionary symbols and to find where they occur in an image in a timely manner. And like many other compression methods, symbol-based decoding is significantly faster than encoding. Finally, we note that both the symbol bitmaps that are stored in the dictionary and the triplets used to reference them themselves can be encoded to further improve compression performance. If, as in Fig. 8.17, only exact symbol matches are allowed, the resulting compression is lossless; if small differences are permitted, some level of reconstruction error will be present.

JBIG2 COMPRESSION

JBIG2 is an international standard for bilevel image compression. By segmenting an image into overlapping and/or non-overlapping regions of text, halftone, and generic content, compression techniques that are specifically optimized for each type of content are employed:

a b c

FIGURE 8.17
(a) A bi-level document, (b) symbol dictionary, and (c) the triplets used to locate the symbols in the document.



Token	Symbol	Triplet
0		(0, 2, 0) (3, 10, 1) (3, 18, 2) (3, 26, 1) (3, 34, 2) (3, 42, 1)
1		
2		

1. *Text regions* are composed of characters that are ideally suited for a symbol-based coding approach. Typically, each symbol will correspond to a character bitmap—a subimage representing a character of text. There is normally only one character bitmap (or subimage) in the symbol dictionary for each upper- and lowercase character of the font being used. For example, there would be one “a” bitmap in the dictionary, one “A” bitmap, one “b” bitmap, and so on.

In lossy JBIG2 compression, often called *perceptually lossless* or *visually lossless*, we neglect differences between dictionary bitmaps (i.e., the reference character bitmaps or character templates) and specific instances of the corresponding characters in the image. In lossless compression, the differences are stored and used in conjunction with the triplets encoding each character (by the decoder) to produce the actual image bitmaps. All bitmaps are encoded either arithmetically or using MMR (see Section 8.6); the triplets used to access dictionary entries are either arithmetically or Huffman encoded.

2. *Halftone regions* are similar to text regions in that they are composed of patterns arranged in a regular grid. The symbols that are stored in the dictionary, however, are not character bitmaps but periodic patterns that represent intensities (e.g., of a photograph) that have been dithered to produce bilevel images for printing.
3. *Generic regions* contain non-text, non-halftone information, like line art and noise, and are compressed using either arithmetic or MMR coding.

As is true of many image compression standards, JBIG2 defines decoder behavior. It does not explicitly define a standard encoder, but is flexible enough to allow various encoder designs. Although the design of the encoder is left unspecified, it is important because it determines the level of compression that is achieved. After all, the encoder must segment the image into regions, choose the text and halftone symbols that are stored in the dictionaries, and decide when those symbols are essentially the same as, or different from, potential instances of the symbols in the image. The decoder simply uses that information to recreate the original image.

EXAMPLE 8.11: JBIG2 compression example.

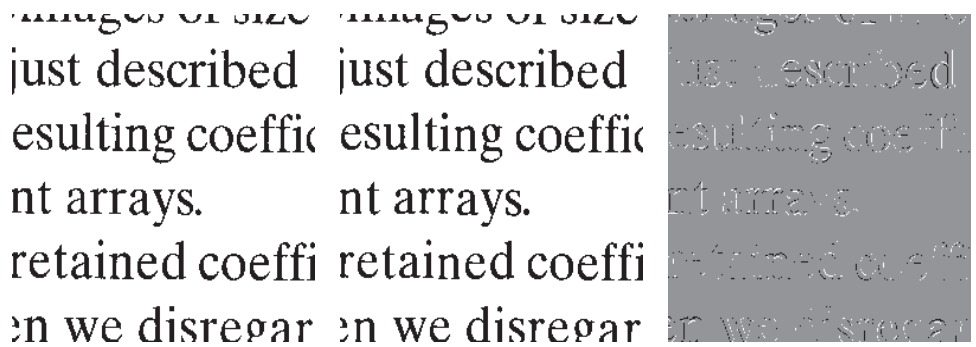
Consider again the bilevel image in Fig. 8.16(a). Figure 8.18(a) shows a reconstructed section of the image after lossless JBIG2 encoding (by a commercially available document compression application). It is an exact replica of the original image. Note that the *ds* in the reconstructed text vary slightly, despite the fact that they were generated from the same *d* entry in the dictionary. The differences between that *d* and the *ds* in the image were used to refine the output of the dictionary. The standard defines an algorithm for accomplishing this during the decoding of the encoded dictionary bitmaps. For the purposes of our discussion, you can think of it as adding the difference between a dictionary bitmap and a specific instance of the corresponding character in the image to the bitmap read from the dictionary.

Figure 8.18(b) is another reconstruction of the area in Fig. 8.18(a) after perceptually lossless JBIG2 compression. Note that the *ds* in this figure are identical. They have been copied directly from the symbol dictionary. The reconstruction is called perceptually lossless because the text is readable and the font is even the same. The small differences shown in Fig. 8.18(c) between the *ds* in the original image and the *d* in the dictionary are considered unimportant because they do not affect readability. Remember that

a b c

FIGURE 8.18

JBIG2 compression comparison: (a) lossless compression and reconstruction; (b) perceptually lossless; and (c) the scaled difference between the two.



we are dealing with bilevel images, so there are only three intensities in Fig. 8.18(c). Intensity 128 indicates areas where there is no difference between the corresponding pixels of the images in Figs. 8.18(a) and (b); intensities 0 (black) and 255 (white) indicate pixels of opposite intensities in the two images—for example, a black pixel in one image that is white in the other, and vice versa.

The lossless JBIG2 compression that was used to generate Fig. 8.18(a) reduces the original 663,445 byte uncompressed PDF image to 32,705 bytes; the compression ratio is $C = 20.3$. Perceptually lossless JBIG2 compression reduces the image to 23,913 bytes, increasing the compression ratio to about 27.7. These compressions are 4 to 5 times greater than the CCITT Group 3 and 4 results from Example 8.10.

8.8 BIT-PLANE CODING

With reference to Tables 8.3–8.5, bit-plane coding is used in

- JBIG2
- JPEG-2000

compression standards.

The run-length and symbol-based techniques of the previous sections can be applied to images with more than two intensities by individually processing their bit planes. The technique, called *bit-plane coding*, is based on the concept of decomposing a multilevel (monochrome or color) image into a series of binary images (see Section 3.2) and compressing each binary image via one of several well-known binary compression methods. In this section, we describe the two most popular decomposition approaches.

The intensities of an m -bit monochrome image can be represented in the form of the base-2 polynomial

$$a_{m-1}2^{m-1} + a_{m-2}2^{m-2} + \dots + a_12^1 + a_02^0 \quad (8-19)$$

Based on this property, a simple method of decomposing the image into a collection of binary images is to separate the m coefficients of the polynomial into m 1-bit bit planes. As noted in Section 3.2, the lowest-order bit plane (the plane corresponding to the least significant bit) is generated by collecting the a_0 bits of each pixel, while the highest-order bit plane contains the a_{m-1} bits or coefficients. In general, each bit plane is constructed by setting its pixels equal to the values of the appropriate bits or polynomial coefficients from each pixel in the original image. The inherent disadvantage of this decomposition approach is that small changes in intensity can have a significant impact on the complexity of the bit planes. If a pixel of intensity 127 (01111111) is adjacent to a pixel of intensity 128 (10000000), for instance, every bit

plane will contain a corresponding 0 to 1 (or 1 to 0) transition. For example, because the most significant bits of the binary codes for 127 and 128 are different, the highest bit plane will contain a zero-valued pixel next to a pixel of value 1, creating a 0 to 1 (or 1 to 0) transition at that point.

An alternative decomposition approach (which reduces the effect of small intensity variations) is to first represent the image by an m -bit *Gray code*. The m -bit Gray code $g_{m-1} \dots g_2 g_1 g_0$ that corresponds to the polynomial in Eq. (8-19) can be computed from

$$\begin{aligned} g_i &= a_i \oplus a_{i+1} & 0 \leq i \leq m-2 \\ g_{m-1} &= a_{m-1} \end{aligned} \quad (8-20)$$

Here, $\oplus$ denotes the exclusive OR operation. This code has the unique property that successive code words differ in only one bit position. Thus, small changes in intensity are less likely to affect all m bit planes. For instance, when intensity levels 127 and 128 are adjacent, only the highest-order bit plane will contain a 0 to 1 transition, because the Gray codes that correspond to 127 and 128 are 01000000 and 11000000, respectively.

EXAMPLE 8.12: Bit-plane coding.

Figures 8.19 and 8.20 show the eight binary and Gray-coded bit planes of the 8-bit monochrome image of the child in Fig. 8.19(a). Note that the high-order bit planes are far less complex than their low-order counterparts. That is, they contain large uniform areas of significantly less detail, busyness, or randomness. In addition, the Gray-coded bit planes are less complex than the corresponding binary bit planes. Both observations are reflected in the JBIG2 coding results of Table 8.11. Note, for instance, that the a_5 and g_5 results are significantly larger than the a_6 and g_6 compressions, and that both g_5 and g_6 are smaller than their a_5 and a_6 counterparts. This trend continues throughout the table, with the single exception of a_0 . Gray-coding provides a compression advantage of about 1.06:1 on average. Combined together, the Gray-coded files compress the original monochrome image by 678,676/475,964 or 1.43:1; the non-Gray-coded files compress the image by 678,676/503,916 or 1.35:1.

Finally, we note that the two least significant bits in Fig. 8.20 have little apparent structure. Because this is typical of most 8-bit monochrome images, bit-plane coding is usually restricted to images of 6 bits/pixel or less. JBIG1, the predecessor to JBIG2, imposes such a limit.

8.9 BLOCK TRANSFORM CODING

In this section, we consider a compression technique that divides an image into small non-overlapping blocks of equal size (e.g., 8×8) and processes the blocks independently using a 2-D transform. In *block transform coding*, a reversible, linear transform (such as the Fourier transform) is used to map each block or subimage into a set of transform coefficients, which are then quantized and coded. For most images, a significant number of the coefficients have small magnitudes and can be coarsely quantized (or discarded entirely) with little image distortion. A variety of

With reference to Tables 8.3–8.5, block transform coding is used in

- JPEG
- M-JPEG
- MPEG-1, 2, 4
- H.261, H.262, H.263, and H.264
- DV and HDV
- VC-1

TABLE 8.11

JBIG2 lossless coding results for the binary and Gray-coded bit planes of Fig. 8.19(a). These results include the overhead of each bit plane's PDF representation.

Coefficient m	Binary Code (PDF bits)	Gray Code (PDF bits)	Compression Ratio
7	6,999	6,999	1.00
6	12,791	11,024	1.16
5	40,104	36,914	1.09
4	55,911	47,415	1.18
3	78,915	67,787	1.16
2	101,535	92,630	1.10
1	107,909	105,286	1.03
0	99,753	107,909	0.92

transformations, including the discrete Fourier transform (DFT) of Chapter 4, can be used to transform the image data.

Figure 8.21 shows a typical block transform coding system. The decoder implements the inverse sequence of steps (with the exception of the quantization function) of the encoder, which performs four relatively straightforward operations: subimage decomposition, transformation, quantization, and coding. An $M \times N$ input image is subdivided first into subimages of size $n \times n$, which are then transformed to generate MN/n^2 subimage transform arrays, each of size $n \times n$. The goal of the transformation process is to decorrelate the pixels of each subimage, or to pack as much information as possible into the smallest number of transform coefficients. The quantization stage then selectively eliminates or more coarsely quantizes the coefficients that carry the least amount of information in a predefined sense (several methods will be discussed later in the section). These coefficients have the smallest impact on reconstructed subimage quality. The encoding process terminates by coding (normally using a variable-length code) the quantized coefficients. Any or all of the transform encoding steps can be adapted to local image content, called *adaptive transform coding*, or fixed for all subimages, called *nonadaptive transform coding*.

TRANSFORM SELECTION

Block transform coding systems based on a variety of discrete 2-D transforms have been constructed and/or studied extensively. The choice of a particular transform in a given application depends on the amount of reconstruction error that can be tolerated and the computational resources available. Compression is achieved during the quantization of the transformed coefficients (not during the transformation step).

EXAMPLE 8.13: Block transform coding with the DFT, WHT, and DCT.

Figures 8.22(a) through (c) show three approximations of the 512×512 monochrome image in Fig. 8.9(a). These pictures were obtained by dividing the original image into subimages of size 8×8 , representing each subimage using three of the transforms described in Chapter 7 (the DFT, WHT, and DCT transforms), truncating 50% of the resulting coefficients, and taking the inverse transform of the truncated coefficient arrays.

In this section, we restrict our attention to square subimages (the most commonly used). It is assumed that the input image is padded, if necessary, so that both M and N are multiples of n .

a	b
c	d
e	f
g	h

FIGURE 8.19

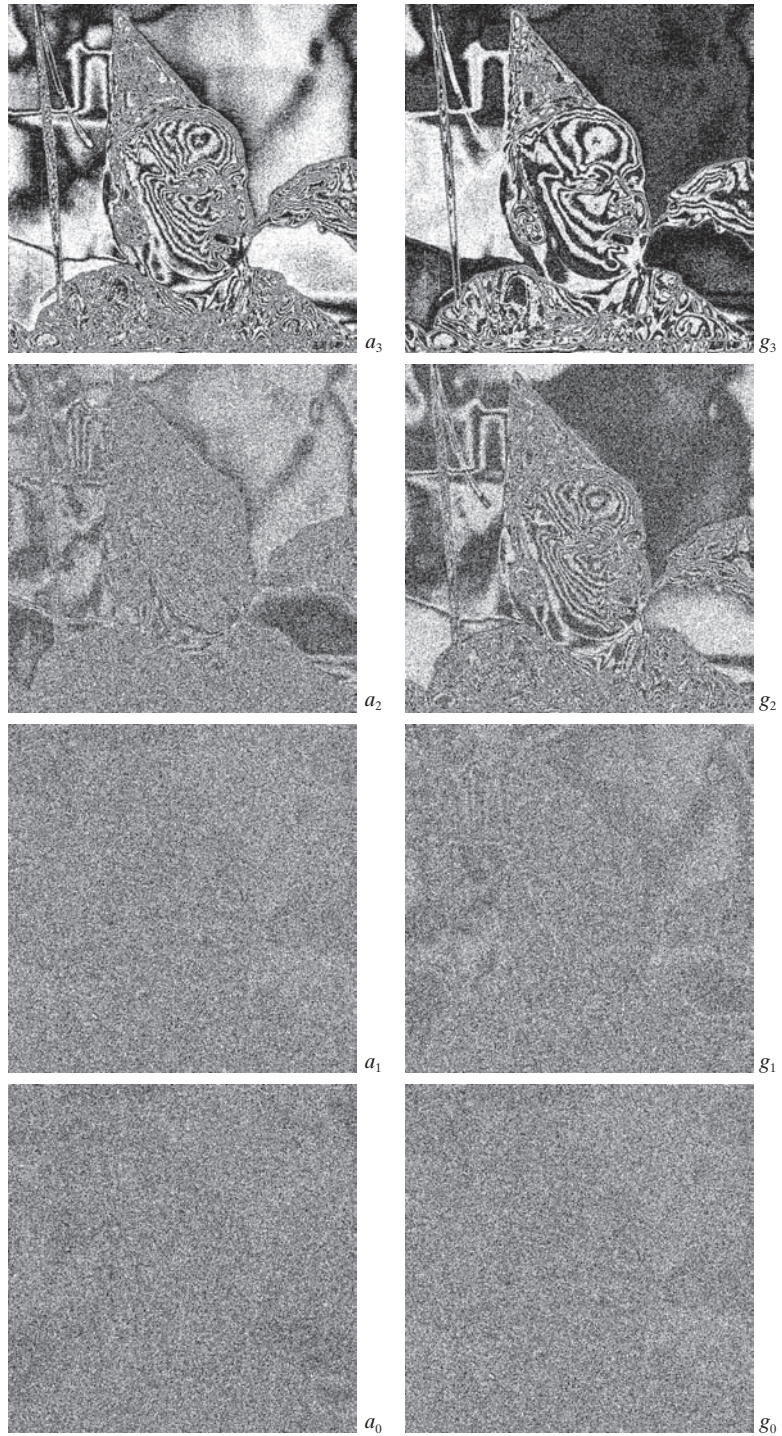
(a) A 256-bit monochrome image.
 (b)–(h) The four most significant binary and Gray-coded bit planes of the image in (a).

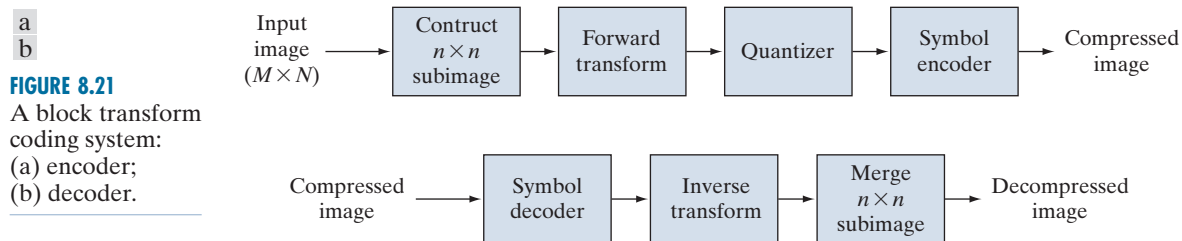


a	b
c	d
e	f
g	h

FIGURE 8.20

(a)–(h) The four least significant binary (left column) and Gray-coded (right column) bit planes of the image in Fig. 8.19(a).





In each case, the 32 retained coefficients were selected on the basis of maximum magnitude. Note that in all cases, the 32 discarded coefficients had little visual impact on the quality of the reconstructed image. Their elimination, however, was accompanied by some mean-squared error, which can be seen in the scaled error images of Figs. 8.22(d) through (f). The actual rms errors were 2.32, 1.78, and 1.13 intensities, respectively.



a **b** **c**
d **e** **f**

FIGURE 8.22 Approximations of Fig. 8.9(a) using the (a) Fourier, (b) Walsh-Hadamard, and (c) cosine transforms, together with the corresponding scaled error images in (d)–(f).

The small differences in mean-squared reconstruction error noted in the preceding example are related directly to the energy or information packing properties of the transforms employed. In accordance with Eqs. (7-75) and (7-76) of Section 7.5, an $n \times n$ subimage $g(x, y)$ can be expressed as a function of its 2-D transform $T(u, v)$:

$$\mathbf{G} = \sum_{u=0}^{n-1} \sum_{v=0}^{n-1} T(u, v) \mathbf{S}_{uv} \quad (8-21)$$

for $x, y = 0, 1, 2, \dots, n-1$. $\mathbf{G}$, the matrix containing the pixels of the input subimage, is explicitly defined as a linear combination of n^2 basis images of size $n \times n$. Recall that the basis images of the DFT, DCT, and WHT transforms for $n = 8$ are shown in Figs. 7.7, 7.10, and 7.16. If we now define a transform coefficient *masking function*

$$\chi(u, v) = \begin{cases} 0 & \text{if } T(u, v) \text{ satisfies a specified truncation criterion} \\ 1 & \text{otherwise} \end{cases} \quad (8-22)$$

for $u, v = 0, 1, 2, \dots, n-1$, an approximation of $\mathbf{G}$ can be obtained from the truncated expansion

$$\hat{\mathbf{G}} = \sum_{u=0}^{n-1} \sum_{v=0}^{n-1} \chi(u, v) T(u, v) \mathbf{S}_{uv} \quad (8-23)$$

where $\chi(u, v)$ is constructed to eliminate the basis images that make the smallest contribution to the total sum in Eq. (8-21). The mean-squared error between subimage $\mathbf{G}$ and approximation $\hat{\mathbf{G}}$ then is

$$\begin{aligned} e_{ms} &= E \left\{ \left\| \mathbf{G} - \hat{\mathbf{G}} \right\|^2 \right\} \\ &= E \left\{ \left\| \sum_{u=0}^{n-1} \sum_{v=0}^{n-1} T(u, v) \mathbf{S}_{uv} - \sum_{u=0}^{n-1} \sum_{v=0}^{n-1} \chi(u, v) T(u, v) \mathbf{S}_{uv} \right\|^2 \right\} \\ &= E \left\{ \left\| \sum_{u=0}^{n-1} \sum_{v=0}^{n-1} T(u, v) \mathbf{S}_{uv} [1 - \chi(u, v)] \right\|^2 \right\} \\ &= \sum_{u=0}^{n-1} \sum_{v=0}^{n-1} \sigma_{T(u, v)}^2 [1 - \chi(u, v)] \end{aligned} \quad (8-24)$$

where $\left\| \mathbf{G} - \hat{\mathbf{G}} \right\|$ is the norm of matrix $(\mathbf{G} - \hat{\mathbf{G}})$ and $\sigma_{T(u, v)}^2$ is the variance of the coefficient at transform location (u, v) . The final simplification is based on the orthogonal nature of the basis images and the assumption that the pixels of $\mathbf{G}$ are generated by a random process with zero mean and known covariance. The total mean-squared error of approximation thus is the sum of the variances of the discarded transform coefficients; that is, the coefficients for which $\chi(u, v) = 0$, so that $[1 - \chi(u, v)]$ in Eq. (8-24) is 1. Transformations that redistribute or pack the most information into the fewest coefficients provide the best subimage approximations and, consequently, the smallest reconstruction errors. Finally, under the assumptions

that led to Eq. (8-24), the mean-squared error of the MN/n^2 subimages of an $M \times N$ image are identical. Thus the mean-squared error (being a measure of *average* error) of the $M \times N$ image equals the mean-squared error of a single subimage.

The earlier example showed that the information packing ability of the DCT is superior to that of the DFT and WHT. Although this condition usually holds for most images, the Karhunen-Loève transform (see Chapter 11), not the DCT, is the optimal transform in an information packing sense. This is due to the fact that the KLT minimizes the mean-squared error in Eq. (8-24) for any input image and any number of retained coefficients (Kramer and Mathews [1956]). However, because the KLT is data dependent, obtaining the KLT basis images for each subimage, in general, is a nontrivial computational task. For this reason, the KLT is used infrequently for image compression. Instead, a transform, such as the DFT, WHT, or DCT, whose basis images are fixed (input independent) is normally used. Of the possible input independent transforms, the nonsinusoidal transforms (such as the WHT transform) are the simplest to implement. The sinusoidal transforms (such as the DFT or DCT) more closely approximate the information packing ability of the optimal KLT.

Hence, most transform coding systems are based on the DCT, which provides a good compromise between information packing ability and computational complexity. In fact, the properties of the DCT have proved to be of such practical value that the DCT is an international standard for transform coding systems. Compared to the other input independent transforms, it has the advantages of having been implemented in a single integrated circuit, packing the most information into the fewest coefficients[†] (for most images), and minimizing the block-like appearance, called *blocking artifact*, that results when the boundaries between subimages become visible. This last property is particularly important in comparisons with the other sinusoidal transforms. As Fig. 7.11(a) of Section 7.6 shows, the implicit n -point periodicity of the DFT gives rise to boundary discontinuities that result in substantial high-frequency transform content. When the DFT transform coefficients are truncated or quantized, the Gibbs phenomenon[‡] causes the boundary points to take on erroneous values, which appear in an image as blocking artifact. That is, the boundaries between adjacent subimages become visible because the boundary pixels of the subimages assume the mean values of discontinuities formed at the boundary points [see Fig. 7.11(a)]. The DCT of Fig. 7.11(b) reduces this effect, because its implicit $2n$ -point periodicity does not inherently produce boundary discontinuities.

SUBIMAGE SIZE SELECTION

Another significant factor affecting transform coding error and computational complexity is subimage size. In most applications, images are subdivided so the correlation (redundancy) between adjacent subimages is reduced to some acceptable level

An additional condition for optimality is that the masking function of Eq. (8-22) selects the KLT coefficients of maximum variance.

[†]Ahmed et al. [1974] first noticed that the KLT basis images of a first-order Markov image source closely resemble the DCT's basis images. As the correlation between adjacent pixels approaches one, the input-dependent KLT basis images become identical to the input-independent DCT basis images (Clarke [1985]).

[‡]This phenomenon, described in most electrical engineering texts on circuit analysis, occurs because the Fourier transform fails to converge uniformly at discontinuities. At discontinuities, Fourier expansions take the mean values of the points of discontinuity.

and so n is an integer power of 2 where, as before, n is the subimage dimension. The latter condition simplifies the computation of the subimage transforms (see the base-2 successive doubling method discussed in Section 4.11). In general, both the level of compression and computational complexity increase as the subimage size increases. The most popular subimage sizes are 8×8 and 16×16 .

EXAMPLE 8.14: Effects of subimage size on transform coding.

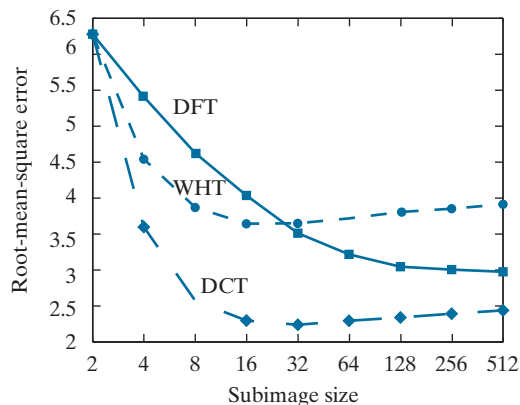
Figure 8.23 illustrates graphically the impact of subimage size on transform coding reconstruction error. The data plotted were obtained by dividing the monochrome image of Fig. 8.9(a) into subimages of size $n \times n$, for $n = 2, 4, 8, 16, \dots, 256, 512$, computing the transform of each subimage, truncating 75% of the resulting coefficients, and taking the inverse transform of the truncated arrays. Note that the Hadamard and cosine curves flatten as the size of the subimage becomes greater than 8×8 , whereas the Fourier reconstruction error continues to decrease in this region. As n further increases, the Fourier reconstruction error crosses the Walsh-Hadamard curve and approaches the cosine result. This result is consistent with the theoretical and experimental findings reported by Netravali and Limb [1980] and by Pratt [2001] for a 2-D Markov image source.

All three curves intersect when 2×2 subimages are used. In this case, only one of the four coefficients (25%) of each transformed array was retained. The coefficient in all cases was the dc component, so the inverse transform simply replaced the four subimage pixels by their average value [see Eq. (4-92)]. This condition is evident in Fig. 8.24(b), which shows a zoomed portion of the 2×2 DCT result. Note that the blocking artifact that is prevalent in this result decreases as the subimage size increases to 4×4 and 8×8 in Figs. 8.24(c) and (d). Figure 8.24(a) shows a zoomed portion of the original image for reference.

BIT ALLOCATION

The reconstruction error associated with the truncated series expansion of Eq. (8-23) is a function of the number and relative importance of the transform coefficients that are discarded, as well as the precision that is used to represent the retained coefficients. In most transform coding systems, the retained coefficients are selected [that is, the masking function of Eq. (8-22) is constructed] on the basis of maximum

FIGURE 8.23
Reconstruction
error versus
subimage size.



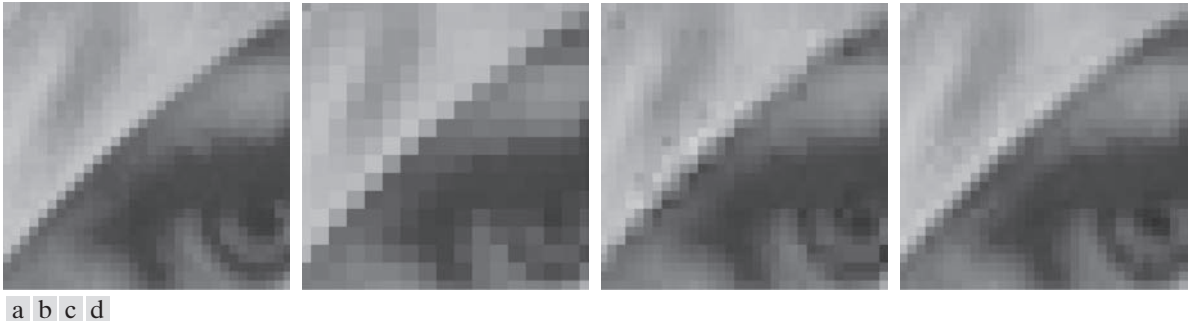


FIGURE 8.24 Approximations of Fig. 8.24(a) using 25% of the DCT coefficients and (b) 2×2 subimages, (c) 4×4 subimages, and (d) 8×8 subimages. The original image in (a) is a zoomed section of Fig. 8.9(a).

variance, called *zonal coding*, or on the basis of maximum magnitude, called *threshold coding*. The overall process of truncating, quantizing, and coding the coefficients of a transformed subimage is commonly called *bit allocation*.

EXAMPLE 8.15: Bit allocation.

Figures 8.25(a) and (c) show two approximations of Fig. 8.9(a) in which 87.5% of the DCT coefficients of each 8×8 subimage were discarded. The first result was obtained via threshold coding by keeping the eight largest transform coefficients, and the second image was generated by using a zonal coding approach. In the latter case, each DCT coefficient was considered a random variable whose distribution could be computed over the ensemble of all transformed subimages. The eight distributions of largest variance (12.5% of the 64 coefficients in the transformed 8×8 subimage) were located and used to determine the coordinates $[u$ and v of the coefficients, $T(u, v)]$, that were retained for all subimages. Note that the threshold coding difference image of Fig. 8.25(b) contains less error than the zonal coding result in Fig. 8.25(d). Both images have been scaled to make the errors more visible. The corresponding rms errors are 4.5 and 6.5 intensities, respectively.

Zonal Coding Implementation

Zonal coding is based on the information theory concept of viewing information as uncertainty. Therefore, the transform coefficients of maximum variance carry the most image information, and should be retained in the coding process. The variances themselves can be calculated directly from the ensemble of MN/n^2 transformed subimage arrays (as in the preceding example) or based on an assumed image model (say, a Markov autocorrelation function). In either case, the zonal sampling process can be viewed, in accordance with Eq. (8-23), as multiplying each $T(u, v)$ by the corresponding element in a *zonal mask*, which is constructed by placing a 1 in the locations of maximum variance and a 0 in all other locations. Coefficients of maximum variance usually are located around the origin of an image transform, resulting in the typical zonal mask shown in Fig. 8.26(a).

The coefficients retained during the zonal sampling process must be quantized and coded, so zonal masks are sometimes depicted showing the number of bits used

a	b
c	d

FIGURE 8.25

Approximations of Fig. 8.9(a) using 12.5% of the DCT coefficients: (a)–(b) threshold coding results; (c)–(d) zonal coding results. The difference images are scaled by 4.



to code each coefficient [see Fig. 8.26(b)]. In most cases, the coefficients are allocated the same number of bits, or some fixed number of bits is distributed among them unequally. In the first case, the coefficients generally are normalized by their standard deviations and uniformly quantized. In the second case, a quantizer, such as an optimal Lloyd-Max quantizer (see Optimal quantizers in Section 8.10), is designed for each coefficient. To construct the required quantizers, the zeroth or DC coefficient normally is modeled by a Rayleigh density function, whereas the remaining coefficients are modeled by a Laplacian or Gaussian density.[†] The number of quantization levels (and thus the number of bits) allotted to each quantizer is made proportional to $\log_2 \sigma_{T(u,v)}^2$. Thus, the retained coefficients in Eq. (8-23)—which (in the context of the current discussion) are selected on the basis of maximum variance—are assigned bits in proportion to the logarithm of the coefficient variances.

Threshold Coding Implementation

Zonal coding usually is implemented by using a single fixed mask for all subimages. Threshold coding, however, is inherently adaptive in the sense that the location of the transform coefficients retained for each subimage vary from one subimage to

[†]As each coefficient is a linear combination of the pixels in its subimage [see Eq. (7-31)], the central-limit theorem suggests that, as subimage size increases, the coefficients tend to become Gaussian. This result does not apply to the dc coefficient, however, because nonnegative images always have positive dc coefficients.

a	b
c	d

FIGURE 8.26

A typical
(a) zonal mask,
(b) zonal bit allo-
cation,
(c) threshold
mask, and
(d) thresholded
coefficient ordering
sequence.
Shading highlights
the coefficients
that are retained.

1	1	1	1	1	0	0	0
1	1	1	1	0	0	0	0
1	1	1	0	0	0	0	0
1	1	0	0	0	0	0	0
1	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0

8	7	6	4	3	2	1	0
7	6	5	4	3	2	1	0
6	5	4	3	3	1	1	0
4	4	3	3	2	1	0	0
3	3	3	2	1	1	0	0
2	2	1	1	1	0	0	0
1	1	1	0	0	0	0	0
0	0	0	0	0	0	0	0

1	1	0	1	1	0	0	0
1	1	1	0	0	0	0	0
1	1	0	0	0	0	0	0
1	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0

0	1	5	6	14	15	27	28
2	4	7	13	16	26	29	42
3	8	12	17	25	30	41	43
9	11	18	24	31	40	44	53
10	19	23	32	39	45	52	54
20	22	33	38	46	51	55	60
21	34	37	47	50	56	59	61
35	36	48	49	57	58	62	63

another. In fact, threshold coding is the adaptive transform coding approach most often used in practice because of its computational simplicity. The underlying concept is that, for any subimage, the transform coefficients of largest magnitude make the most significant contribution to reconstructed subimage quality, as demonstrated in the last example. Because the locations of the maximum coefficients vary from one subimage to another, the elements of $X(u, v)T(u, v)$ normally are reordered (in a predefined manner) to form a 1-D, run-length coded sequence. Figure 8.26(c) shows a typical *threshold mask* for one subimage of a hypothetical image. This mask provides a convenient way to visualize the threshold coding process for the corresponding subimage, as well as to mathematically describe the process using Eq. (8-23). When the mask is applied [via Eq. (8-23)] to the subimage for which it was derived, and the resulting $n \times n$ array is reordered to form an n^2 -element coefficient sequence in accordance with the zigzag ordering pattern of Fig. 8.26(d), the reordered 1-D sequence contains several long runs of 0's. [The zigzag pattern becomes evident by starting at 0 in Fig. 8.26(d) and following the numbers in sequence.] These runs normally are run-length coded. The nonzero or retained coefficients, corresponding to the mask locations that contain a 1, are represented using a variable-length code.

There are three basic ways to threshold a transformed subimage or, stated differently, to create a subimage threshold masking function of the form given in Eq. (8-22): (1) A single global threshold can be applied to all subimages; (2) a different threshold can be used for each subimage, or; (3) the threshold can be varied as a function of the location of each coefficient within the subimage. In the first approach,

The N in “ N -largest coding” is not an image dimension, but refers to the number of coefficients that are kept.

the level of compression differs from image to image, depending on the number of coefficients that exceed the global threshold. In the second, called *N -largest coding*, the same number of coefficients is discarded for each subimage. As a result, the code rate is constant and known in advance. The third technique, like the first, results in a variable code rate, but offers the advantage that thresholding *and* quantization can be combined by replacing $\chi(u,v)T(u,v)$ in Eq. (8-23) with

$$\hat{T}(u,v) = \text{round} \left[\frac{T(u,v)}{Z(u,v)} \right] \quad (8-25)$$

where $\hat{T}(u,v)$ is a thresholded and quantized approximation of $T(u,v)$, and $Z(u,v)$ is an element of the following transform normalization array:

$$\mathbf{Z} = \begin{bmatrix} Z(0,0) & Z(0,1) & \dots & Z(0,n-1) \\ Z(1,0) & \vdots & \dots & \vdots \\ \vdots & \vdots & \dots & \vdots \\ \vdots & \vdots & \dots & \vdots \\ \vdots & \vdots & \dots & \vdots \\ Z(n-1,0) & Z(n-1,1) & \dots & Z(n-1,n-1) \end{bmatrix} \quad (8-26)$$

Before a normalized (thresholded and quantized) subimage transform, $\hat{T}(u,v)$, can be inverse transformed to obtain an approximation of subimage $g(x,y)$, it must be multiplied by $Z(u,v)$. The resulting denormalized array, denoted $\hat{T}(u,v)$ is an approximation of $\hat{T}(u,v)$:

$$\hat{T}(u,v) = \hat{T}(u,v)Z(u,v) \quad (8-27)$$

The inverse transform of $\hat{T}(u,v)$ yields the decompressed subimage approximation.

Figure 8.27(a) graphically depicts Eq. (8-25) for the case in which $Z(u,v)$ is assigned a particular value c . Note that $\hat{T}(u,v)$ assumes integer value k if and only if

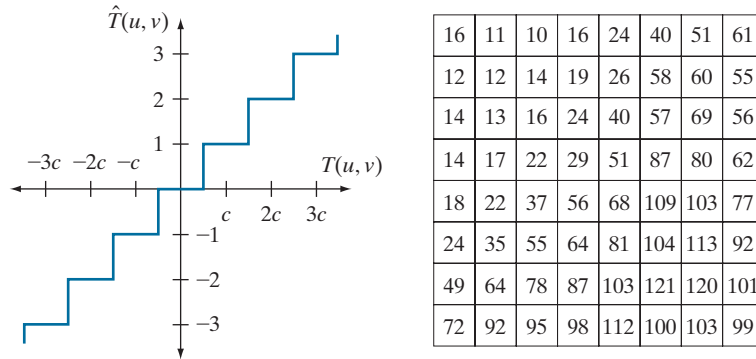
$$kc - \frac{c}{2} \leq T(u,v) < kc + \frac{c}{2} \quad (8-28)$$

If $Z(u,v) > 2T(u,v)$, then $\hat{T}(u,v) = 0$ and the transform coefficient is completely truncated or discarded. When $\hat{T}(u,v)$ is represented with a variable-length code that increases in length as the magnitude of k increases, the number of bits used to represent $T(u,v)$ is controlled by the value of c . Thus, the elements of $\mathbf{Z}$ can be scaled to achieve a variety of compression levels. Figure 8.27(b) shows a typical normalization array. This array, which has been used extensively in the JPEG standardization efforts (see the next section), weighs each coefficient of a transformed subimage according to heuristically determined perceptual or psychovisual importance.

a b

FIGURE 8.27

(a) A threshold coding quantization curve [see Eq. (8-28)]. (b) A typical normalization matrix.



EXAMPLE 8.16: Illustration of threshold coding.

Figures 8.28(a) through (f) show six threshold-coded approximations of the monochrome image in Fig. 8.9(a). All images were generated using an 8×8 DCT and the normalization array of Fig. 8.27(b). The first result, which provides a compression ratio of about 12 to 1 (i.e., $C = 12$), was obtained by direct application of that normalization array. The remaining results, which compress the original image by 19, 30, 49, 85, and 182 to 1, were generated after multiplying (scaling) the normalization arrays by 2, 4, 8, 16, and 32, respectively. The corresponding rms errors are 3.83, 4.93, 6.62, 9.35, 13.94, and 22.46 intensity levels.

JPEG

One of the most popular and comprehensive continuous tone, still-frame compression standards is the JPEG standard. It defines three different coding systems: (1) a lossy baseline coding system, which is based on the DCT and is adequate for most compression applications; (2) an extended coding system for greater compression, higher precision, or progressive reconstruction applications; and (3) a lossless independent coding system for reversible compression. To be JPEG compatible, a product or system must include support for the baseline system. No particular file format, spatial resolution, or color space model is specified.

In the baseline system, often called the *sequential baseline system*, the input and output data precision is limited to 8 bits, whereas the quantized DCT values are restricted to 11 bits. The compression itself is performed in three sequential steps: DCT computation, quantization, and variable-length code assignment. The image is first subdivided into pixel blocks of size 8×8 , which are processed left-to-right, top-to-bottom. As each 8×8 block or subimage is encountered, its 64 pixels are level-shifted by subtracting the quantity 2^{k-1} , where 2^k is the maximum number of intensity levels. The 2-D discrete cosine transform of the block is then computed, quantized in accordance with Eq. (8-25), and reordered, using the zigzag pattern of Fig. 8.26(d), to form a 1-D sequence of quantized coefficients.

Because the one-dimensionally reordered array generated under the zigzag pattern of Fig. 8.26(d) is arranged qualitatively according to increasing spatial frequency, the JPEG coding procedure is designed to take advantage of the long runs



FIGURE 8.28 Approximations of Fig. 8.9(a) using the DCT and normalization array of Fig. 8.27(b): (a) Z , (b) $2Z$, (c) $4Z$, (d) $8Z$, (e) $16Z$, and (f) $32Z$.

Consult the book website for the JPEG default Huffman code tables: (1) a JPEG coefficient category table, (2) a default DC code table, and (3) a default AC code table.

of zeros that normally result from the reordering. In particular, the nonzero AC[†] coefficients are coded using a variable-length code that defines the coefficient values and number of preceding zeros. The DC coefficient is difference coded relative to the DC coefficient of the previous subimage. The default JPEG Huffman codes for the luminance component of a color image, or intensity of a monochrome image, are available on the book website. The JPEG recommended luminance quantization array is given in Fig. 8.27(b) and can be scaled to provide a variety of compression levels. The scaling of this array allows users to select the “quality” of JPEG compressions. Although default coding tables and quantization arrays are provided for both color and monochrome processing, the user is free to construct custom tables and/or arrays, which may be adapted to the characteristics of the image being compressed.

[†]In the standard, the term AC denotes all transform coefficients with the exception of the zeroth or DC coefficient.

EXAMPLE 8.17: JPEG baseline coding and decoding.

Consider compression and reconstruction of the following 8×8 subimage with the JPEG baseline standard:

52	55	61	66	70	61	64	73
63	59	66	90	109	85	69	72
62	59	68	113	144	104	66	73
63	58	71	122	154	106	70	69
67	61	68	104	126	88	68	70
79	65	60	70	77	63	58	75
85	71	64	59	55	61	65	83
87	79	69	68	65	76	78	94

The original image consists of 256 or 2^8 possible intensities, so the coding process begins by level shifting the pixels of the original subimage by -2^7 or -128 intensity levels. The resulting shifted array is

-76	-73	-67	-62	-58	-67	-64	-55
-65	-69	-62	-38	-19	-43	-59	-56
-66	-69	-60	-15	16	-24	-62	-55
-65	-70	-57	-6	26	-22	-58	-59
-61	-67	-60	-24	-2	-40	-60	-58
-49	-63	-68	-58	-51	-65	-70	-53
-43	-57	-64	-69	-73	-67	-63	-45
-41	-49	-59	-60	-63	-52	-50	-34

which, when transformed in accordance with the forward DCT of Eq. (7-31) with $r(x, y, u, v) = s(x, y, u, v)$ of Eq. (7-85) for $n = 8$ becomes

-415	-29	-62	25	55	-20	-1	3
7	-21	-62	9	11	-7	-6	6
-46	8	77	-25	-30	10	7	-5
-50	13	35	-15	-9	6	0	3
11	-8	-13	-2	-1	1	-4	1
-10	1	3	-3	-1	0	2	-1
-4	-1	2	-1	2	-3	1	-2
-1	-1	-1	-2	-1	-1	0	-1

If the JPEG recommended normalization array of Fig. 8.27(b) is used to quantize the transformed array, the scaled and truncated [that is, normalized in accordance with Eq. (8-25)] coefficients are

-26	-3	-6	2	2	0	0	0
1	-2	-4	0	0	0	0	0
-3	1	5	-1	-1	0	0	0
-4	1	2	-1	0	0	0	0
1	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0

where, for instance, the DC coefficient is computed as

$$\begin{aligned}\hat{T}(0,0) &= \text{round} \left[\frac{T(0,0)}{Z(0,0)} \right] \\ &= \text{round} \left[\frac{-415}{16} \right] = -26\end{aligned}$$

Note that the transformation and normalization process produces a large number of zero-valued coefficients. When the coefficients are reordered in accordance with the zigzag ordering pattern of Fig. 8.26(d), the resulting 1-D coefficient sequence is

$$[-26 \ -3 \ 1 \ -3 \ -2 \ -6 \ 2 \ -4 \ 1 \ -4 \ 1 \ 1 \ 5 \ 0 \ 2 \ 0 \ 0 \ -1 \ 2 \ 0 \ 0 \ 0 \ 0 \ 0 \ -1 \ -1 \ \text{EOB}]$$

where the EOB symbol denotes the end-of-block condition. A special EOB Huffman code word (see category 0 and run-length 0 of the JPEG default AC code table on the book website) is provided to indicate that the remainder of the coefficients in a reordered sequence are zeros.

The construction of the default JPEG code for the reordered coefficient sequence begins with the computation of the difference between the current DC coefficient and that of the previously encoded subimage. Assuming the DC coefficient of the transformed and quantized subimage to its immediate left was -17 , the resulting DPCM difference is $[-26 - (-17)]$ or -9 , which lies in DC difference category 4 of the JPEG coefficient category table (see the book website). In accordance with the default Huffman difference code, the proper base code for a category 4 difference is 101 (a 3-bit code), while the total length of a completely encoded category 4 coefficient is 7 bits. The remaining 4 bits must be generated from the least significant bits (LSBs) of the difference value. For a general DC difference category (say, category K), an additional K bits are needed and computed as either the K LSBs of the positive difference or the K LSBs of the negative difference minus 1. For a difference of -9 , the appropriate LSBs are $(0111) - 1$ or 0110, and the complete DPCM coded DC code word is 1010110.

The nonzero AC coefficients of the reordered array are coded similarly. The principal difference is that each default AC Huffman code word depends on the number of zero-valued coefficients preceding the nonzero coefficient to be coded, as well as the magnitude category of the nonzero coefficient. (See the column labeled Run/Category in the JPEG AC code table on the book website.) Thus, the first nonzero AC coefficient of the reordered array (-3) is coded as 0100. The first 2 bits of this code indicate that the coefficient was in magnitude category 2 and preceded by no zero-valued coefficients; the last 2 bits are generated by the same process used to arrive at the LSBs of the DC difference code. Continuing in

this manner, the completely coded (reordered) array is

```
1010110 0100 001 0100 0101 100001 0110 100011 001 100011 001
001 100101 11100110 110110 0110 11110100 000 1010
```

where the spaces have been inserted solely for readability. Although it was not needed in this example, the default JPEG code contains a special code word for a run of 15 zeros followed by a zero. The total number of bits in the completely coded reordered array (and thus the number of bits required to represent the entire 8 × 8, 8-bit subimage of this example) is 92. The resulting compression ratio is 512/92, or about 5.6:1.

To decompress a JPEG compressed subimage, the decoder must first recreate the normalized transform coefficients that led to the compressed bit stream. Because a Huffman-coded binary sequence is instantaneous and uniquely decodable, this step is easily accomplished in a simple lookup table manner. Here, the regenerated array of quantized coefficients is

−26	−3	−6	2	2	0	0	0
1	−2	−4	0	0	0	0	0
−3	1	5	−1	−1	0	0	0
−4	1	2	−1	0	0	0	0
1	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0

After denormalization in accordance with Eq. (8-27), the array becomes

−416	−33	−60	32	48	0	0	0
12	−24	−56	0	0	0	0	0
−42	13	80	−24	−40	0	0	0
−56	17	44	−29	0	0	0	0
18	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0

where, for example, the DC coefficient is computed as

$$\hat{T}(0,0) = \hat{T}(0,0)Z(0,0) = (-26)(16) = -416$$

The completely reconstructed subimage is obtained by taking the inverse DCT of the denormalized array in accordance with Eqs. (7-32) and (7-85) to obtain

-70	-64	-61	-64	-69	-66	-58	-50
-72	-73	-61	-39	-30	-40	-54	-59
-68	-78	-58	-9	13	-12	-48	-64
-59	-77	-57	0	22	-13	-51	-60
-54	-75	-64	-23	-13	-44	-63	-56
-52	-71	-72	-54	-54	-71	-71	-54
-45	-59	-70	-68	-67	-67	-61	-50
-35	-47	-61	-66	-60	-48	-44	-44

and level shifting each inverse transformed pixel by 2^7 (or +128) to yield

58	64	67	64	59	62	70	78
56	55	67	89	98	88	74	69
60	50	70	119	141	116	80	64
69	51	71	128	149	115	77	68
74	53	64	105	115	84	65	72
76	57	56	74	75	57	57	74
83	69	59	60	61	61	67	78
93	81	67	62	69	80	84	84

Any differences between the original and reconstructed subimage are a result of the lossy nature of the JPEG compression and decompression process. In this example, the errors range from -14 to +11 and are distributed as follows:

-6	-9	-6	2	11	-1	-6	-5
7	4	-1	1	11	-3	-5	3
2	9	-2	-6	-3	-12	-14	9
-6	7	0	-4	-5	-9	-7	1
-7	8	4	-1	6	4	3	-2
3	8	4	-4	2	6	1	1
2	2	5	-1	-6	0	-2	5
-6	-2	2	6	-4	-4	-6	10

The root-mean-squared error of the overall compression and reconstruction process is approximately 5.8 intensity levels.

EXAMPLE 8.18: Illustration of JPEG coding.

Figures 8.29(a) and (d) show two JPEG approximations of the monochrome image in Fig. 8.9(a). The first result provides a compression of 25:1; the second compresses the original image by 52:1. The differences between the original image and the reconstructed images in Figs. 8.29(a) and (d) are shown in Figs. 8.29(b) and (e), respectively. The corresponding rms errors are 5.4 and 10.7 intensities. The errors are clearly visible in the zoomed images in Figs. 8.29(c) and (f). These images show a magnified section of Figs. 8.29(a) and (d), respectively. Note that the JPEG blocking artifact increases with compression.

With reference to
Tables 8.3–8.5, predictive
coding is used in

- JBIG2
- JPEG
- JPEG-LS
- MPEG-1,2,4
- H.261, H.262,
H.263, and H.264
- HDV
- VC-1

and other compres-
sion standards and file
formats.

8.10 PREDICTIVE CODING

We now turn to a simpler approach that achieves good compression without significant computational overhead *and* can be either error-free or lossy. The approach, commonly referred to as *predictive coding*, is based on eliminating the redundancies of closely spaced pixels—in space and/or time—by extracting and coding only the *new information* in each pixel. The new information of a pixel is defined as the difference between the actual and predicted value of the pixel.

LOSSLESS PREDICTIVE CODING

Figure 8.30 shows the basic components of a *lossless predictive coding* system. The system consists of an encoder and a decoder, each containing an identical *predictor*. As successive samples of discrete time input signal, $f(n)$, are introduced to the encoder, the predictor generates the anticipated value of each sample based on a specified number of past samples. The output of the predictor is then rounded to the nearest integer, denoted $\hat{f}(n)$, and used to form the difference or *prediction error*

$$e(n) = f(n) - \hat{f}(n) \quad (8-29)$$

which is encoded using a variable-length code (by the symbol encoder) to generate the next element of the compressed data stream. The decoder in Fig. 8.30(b) reconstructs $e(n)$ from the received variable-length code words and performs the inverse operation

$$f(n) = e(n) + \hat{f}(n) \quad (8-30)$$

to decompress or recreate the original input sequence.

Various local, global, and adaptive methods (see the later subsection entitled Lossy predictive coding) can be used to generate $\hat{f}(n)$. In many cases, the prediction is formed as a linear combination of m previous samples. That is,

$$\hat{f}(n) = \text{round} \left[\sum_{i=1}^m \alpha_i f(n-i) \right] \quad (8-31)$$

where m is the *order* of the linear predictor, round is a function used to denote the rounding or nearest integer operation, and the α_i for $i = 1, 2, \dots, m$ are prediction

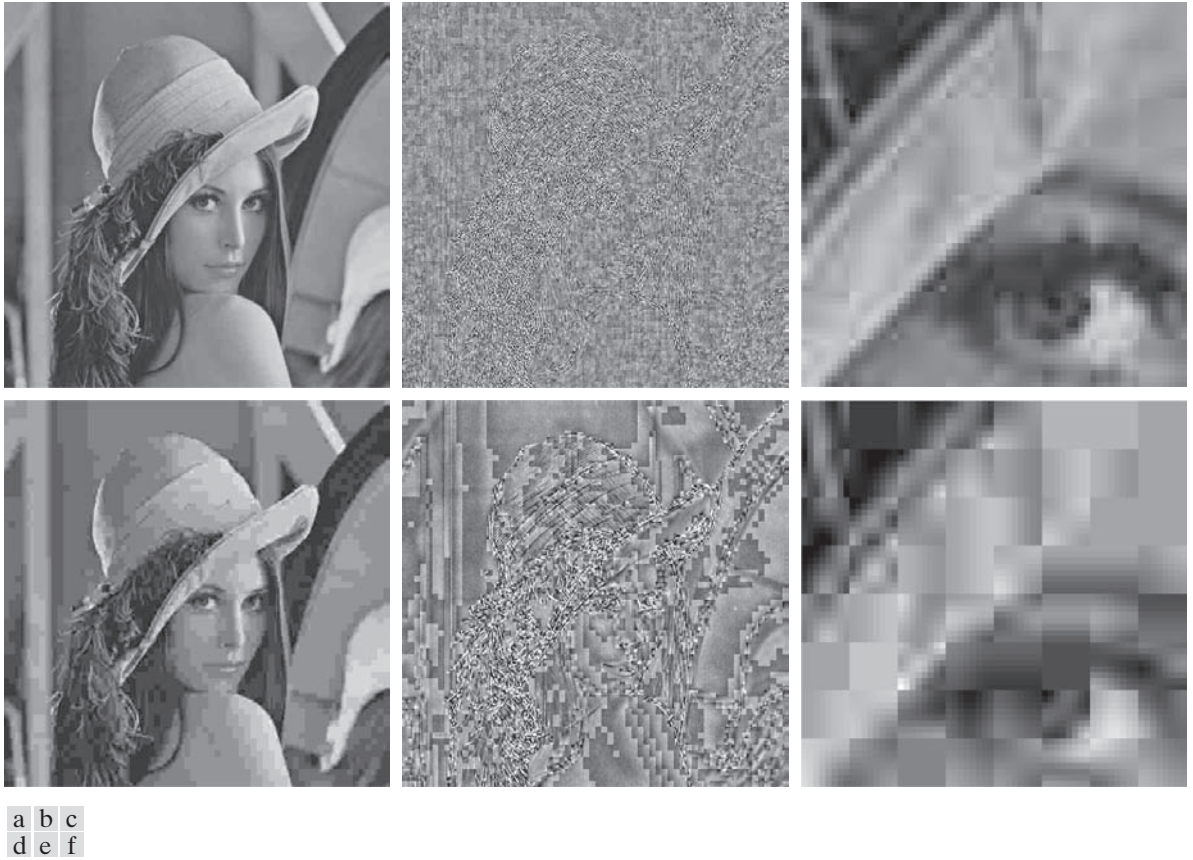


FIGURE 8.29 Two JPEG approximations of Fig. 8.9(a). Each row contains a result after compression and reconstruction, the scaled difference between the result and the original image, and a zoomed portion of the reconstructed image.

coefficients. If the input sequence in Fig. 8.30(a) is considered to be samples of an image, the $f(n)$ in Eqs. (8-29) through (8-31) are pixels—and the m samples used to predict the value of each pixel come from the current scan line (called 1-D linear predictive coding), from the current and previous scan lines (called 2-D linear predictive coding), or from the current image and previous images in a sequence of images (called 3-D linear predictive coding). Thus, for 1-D linear predictive image coding, Eq. (8-31) can be written as

$$\hat{f}(x, y) = \text{round} \left[\sum_{i=1}^m \alpha_i f(x, y - i) \right] \quad (8-32)$$

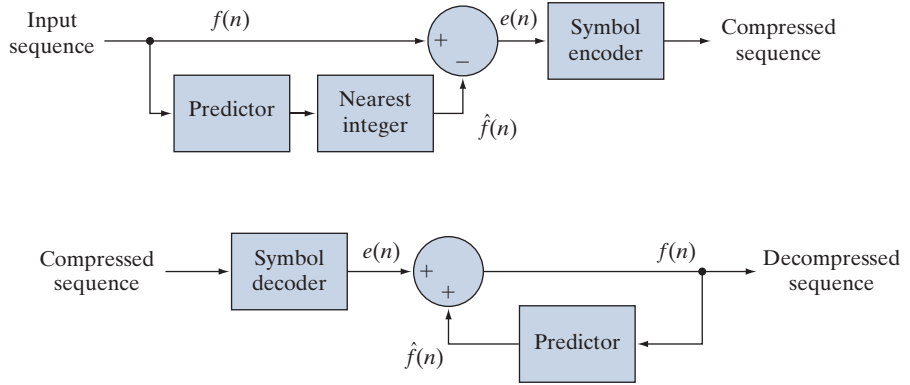
where each sample is now expressed explicitly as a function of the input image's spatial coordinates, x and y . Note that Eq. (8-32) indicates that the 1-D linear prediction is a function of the previous pixels on the current line alone. In 2-D predictive

a
b**FIGURE 8.30**

A lossless predictive coding model:

(a) encoder;

(b) decoder.



coding, the prediction is a function of the previous pixels in a left-to-right, top-to-bottom scan of an image. In the 3-D case, it is based on these pixels and the previous pixels of preceding frames. Equation (8-32) cannot be evaluated for the first m pixels of each line, so those pixels must be coded by using other means (such as a Huffman code) and considered as an overhead of the predictive coding process. Similar comments apply to the higher-dimensional cases.

EXAMPLE 8.19: Predictive coding and spatial redundancy.

Consider encoding the monochrome image of Fig. 8.31(a) using the simple first-order (i.e., $m = 1$) linear predictor from Eq. (8-32)

$$\hat{f}(x, y) = \text{round}[\alpha f(x, y - 1)] \quad (8-33)$$

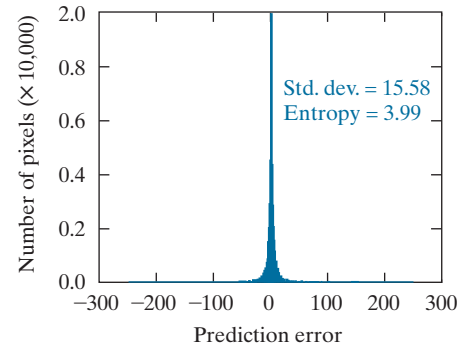
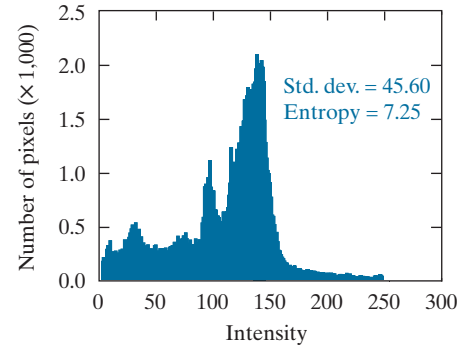
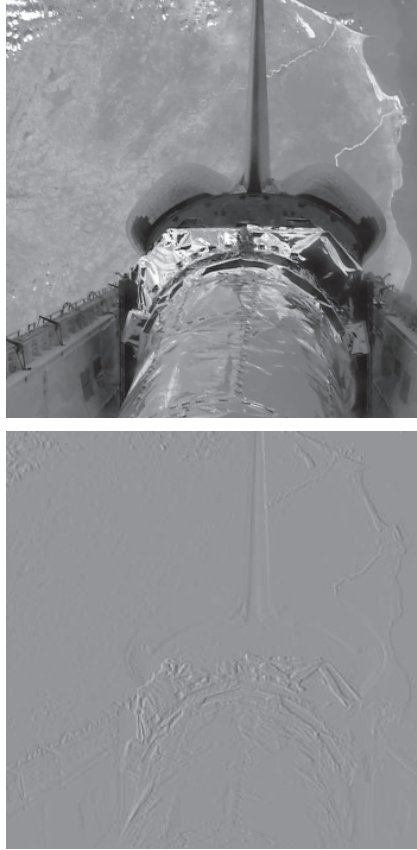
This equation is a simplification of Eq. (8-32), with $m = 1$ and the subscript of lone prediction coefficient α_1 removed as unnecessary. A predictor of this general form is called a *previous pixel* predictor, and the corresponding predictive coding procedure is known as *differential coding* or *previous pixel coding*. Figure 8.31(c) shows the prediction error image, $e(x, y) = f(x, y) - \hat{f}(x, y)$ that results from Eq. (8-33) with $\alpha = 1$. The scaling of this image is such that intensity 128 represents a prediction error of zero, while all nonzero positive and negative prediction errors (under and over estimates) are displayed as lighter and darker shades of gray, respectively. The mean value of the prediction image is 128.26. Because intensity 128 corresponds to a prediction error of 0, the average prediction error is only 0.26 bits.

Figures 8.31(b) and (d) show the intensity histogram of the image in Fig. 8.31(a) and the histogram of prediction error $e(x, y)$, respectively. Note that the standard deviation of the prediction error in Fig. 8.31(d) is much smaller than the standard deviation of the intensities in the original image. Moreover, the entropy of the prediction error, as estimated using Eq. (8-7), is significantly less than the estimated entropy of the original image (3.99 bits/pixel as opposed to 7.25 bits/pixel). This decrease in entropy reflects removal of a great deal of spatial redundancy, despite the fact that for k -bit images, $(k + 1)$ -bit numbers are needed to represent accurately the prediction error sequence $e(x, y)$. (Note that the variable-length encoded prediction error is the compressed image.) In general, the maximum compression of a predictive coding approach can be estimated by dividing the average number of bits used

a	b
c	d

FIGURE 8.31

(a) A view of the Earth from an orbiting space shuttle. (b) The intensity histogram of (a). (c) The prediction error image resulting from Eq. (8-33). (d) A histogram of the prediction error. (Original image courtesy of NASA.)



to represent each pixel in the original image by an estimate of the entropy of the prediction error. In this example, any variable-length coding procedure can be used to code $e(x, y)$, but the resulting compression will be limited to about $8/3.99$, or 2:1.

The preceding example illustrates that the compression achieved in predictive coding is related directly to the entropy reduction that results from mapping an input image into a prediction error sequence, often called a *prediction residual*. Because spatial redundancy is removed by the prediction and differencing process, the probability density function of the prediction residual is, in general, highly peaked at zero, and characterized by a relatively small (in comparison to the input intensity distribution) variance. In fact, it is often modeled by a zero mean uncorrelated Laplacian PDF

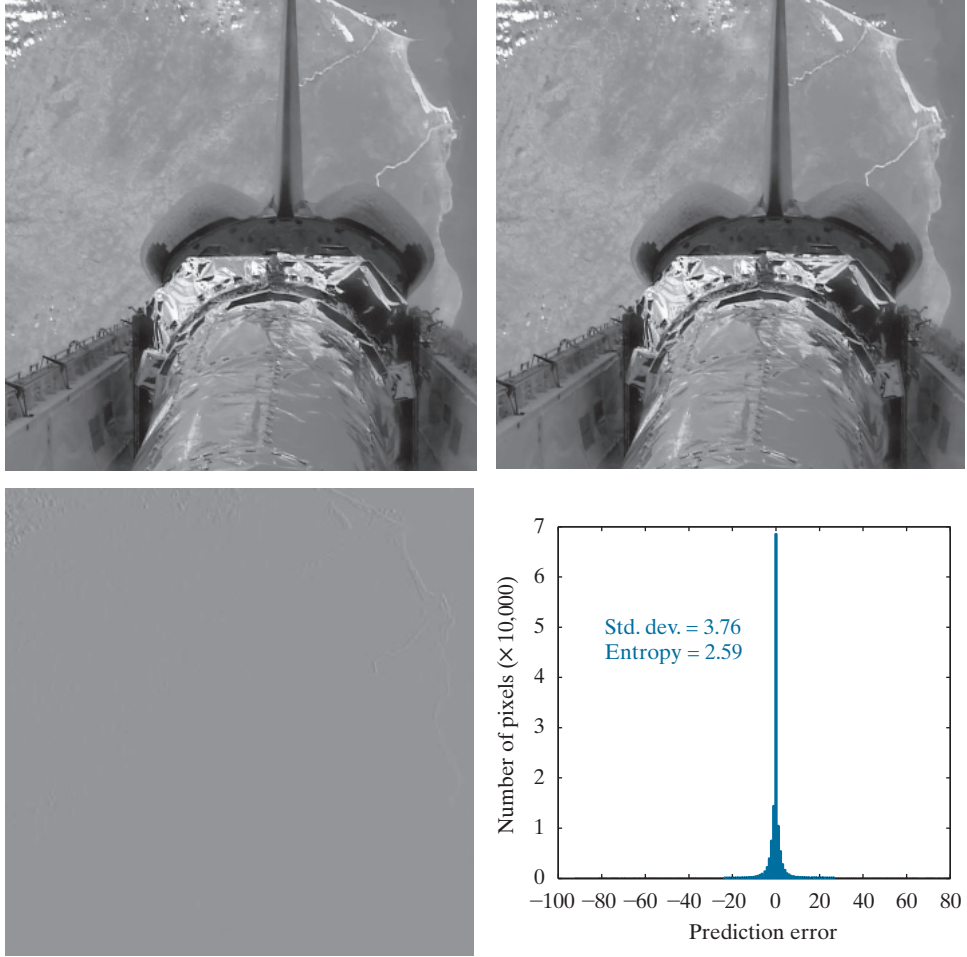
$$p_e(e) = \frac{1}{\sqrt{2}\sigma_e} e^{\frac{-\sqrt{2}|e|}{\sigma_e}} \quad (8-34)$$

where σ_e is the standard deviation of e .

a	b
c	d

FIGURE 8.32

(a) and (b) Two views of Earth from an orbiting space shuttle video. (c) The prediction error image resulting from Eq. (8-35). (d) A histogram of the prediction error. (Original images courtesy of NASA.)

**EXAMPLE 8.20: Predictive coding and temporal redundancy.**

The image in Fig. 8.31(a) is a portion of a frame of NASA video in which the Earth is moving from left to right with respect to a stationary camera attached to the space shuttle. It is repeated in Fig. 8.32(b), along with its immediately preceding frame in Fig. 8.32(a). Using the first-order linear predictor

$$\hat{f}(x, y, t) = \text{round}[\alpha f(x, y, t-1)] \quad (8-35)$$

with $\alpha = 1$, the intensities of the pixels in Fig. 8.32(b) can be predicted from the corresponding pixels in (a). Figure 8.34(c) is the resulting prediction residual image, $e(x, y, t) = f(x, y, t) - \hat{f}(x, y, t)$. Figure 8.31(d) is the histogram of $e(x, y, t)$. Note there is very little prediction error. The standard deviation of the error is much smaller than in the previous example: 3.76 bits/pixel as opposed to 15.58 bits/pixel. In addition, the entropy of the prediction error [computed using Eq. (8-7)] has decreased from 3.99 to 2.59 bits/pixel. (Recall again that the variable-length encoded prediction error is the compressed

image.) By variable-length coding the resulting prediction residual, the original image is compressed by approximately 8/2.59 or 3.1:1, a 50% improvement over the 2:1 compression obtained using the spatially oriented previous pixel predictor in Example 8.19.

MOTION COMPENSATED PREDICTION RESIDUALS

As you saw in Example 8.20, successive frames in a video sequence often are very similar. Coding their differences can reduce temporal redundancy and provide significant compression. However, when a sequence of frames contains rapidly moving objects—or involves camera zoom and pan, sudden scene changes, or fade-ins and fade-outs—the similarity between neighboring frames is reduced, and compression is affected negatively. That is, like most compression techniques (see Example 8.5), temporally based predictive coding works best with certain kinds of inputs, namely, a sequence of images with significant temporal redundancy. When used on images with little temporal redundancy, data expansion can occur. Video compression systems avoid the problem of data expansion in two ways:

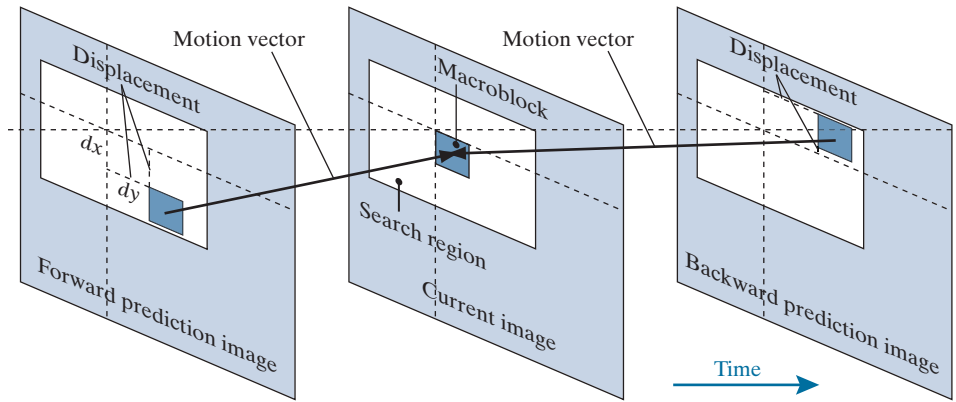
1. By tracking object movement and compensating for it during the prediction and differencing process.
2. By switching to an alternate coding method when there is insufficient *inter-frame* correlation (similarity between frames) to make predictive coding advantageous.

The first of these, called *motion compensation*, is the subject of the remainder of this section. Before proceeding, however, we should note that when there is insufficient interframe correlation to make predictive coding effective, the second problem is typically addressed using a block-oriented 2-D transform, like JPEG's DCT-based coding (see the previous section). Frames compressed in this way (i.e., without a prediction residual) are called *intraframes* or *Independent frames (I-frames)*. They can be decoded without access to other frames in the video to which they belong. I-frames usually resemble JPEG encoded images, and are ideal starting points for the generation of prediction residuals. Moreover, they provide a high degree of random access, ease of editing, and resistance to the propagation of transmission error. As a result, all standards require the periodic insertion of I-frames into the compressed video codestream.

Figure 8.33 illustrates the basics of motion-compensated predictive coding. Each video frame is divided into non-overlapping rectangular regions (typically of size 4×4 to 16×16) called *macroblocks*. (Only one macroblock is shown in Fig. 8.33.) The “movement” of each macroblock with respect to its “most likely” position in the previous (or subsequent) video frame, called the *reference frame*, is encoded in a *motion vector*. The vector describes the motion by defining the horizontal and vertical *displacement* from the “most likely” position. The displacements typically are specified to the nearest pixel, $\frac{1}{2}$ pixel, or $\frac{1}{4}$ pixel precision. If subpixel precision is used, the predictions must be interpolated [e.g., using bilinear interpolation (see Section 2.4)] from a combination of pixels in the reference frame. An encoded frame that is based on the previous frame (a *forward prediction* in Fig. 8.33) is called a *Pre-*

The “most likely” position is the one that minimizes an error measure between the reference macroblock and the macroblock being encoded. The two blocks do not have to be representations of the same object, but they must minimize the error measure.

FIGURE 8.33
Macroblock
motion specifica-
tion.



dictive frame (*P-frame*); one that is based on the subsequent frame (a *backward prediction* in Fig. 8.33) is called a *Bidirectional frame* (*B-frame*). B-frames require the compressed codestream to be reordered so that frames are presented to the decoder in the proper decoding sequence, rather than the natural display order.

As you might expect, *motion estimation* is the key component of motion compensation. During motion estimation, the motion of objects is measured and encoded into motion vectors. The search for the “best” motion vector requires that a criterion of optimality be defined. For example, motion vectors may be selected on the basis of maximum correlation or minimum error between macroblock pixels and the predicted pixels (or interpolated pixels for sub-pixel motion vectors) from the chosen reference frame. One of the most commonly used error measures is *mean absolute distortion* (*MAD*)

$$MAD(x, y) = \frac{1}{mn} \sum_{i=0}^{m-1} \sum_{j=0}^{n-1} |f(x+i, y+j) - p(x+i+dx, y+j+dy)| \quad (8-36)$$

where x and y are the coordinates of the upper-left pixel of the $m \times n$ macroblock being coded, dx and dy are displacements from the reference frame as shown in Fig. 8.33, and p is an array of predicted macroblock pixel values. For sub-pixel motion vector estimation, p is interpolated from pixels in the reference frame. Typically, dx and dy must fall within a limited search region (see Fig. 8.33) around each macroblock. Values from ± 8 to ± 64 pixels are common, and the horizontal search area is often slightly larger than the vertical area. A more computationally efficient error measure, called the *sum of absolute distortions* (*SAD*), omits the $1/mn$ factor in Eq. (8-36).

Given a selection criterion like that of Eq. (8-36), motion estimation is performed by searching for the dx and dy that minimize $MAD(x, y)$ over the allowed range of motion vector displacements, including subpixel displacements. This process often is called *block matching*. An exhaustive search guarantees the best possible result, but is computationally expensive because every possible motion must be tested over the entire displacement range. For 16×16 macroblocks and a ± 32 pixel displacement

range (not out of the question for action films and sporting events), 4225 16×16 *MAD* calculations must be performed for each macroblock in a frame when integer displacement precision is used. If $\frac{1}{2}$ or $\frac{1}{4}$ pixel precision is desired, the number of calculations is multiplied by a factor of 4 or 16, respectively. Fast search algorithms can reduce the computational burden, but may or may not yield optimal motion vectors. A number of fast block-based motion estimation algorithms have been proposed and studied in the literature (see Furht et al. [1997] or Mitchell et al. [1997]).

EXAMPLE 8.21: Motion compensated prediction.

Figures 8.34(a) and (b) were taken from the same NASA video sequence used in Examples 8.19 and 8.20. Figure 8.34(b) is identical to Figs. 8.31(a) and 8.32(b); Fig. 8.34(a) is the corresponding section of a frame occurring thirteen frames earlier. Figure 8.34(c) is the difference between the two frames, scaled to the full intensity range. Note that the difference is 0 in the area of the stationary (with respect to the camera) space shuttle, but there are significant differences in the remainder of the image due to the relative motion of the Earth. The standard deviation of the prediction residual in Fig. 8.34(c) is 12.73 intensity levels; its entropy [using Eq. (8-7)] is 4.17 bits/pixel. The maximum compression achievable, when variable-length coding the prediction residual, is $C = 8/4.17 = 1.92$.

Figure 8.34(d) shows a motion compensated prediction residual with a much lower standard deviation (5.62 as opposed to 12.73 intensity levels) and slightly lower entropy (3.04 vs. 4.17 bits/pixel). The entropy was computed using Eq. (8-7). If the prediction residual in Fig. 8.34(d) is variable-length coded, the resulting compression ratio is $C = 8/3.04 = 2.63$. To generate this prediction residual, we divided Fig. 8.34(b) into non-overlapping 16×16 macroblocks and compared each macroblock against every 16×16 region in Fig. 8.34(a)—the reference frame—that fell within ± 16 pixels of the macroblock's position in (b). We then used Eq. (8-36) to determine the best match by selecting displacement (dx, dy) with the lowest *MAD*. The resulting displacements are the x and y components of the motion vectors shown in Fig. 8.34(e). The white dots in the figure are the heads of the motion vectors; they indicate the upper-left-hand corner of the coded macroblocks. As you can see from the pattern of the vectors, the predominant motion in the image is from left to right. In the lower portion of the image, which corresponds to the area of the space shuttle in the original image, there is no motion, and therefore no motion vectors displayed. Macroblocks in this area are predicted from similarly located (i.e., the corresponding) macroblocks in the reference frame. Because the motion vectors in Fig. 8.34(e) are highly correlated, they can be variable-length coded to reduce their storage and transmission requirements

The visual difference between Figs. 8.34(c) and 8.35(a) is due to scaling. The image in Fig. 8.35(a) has been scaled to match Figs. 8.35(b)–(d).

Figure 8.35 illustrates the increased prediction accuracy that is possible with sub-pixel motion compensation. Figure 8.35(a) is repeated from Fig. 8.34(c) and included as a point of reference; it shows the prediction error that results without motion compensation. The images in Figs. 8.35(b), (c), and (d) are motion compensated prediction residuals. They are based on the same two frames that were used in Example 8.21 and computed with macroblock displacements to 1, $\frac{1}{2}$, and $\frac{1}{4}$ pixel resolution (i.e., precision), respectively. Macroblocks of size 8×8 were used; displacements were limited to ± 8 pixels.

The most significant visual difference between the prediction residuals in Fig. 8.35 is the number and size of intensity peaks and valleys—their darkest and lightest areas of intensity. The $\frac{1}{4}$ pixel residual in Fig. 8.35(d) is the “flattest” of the four

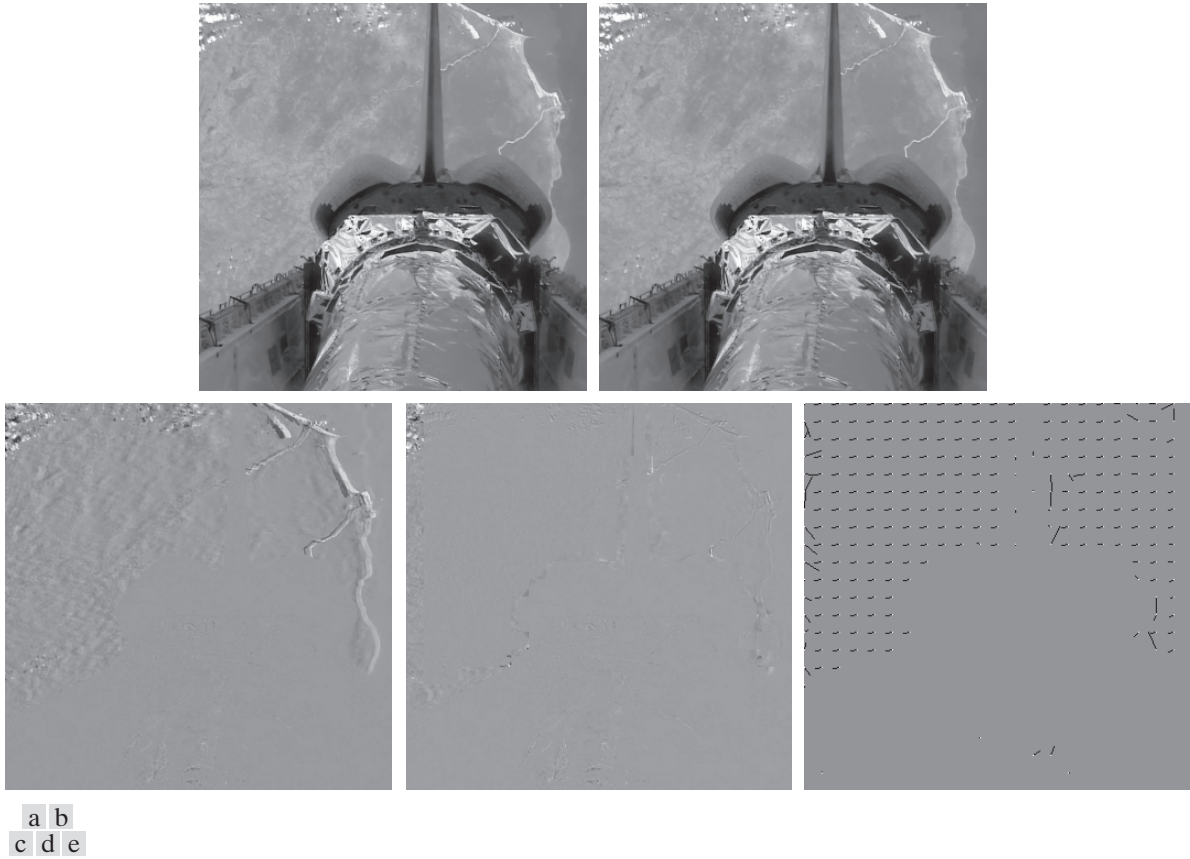


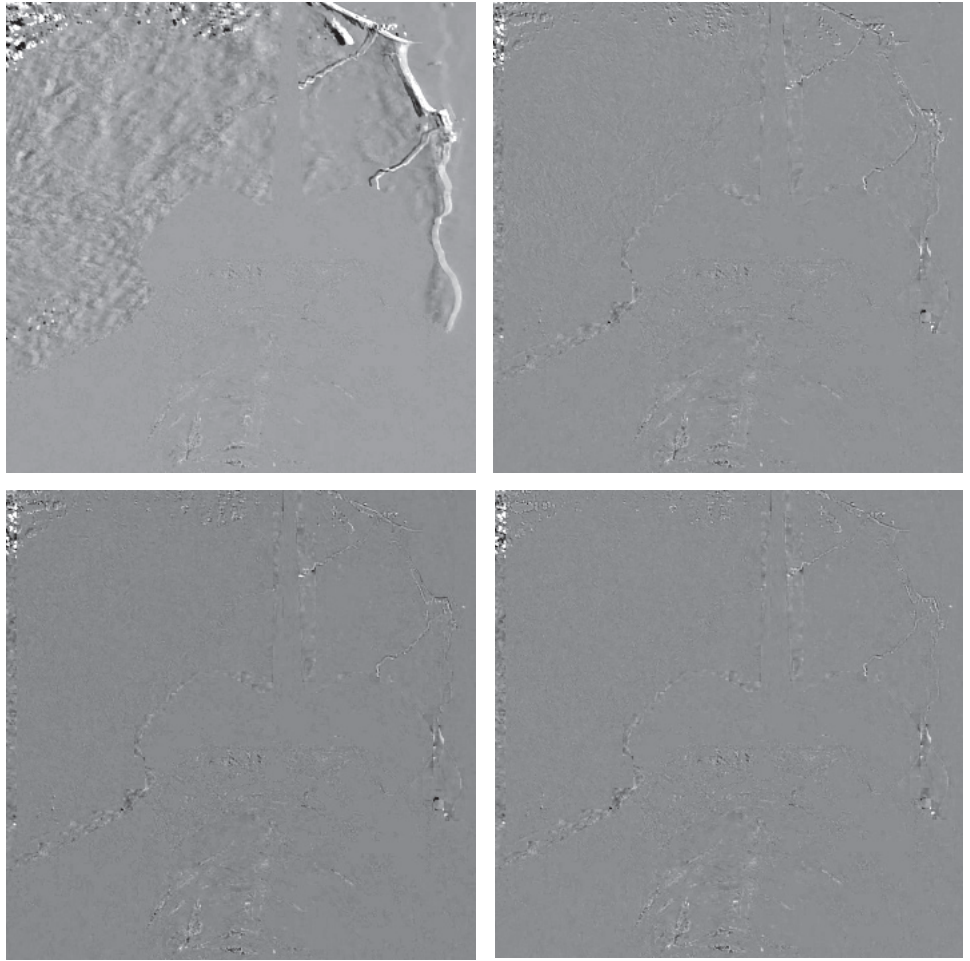
FIGURE 8.34 (a) and (b) Two views of Earth that are thirteen frames apart in an orbiting space shuttle video. (c) A prediction error image without motion compensation. (d) The prediction residual with motion compensation. (e) The motion vectors associated with (d). The white dots in (e) represent the arrow heads of the motion vectors that are depicted. (Original images courtesy of NASA.)

images, with the fewest excursions to black or white. As would be expected, it has the narrowest histogram. The standard deviations of the prediction residuals in Figs. 8.35(a) through (d) decrease as motion vector precision increases from 12.7 to 4.4, 4, and 3.8 pixels, respectively. The entropies of the residuals, as determined using Eq. (8-7), are 4.17, 3.34, 3.35, and 3.34 bits/pixel, respectively. Thus, the motion compensated residuals contain about the same amount of information, despite the fact that the residuals in Figs. 8.35(c) and (d) use additional bits to accommodate $\frac{1}{2}$ and $\frac{1}{4}$ pixel interpolation. Finally, we note that there is an obvious strip of increased prediction error on the left side of each motion compensated residual. This is due to the left-to-right motion of the Earth, which introduces new or previously unseen areas of the Earth's terrain into the left side of each image. Because these areas are absent from the previous frames, they cannot be accurately predicted, regardless of the precision used to compute motion vectors.

a	b
c	d

FIGURE 8.35

Sub-pixel motion compensated prediction residuals: (a) without motion compensation; (b) single pixel precision; (c) $\frac{1}{2}$ pixel precision; and (d) $\frac{1}{4}$ pixel precision. (All prediction errors have been scaled to the full intensity range and then multiplied by 2 to increase their visibility.)



Motion estimation is a computationally demanding task. Fortunately, only the encoder must estimate macroblock motion. Given the motion vectors of the macroblocks, the decoder simply accesses the areas of the reference frames that were used in the encoder to form the prediction residuals. Because of this, motion estimation is not included in most video compression standards. Compression standards focus on the decoder, placing constraints on macroblock dimensions, motion vector precision, horizontal and vertical displacement ranges, and the like. Table 8.12 gives the key predictive coding parameters of some of the most important video compression standards. Note that most of the standards use an 8×8 DCT for I-frame encoding, but specify a larger area (i.e., 16×16 macroblock) for motion compensation. In addition, even the P- and B-frame prediction residuals are transform coded due to the effectiveness of DCT coefficient quantization. Finally, we note that the H.264 and MPEG-4 AVC standards support intraframe predictive coding (in I-frames) to reduce spatial redundancy.

TABLE 8.12
Predictive coding in video compression standards.

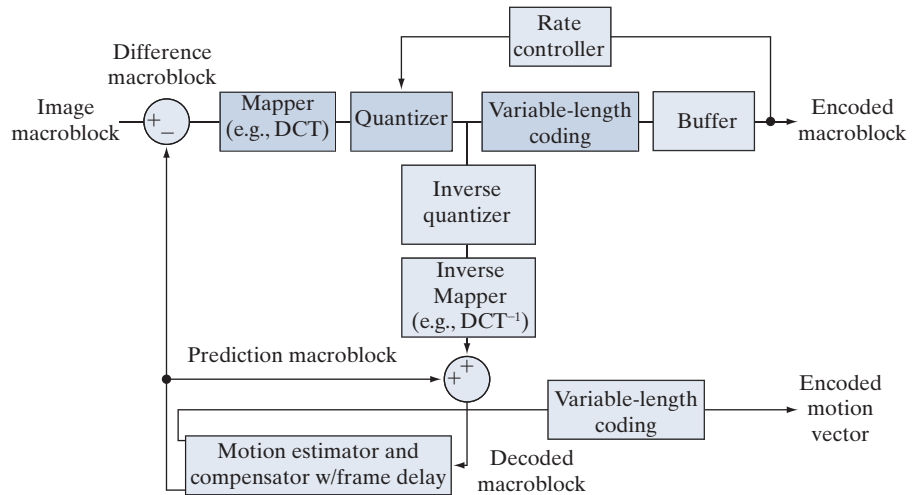
Parameter	H.261	MPEG-1	H.262 MPEG-2	H.263	MPEG-4	VC-1 WMV-9	H.264 MPEG-4 AVC
Motion vector precision	1	½	½	½	¼	¼	¼
Macroblock sizes	16 × 16	16 × 16	16 × 16 16 × 8	16 × 16 8 × 8	16 × 16 8 × 8	16 × 16 8 × 8	16 × 16 16 × 8 8 × 8 8 × 4 4 × 8 4 × 4
Transform	8 × 8 DCT	8 × 8 DCT	8 × 8 DCT	8 × 8 DCT	8 × 8 DCT	8 × 8 8 × 4 4 × 8 4 × 4 Integer DCT	4 × 4 8 × 8 Integer
Interframe predictions	P	P, B	P, B	P, B	P, B	P, B	P, B
I-frame intra-predictions	No	No	No	No	No	No	Yes

Figure 8.36 shows a typical motion compensated video encoder. It exploits redundancies within and between adjacent video frames, motion uniformity between frames, and the psychovisual properties of the human visual system. We can think of the input to the encoder as sequential macroblocks of video. For color video, each macroblock is composed of a luminance block and two chrominance blocks. Because the eye has far less spatial acuity for color than for luminance, the chrominance blocks often are sampled at half the horizontal and vertical resolution of the luminance block. The dark blue elements in the figure parallel the transformation, quantization, and variable-length coding operations of a JPEG encoder. The principal difference is the input, which may be a conventional macroblock of image data (for I-frames), or the difference between a conventional macroblock and a prediction of it based on previous and/or subsequent video frames (for P- and B-frames). The encoder includes an *inverse quantizer* and *inverse mapper* (e.g., inverse DCT) so that its predictions match those of the complementary decoder. Also, it is designed to produce compressed bit streams that match the capacity of the intended video channel. To accomplish this, the quantization parameters are adjusted by a *rate controller* as a function of the occupancy of an output *buffer*. As the buffer becomes fuller, the quantization is made coarser, so fewer bits stream into the buffer.

Quantization as defined earlier in the chapter is irreversible. The “inverse quantizer” in Fig. 8.36 does not prevent information loss.

FIGURE 8.36

A typical motion compensated video encoder.



EXAMPLE 8.22: Video compression example.

We conclude our discussion of motion compensated predictive coding with an example illustrating the kind of compression that is possible with modern video compression methods. Figure 8.37 shows fifteen frames of a 1 minute HD (1280×720) full-color NASA video, parts of which have been used throughout this section. Although the images shown are monochrome, the video is a sequence of 1,829 full-color frames. Note that there are a variety of scenes, a great deal of motion, and multiple fade effects. For example, the video opens with a 150 frame fade-in from black, which includes frames 21 and 44 in Fig. 8.37, and concludes with a fade sequence containing frames 1595, 1609, and 1652 in Fig. 8.37, followed by a final fade to black. There are also several abrupt scene changes, like the change involving frames 1303 and 1304 in Fig. 8.37.

An H.264 compressed version of the NASA video stored as a Quicktime file (see Table 8.5) requires 44.56 MB of storage, plus another 1.39 MB for the associated audio. The video quality is excellent. About 5 GB of data would be needed to store the video frames as uncompressed full-color images. It should be noted that the video contains sequences involving both rotation and scale change (e.g., the sequence including frames 959, 1023, and 1088 in Fig. 8.37). The discussion in this section, however, has been limited to translation alone. (See the book website for the NASA video segment used in this example.)

LOSSY PREDICTIVE CODING

In this section, we add a quantizer to the lossless predictive coding model introduced earlier, and examine the trade-off between reconstruction accuracy and compression performance within the context of spatial predictors. As Fig. 8.38 shows, the quantizer, which replaces the nearest integer function of the error-free encoder, is inserted between the symbol encoder and the point at which the prediction error is formed. It maps the prediction error into a limited range of outputs, denoted $\hat{e}(n)$, which establish the amount of compression and distortion that occurs.

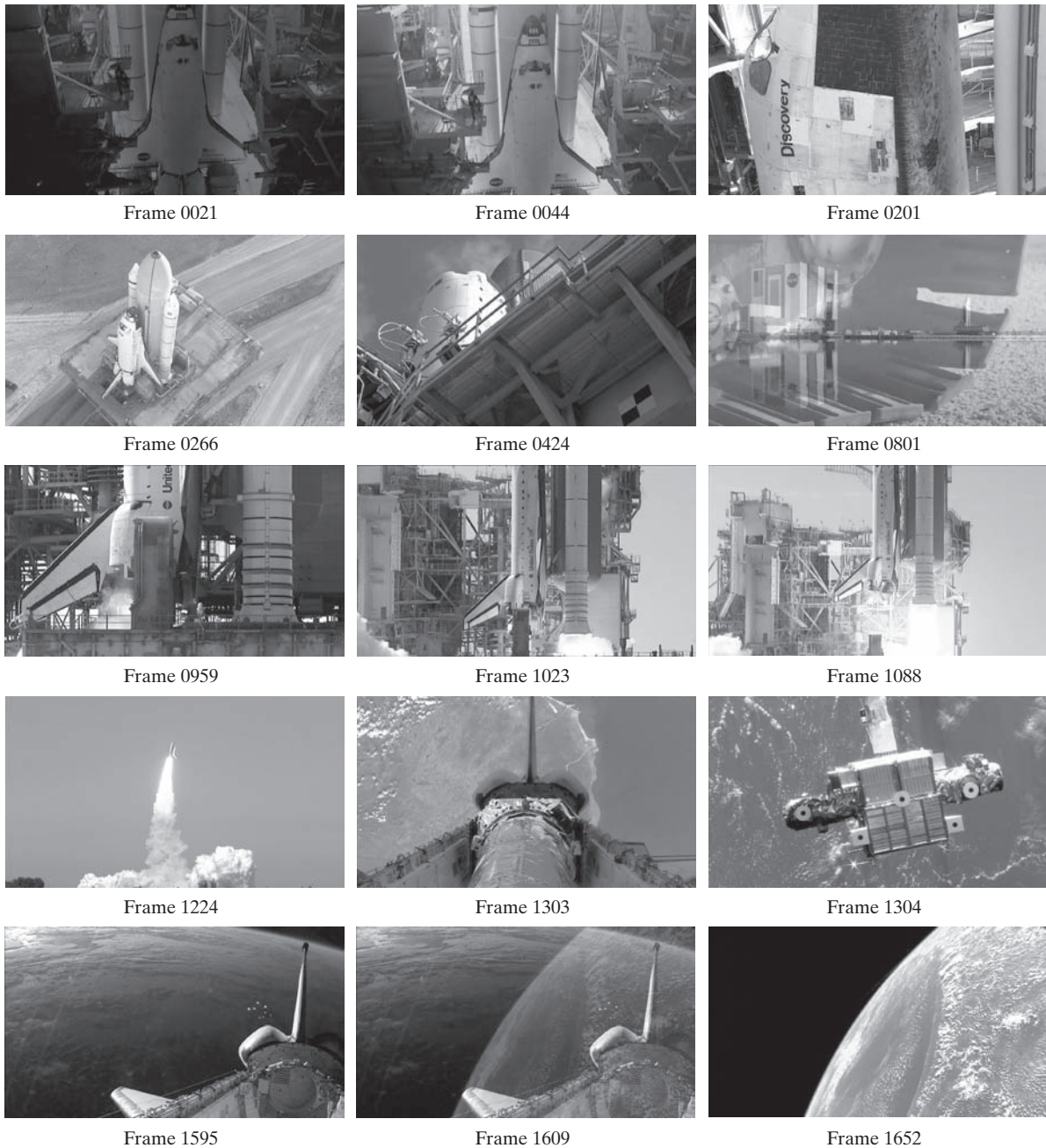


FIGURE 8.37 Fifteen frames from an 1829-frame, 1-minute NASA video. The original video is in HD full color. (Courtesy of NASA.)

In order to accommodate the insertion of the quantization step, the error-free encoder of Fig. 8.30(a) must be altered so the predictions generated by the encoder and decoder are equivalent. As Fig. 8.38(a) shows, this is accomplished by placing the lossy encoder's predictor within a feedback loop, where its input, denoted as $\hat{f}(n)$, is generated as a function of past predictions and the corresponding quantized errors. That is,

$$\hat{f}(n) = \dot{e}(n) + \hat{f}(n) \quad (8-37)$$

where $\hat{f}(n)$ is as defined earlier. This closed loop configuration prevents error buildup at the decoder's output. Note in Fig. 8.38(b) that the output of the decoder is given also by Eq. (8-37).

EXAMPLE 8.23: Delta modulation.

Delta modulation (DM) is a simple but well-known form of lossy predictive coding in which the predictor and quantizer are defined as

$$\hat{f}(n) = \alpha \hat{f}(n-1) \quad (8-38)$$

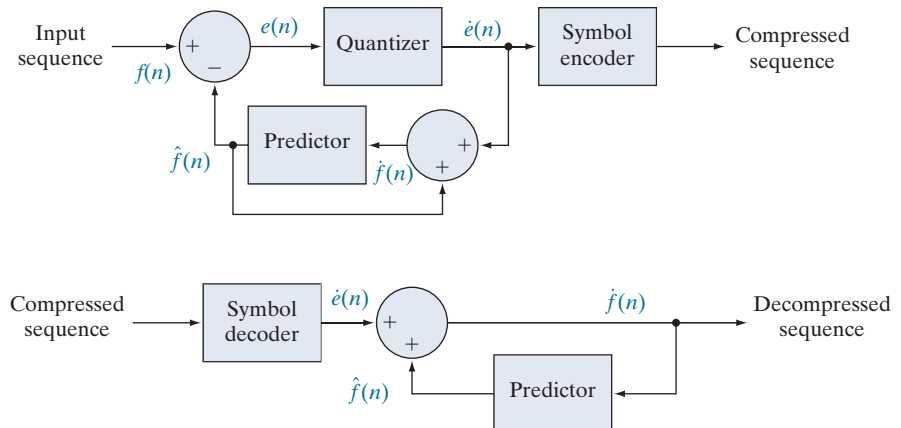
and

$$\dot{e}(n) = \begin{cases} +\zeta & \text{for } e(n) > 0 \\ -\zeta & \text{otherwise} \end{cases} \quad (8-39)$$

where α is a prediction coefficient (normally less than 1), and ζ is a positive constant. The output of the quantizer, $\dot{e}(n)$, can be represented by a single bit [see Fig. 8.39(a)], so the symbol encoder of Fig. 8.38(a) can utilize a 1-bit fixed-length code. The resulting DM code rate is 1 bit/pixel.

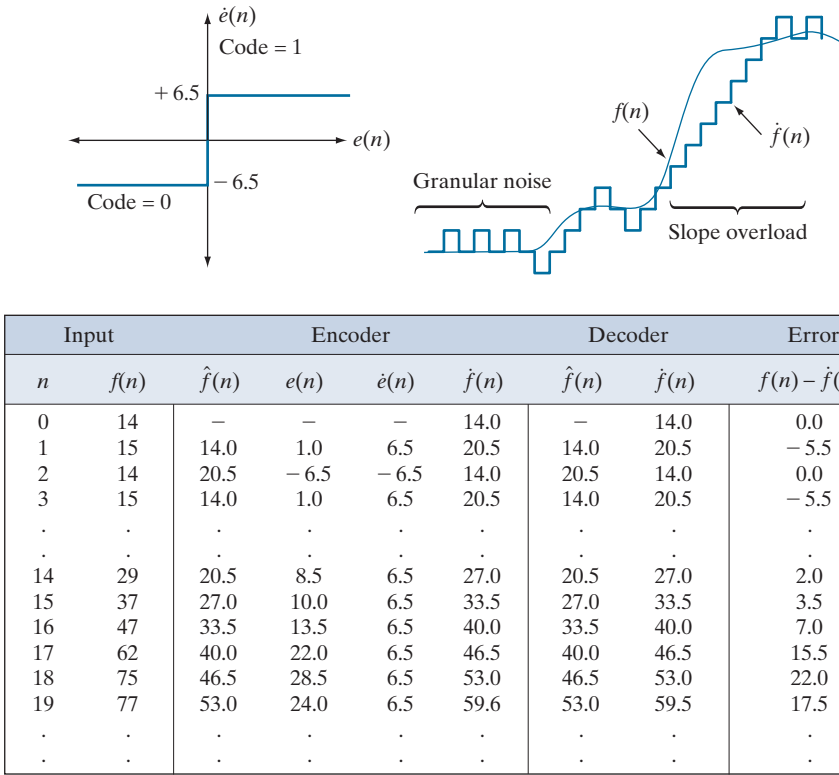
Figure 8.39(c) illustrates the mechanics of the delta modulation process, where the calculations needed to compress and reconstruct input sequence {14, 15, 14, 15, 13, 15, 15, 14, 20, 26, 27, 28, 27, 27, 29, 37, 47, 62, 75, 77, 78, 79, 80, 81, 81, 82, 82} with $\alpha = 1$ and $\zeta = 6.5$ are tabulated. The process begins with the

FIGURE 8.38
A lossy predictive coding model:
(a) encoder;
(b) decoder.



a b
c

FIGURE 8.39
An example of delta modulation.



error-free transfer of the first input sample to the decoder. With the initial condition $\hat{f}(0) = f(0) = 14$ established at both the encoder and decoder, the remaining outputs can be computed by repeatedly evaluating Eqs. (8-38), (8-29), (8-39), and (8-37). Thus, when $n = 1$, for example, $\hat{f}(1) = (1)(14) = 14$, $e(1) = 15 - 14 = 1$, $\hat{e}(1) = +6.5$ (because $e(1) > 0$), $\hat{f}(1) = 6.4 + 14 = 20.5$, and the resulting reconstruction error is $(15 - 20.5)$, or -5.5 .

Figure 8.39(b) graphically shows the tabulated data in Fig. 8.39(c). Both the input and completely decoded output $[f(n) \text{ and } \hat{f}(n)]$ are shown. Note that in the rapidly changing area from $n = 14$ to 19 , where ζ was too small to represent the input's largest changes, a distortion known as *slope overload* occurs. Moreover, when ζ was too large to represent the input's smallest changes, as in the relatively smooth region from $n = 0$ to $n = 7$, *granular noise* appears. In images, these two phenomena lead to blurred object edges and grainy or noisy surfaces (that is, distorted smooth areas).

The distortions noted in the preceding example are common to all forms of lossy predictive coding. The severity of these distortions depends on a complex set of interactions between the quantization and prediction methods employed. Despite these interactions, the predictor normally is designed with the assumption of no quantization error, and the quantizer is designed to minimize its own error. That is, the predictor and quantizer are designed independently of each other.

OPTIMAL PREDICTORS

In many predictive coding applications, the predictor is chosen to minimize the encoder's mean-squared prediction error

The notation $E\{\cdot\}$ denotes the statistical expectation operator.

$$E\{e^2(n)\} = E\{[f(n) - \hat{f}(n)]^2\} \quad (8-40)$$

subject to the constraint that

$$\dot{f}(n) = \dot{e}(n) + \dot{\hat{f}}(n) \approx e(n) + \dot{\hat{f}}(n) = f(n) \quad (8-41)$$

and

$$\hat{f}(n) = \sum_{i=1}^m \alpha_i f(n-i) \quad (8-42)$$

That is, the optimization criterion is minimal mean-squared prediction error, the quantization error is assumed to be negligible $[\dot{e}(n) \approx e(n)]$, and the prediction is constrained to a linear combination of m previous samples. These restrictions are not essential, but they considerably simplify the analysis and, at the same time, decrease the computational complexity of the predictor. The resulting predictive coding approach is referred to as *differential pulse code modulation* (DPCM).

In general, the optimal predictor for a non-Gaussian sequence is a nonlinear function of the samples used to form the estimate.

Under these conditions, the optimal predictor design problem is reduced to the relatively straightforward exercise of selecting the m prediction coefficients that minimize the expression

$$E\{e^2(n)\} = E\left\{\left[f(n) - \sum_{i=1}^m \alpha_i f(n-i)\right]^2\right\} \quad (8-43)$$

Differentiating Eq. (8-43) with respect to each coefficient, equating the derivatives to zero, and solving the resulting set of simultaneous equations under the assumption that $f(n)$ has mean zero and variance σ^2 yields

$$\alpha = \mathbf{R}^{-1} \mathbf{r} \quad (8-44)$$

where $\mathbf{R}^{-1}$ is the inverse of the $m \times m$ autocorrelation matrix

$$\mathbf{R} = \begin{bmatrix} E\{f(n-1)f(n-1)\} & E\{f(n-1)f(n-2)\} & \cdots & E\{f(n-1)f(n-m)\} \\ E\{f(n-2)f(n-1)\} & \vdots & \cdots & \vdots \\ \vdots & \vdots & \cdots & \vdots \\ \vdots & \vdots & \cdots & \vdots \\ \vdots & \vdots & \cdots & \vdots \\ E\{f(n-m)f(n-1)\} & E\{f(n-m)f(n-2)\} & \cdots & E\{f(n-m)f(n-m)\} \end{bmatrix} \quad (8-45)$$

and $\mathbf{r}$ and $\boldsymbol{\alpha}$ are the m -element vectors

$$\mathbf{r} = \begin{bmatrix} E\{f(n)f(n-1)\} \\ E\{f(n)f(n-2)\} \\ \vdots \\ E\{f(n)f(n-m)\} \end{bmatrix} \quad \boldsymbol{\alpha} = \begin{bmatrix} \alpha_1 \\ \alpha_2 \\ \vdots \\ \alpha_m \end{bmatrix} \quad (8-46)$$

Thus for any input sequence, the coefficients that minimize Eq. (8-43) can be determined via a series of elementary matrix operations. Moreover, the coefficients depend only on the autocorrelations of the samples in the original sequence. The variance of the prediction error that results from the use of these optimal coefficients is

$$\sigma_e^2 = \sigma^2 - \boldsymbol{\alpha}^T \mathbf{r} = \sigma^2 - \sum_{i=1}^m E\{f(n)f(n-i)\} \alpha_i \quad (8-47)$$

Although the mechanics of evaluating Eq. (8-44) are quite simple, computation of the autocorrelations needed to form $\mathbf{R}$ and $\mathbf{r}$ is so difficult in practice that *local* predictions (those in which the prediction coefficients are computed for each input sequence) are almost never used. In most cases, a set of *global* coefficients is computed by assuming a simple input model and substituting the corresponding autocorrelations into Eqs. (8-45) and (8-46). For instance, when a 2-D Markov image source (see Section 8.1) with separable autocorrelation function

$$E\{f(x,y)f(x-i,y-j)\} = \sigma^2 \rho_v^i \rho_h^j \quad (8-48)$$

and generalized fourth-order linear predictor

$$\begin{aligned} \hat{f}(x,y) = & \alpha_1 f(x,y-1) + \alpha_2 f(x-1,y-1) \\ & + \alpha_3 f(x-1,y) + \alpha_4 f(x-1,y+1) \end{aligned} \quad (8-49)$$

are assumed, the resulting optimal coefficients (Jain [1989]) are

$$\alpha_1 = \rho_h \quad \alpha_2 = -\rho_v \rho_h \quad \alpha_3 = \rho_v \quad \alpha_4 = 0 \quad (8-50)$$

where ρ_h and ρ_v are the horizontal and vertical correlation coefficients, respectively, of the image under consideration.

Finally, the sum of the prediction coefficients in Eq. (8-42) is normally required to be less than or equal to one. That is,

$$\sum_{i=1}^m \alpha_i \leq 1 \quad (8-51)$$

This restriction is made to ensure that the output of the predictor falls within the allowed range of the input, and to reduce the impact of transmission noise [which generally is seen as horizontal streaks in reconstructed images when the input to Fig. 8.38(a) is an image]. Reducing the DPCM decoder's susceptibility to input noise is important, because a single error (under the right circumstances) can propagate to all future outputs. That is, the decoder's output may become unstable. Further restricting Eq. (8-51) to be strictly less than 1 confines the impact of an input error to a small number of outputs.

EXAMPLE 8.24: Comparison of prediction techniques.

Consider the prediction error that results from DPCM coding the monochrome image of Fig. 8.9(a) under the assumption of zero quantization error and with each of four predictors:

$$\hat{f}(x, y) = 0.97f(x, y - 1) \quad (8-52)$$

$$\hat{f}(x, y) = 0.5f(x, y - 1) + 0.5f(x - 1, y) \quad (8-53)$$

$$\hat{f}(x, y) = 0.75f(x, y - 1) + 0.75f(x - 1, y) - 0.5f(x - 1, y - 1) \quad (8-54)$$

$$\hat{f}(x, y) = \begin{cases} 0.97f(x, y - 1) & \text{if } \Delta h \leq \Delta v \\ 0.97f(x - 1, y) & \text{otherwise} \end{cases} \quad (8-55)$$

where $\Delta h = |f(x - 1, y) - f(x - 1, y - 1)|$ and $\Delta v = |f(x, y - 1) - f(x - 1, y - 1)|$ denote the horizontal and vertical gradients at point (x, y) . Equations (8-52) through (8-55) define a relatively robust set of α_i that provide satisfactory performance over a wide range of images. The adaptive predictor of Eq. (8-55) is designed to improve edge rendition by computing a local measure of the directional properties of an image (Δh and Δv), and selecting a predictor specifically tailored to the measured behavior.

Figures 8.40(a) through (d) show the prediction error images that result from using the predictors of Eqs. (8-52) through (8-55). Note that the visually perceptible error decreases as the order of the predictor increases.[†] The standard deviations of the prediction errors follow a similar pattern. They are 11.1, 9.8, 9.1, and 9.7 intensity levels, respectively.

OPTIMAL QUANTIZATION

The staircase quantization function $t = q(s)$ in Fig. 8.41 is an odd function of s [that is, $q(-s) = -q(s)$] that can be described completely by the $L/2$ values of s_i and t_i shown in the first quadrant of the graph. These break points define function discontinuities, and are called the *decision* and *reconstruction levels* of the quantizer. As a matter of convention, s is considered to be mapped to t_i if it lies in the half-open interval $(s_i, s_{i+1}]$.

The quantizer design problem is to select the best s_i and t_i for a particular optimization criterion and input probability density function $p(s)$. If the optimization

[†]Predictors that use more than three or four previous pixels provide little compression gain for the added predictor complexity (Habibi [1971]).

a	b
c	d

FIGURE 8.40

A comparison of four linear prediction techniques.



criterion, which could be either a statistical or psychovisual measure,[†] is the minimization of the mean-squared quantization error (that is $E\{(s_i - t_i)^2\}$) and $p(s)$ is an even function, the conditions for minimal error (Max [1960]) are

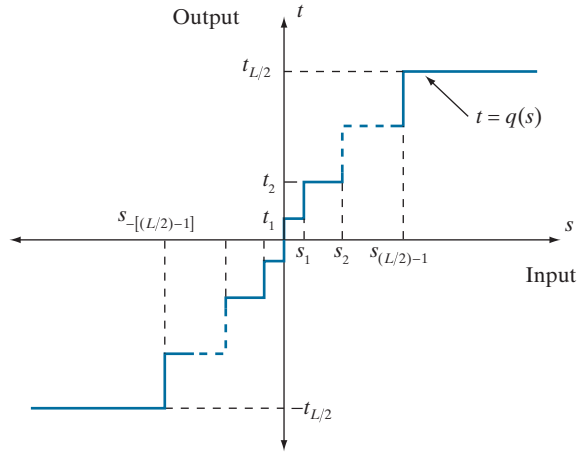
$$\int_{s_{i-1}}^{s_i} (s - t_i) p(s) ds = 0 \quad i = 1, 2, \dots, \frac{L}{2} \quad (8-56)$$

$$s_i = \begin{cases} 0 & i = 0 \\ \frac{t_i + t_{i+1}}{2} & i = 1, 2, \dots, \frac{L}{2} - 1 \\ \infty & i = \frac{L}{2} \end{cases} \quad (8-57)$$

[†]See Netravali [1977] and Limb for more on psychovisual measures.

FIGURE 8.41

A typical quantization function.



and

$$s_{-i} = -s_i \quad t_{-i} = -t_i \quad (8-58)$$

Equation (8-56) indicates that the reconstruction levels are the centroids of the areas under $p(s)$ over the specified decision intervals, whereas Eq. (8-57) indicates that the decision levels are halfway between the reconstruction levels. Equation (8-58) is a consequence of the fact that q is an odd function. For any L , the s_i and t_i that satisfy Eqs. (8-56) through (8-58) are optimal in the mean-squared error sense; the corresponding quantizer is called an L -level *Lloyd-Max* quantizer.

Table 8.13 lists the 2-, 4-, and 8-level Lloyd-Max decision and reconstruction levels for a unit variance Laplacian probability density function [see Eq. (8-34)]. Because obtaining an explicit or closed-form solution to Eqs. (8-56) through (8-58) for most nontrivial $p(s)$ is difficult, these values were generated numerically (Paez and Glisson [1972]). The three quantizers shown provide fixed output rates of 1, 2, and 3 bits/pixel, respectively. As Table 8.13 was constructed for a unit variance distribution, the reconstruction and decision levels for the case of $\sigma \neq 1$ are obtained by multiplying the tabulated values by the standard deviation of the probability density

TABLE 8.13

Lloyd-Max quantizers for a Laplacian probability density function of unit variance.

Levels	2		4		8	
	s_i	t_i	s_i	t_i	s_i	t_i
1	∞	0.707	1.102	0.395	0.504	0.222
2			∞	1.810	1.181	0.785
3					2.285	1.576
4					∞	2.994
θ	1.414		1.087		0.731	

function under consideration. The final row of the table lists the step size, θ , that simultaneously satisfies Eqs. (8-56) through (8-58) and the additional constraint that

$$t_i - t_{i-1} = s_i - s_{i-1} = \theta \quad (8-59)$$

If a symbol encoder that utilizes a variable-length code is used in the general lossy predictive encoder of Fig. 8.38(a), an *optimum uniform quantizer* of step size θ will provide a lower code rate (for a Laplacian PDF) than a fixed-length coded Lloyd-Max quantizer with the same output fidelity (O'Neil [1971]).

Although the Lloyd-Max and optimum uniform quantizers are not adaptive, much can be gained from adjusting the quantization levels based on the local behavior of an image. In theory, slowly changing regions can be finely quantized, while the rapidly changing areas are quantized more coarsely. This approach simultaneously reduces both granular noise and slope overload, while requiring only a minimal increase in code rate. The trade-off is increased quantizer complexity.

8.11 WAVELET CODING

With reference to
Tables 8.3–8.5, wavelet
coding is used in the

- JPEG-2000
compression standard.

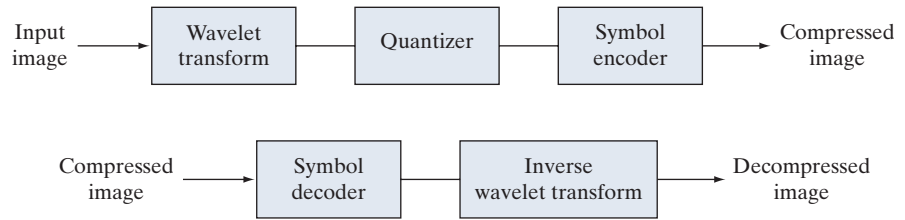
As with the block transform coding techniques presented earlier, wavelet coding is based on the idea that the coefficients of a transform that decorrelates the pixels of an image can be coded more efficiently than the original pixels themselves. If the basis functions of the transform (in this case wavelets) pack most of the important visual information into a small number of coefficients, the remaining coefficients can be quantized coarsely or truncated to zero with little image distortion.

Figure 8.42 shows a typical wavelet coding system. To encode a $2^J \times 2^J$ image, an analyzing wavelet, ψ , and minimum decomposition level, $J - P$, are selected and used to compute the discrete wavelet transform of the image. If the wavelet has a complementary scaling function φ , the fast wavelet transform (see Section 7.10) can be used. In either case, the computed transform converts a large portion of the original image to horizontal, vertical, and diagonal decomposition coefficients with zero mean and Laplacian-like probabilities. Because many of the computed coefficients carry little visual information, they can be quantized and coded to minimize intercoefficient and coding redundancy. Moreover, the quantization can be adapted to exploit any positional correlation across the P decomposition levels. One or more lossless coding methods, such as run-length, Huffman, arithmetic, and bit-plane coding, can be incorporated into the final symbol coding step. Decoding is accomplished by inverting the encoding operations, with the exception of quantization, which cannot be reversed exactly.

The principal difference between the wavelet-based system of Fig. 8.42 and the transform coding system of Fig. 8.21 is the omission of the subimage processing stages of the transform coder. Because wavelet transforms are both computationally efficient and inherently local (i.e., their basis functions are limited in duration), subdivision of the original image is unnecessary. As you will see later in this section, the removal of the subdivision step eliminates the blocking artifact that characterizes DCT-based approximations at high compression ratios.

a
b

FIGURE 8.42
A wavelet coding system:
(a) encoder;
(b) decoder.



WAVELET SELECTION

The wavelets chosen as the basis of the forward and inverse transforms in Fig. 8.42 affect all aspects of wavelet coding system design and performance. They impact directly the computational complexity of the transforms and, less directly, the system's ability to compress and reconstruct images of acceptable error. When the transforming wavelet has a companion scaling function, the transformation can be implemented as a sequence of digital filtering operations, with the number of filter taps equal to the number of nonzero wavelet and scaling vector coefficients. The ability of the wavelet to pack information into a small number of transform coefficients determines its compression and reconstruction performance.

The most widely used expansion functions for wavelet-based compression are the Daubechies wavelets and biorthogonal wavelets. The latter allow useful analysis properties, like the number of vanishing moments (see Section 7.10), to be incorporated into the decomposition filters, while important synthesis properties, like smoothness of reconstruction, are built into the reconstruction filters.

EXAMPLE 8.25: Wavelet bases in wavelet coding.

Figure 8.43 contains four discrete wavelet transforms of Fig. 8.9(a). Haar wavelets, the simplest and only discontinuous wavelets considered in this example, were used as the expansion or basis functions in Fig. 8.43(a). Daubechies wavelets, among the most popular imaging wavelets, were used in Fig. 8.43(b), and symlets, which are an extension of the Daubechies wavelets with increased symmetry, were used in Fig. 8.43(c). The Cohen-Daubechies-Feauveau wavelets employed in Fig. 8.43(d) are included to illustrate the capabilities of biorthogonal wavelets. As in previous results of this type, all detail coefficients were scaled to make the underlying structure more visible, with intensity 128 corresponding to coefficient value 0.

As you can see in Table 8.14, the number of operations involved in the computation of the transforms in Fig. 8.43 increases from 4 to 28 multiplications and additions per coefficient (for each decomposition

TABLE 8.14

Wavelet transform filter taps and zeroed coefficients when truncating the transforms in Fig. 8.43 below 1.5.

Wavelet	Filter Taps (Scaling + Wavelet)	Zeroed Coefficients
Haar	2 + 2	33.3%
Daubechies	8 + 8	40.9%
Symlet	8 + 8	41.2%
Biorthogonal	17 + 11	42.1%

a	b
c	d

FIGURE 8.43

Three-scale wavelet transforms of Fig. 8.9(a) with respect to (a) Haar wavelets, (b) Daubechies wavelets, (c) symlets, and (d) Cohen-Daubechies-Feauveau biorthogonal wavelets.



level) as you move from Fig. 8.43(a) to (d). All four transforms were computed using a fast wavelet transform (i.e., filter bank) formulation. Note that as the computational complexity (i.e., the number of filter taps) increases, the information packing performance does as well. When Haar wavelets are employed and the detail coefficients below 1.5 are truncated to zero, 33.8% of the total transform is zeroed. With the more complex biorthogonal wavelets, the number of zeroed coefficients rises to 42.1%, increasing the potential compression by almost 10%.

DECOMPOSITION LEVEL SELECTION

Another factor affecting wavelet coding computational complexity and reconstruction error is the number of transform decomposition levels. Because a P -scale fast wavelet transform involves P filter bank iterations, the number of operations in the computation of the forward and inverse transforms increases with the number of decomposition levels. Moreover, quantizing the increasingly lower-scale

coefficients that result with more decomposition levels affects increasingly larger areas of the reconstructed image. In many applications, like searching image databases or transmitting images for progressive reconstruction, the resolution of the stored or transmitted images, and the scale of the lowest useful approximations, normally determine the number of transform levels.

EXAMPLE 8.26: Decomposition levels in wavelet coding.

Table 8.15 illustrates the effect of decomposition level selection on the coding of Fig. 8.9(a) using biorthogonal wavelets and a fixed global threshold of 25. As in the previous wavelet coding example, only detail coefficients are truncated. The table lists both the percentage of zeroed coefficients and the resulting rms reconstruction errors from Eq. (8-10). Note that the initial decompositions are responsible for the majority of the data compression. There is little change in the number of truncated coefficients above three decomposition levels.

QUANTIZER DESIGN

The most important factor affecting wavelet coding compression and reconstruction error is coefficient quantization. Although the most widely used quantizers are uniform, the effectiveness of the quantization can be improved significantly by (1) introducing a larger quantization interval around zero, called a *dead zone*, or (2) adapting the size of the quantization interval from scale to scale. In either case, the selected quantization intervals must be transmitted to the decoder with the encoded image bit stream. The intervals themselves may be determined heuristically, or computed automatically based on the image being compressed. For example, a global coefficient threshold could be computed as the median of the absolute values of the first-level detail coefficients or as a function of the number of zeroes that are truncated and the amount of energy that is retained in the reconstructed image.

One measure of the energy of a digital signal is the sum of the squared samples.

EXAMPLE 8.27: Dead zone interval selection in wavelet coding.

Figure 8.44 illustrates the impact of dead zone interval size on the percentage of truncated detail coefficients for a three-scale biorthogonal wavelet-based encoding of Fig. 8.9(a). As the size of the dead zone increases, the number of truncated coefficients does as well. Above the knee of the curve (i.e., beyond 5),

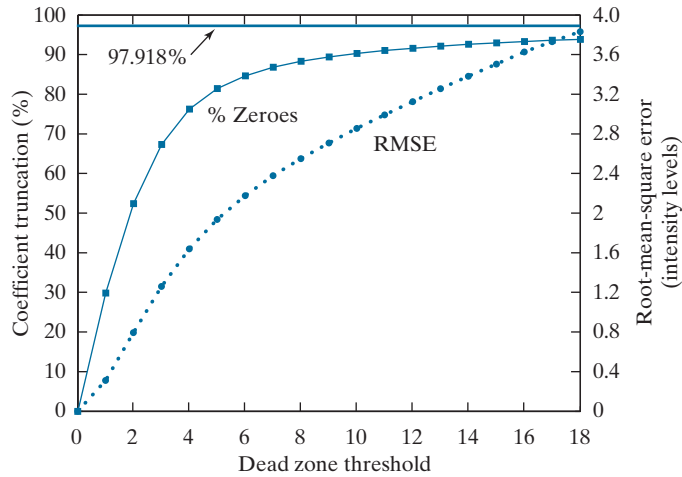
TABLE 8.15

Decomposition level impact on wavelet coding the 512×512 image of Fig. 8.9(a).

Decomposition Level (Scales or Filter Bank Iterations)	Approximation Coefficient Image	Truncated Coefficients (%)	Reconstruction Error (rms)
1	256×256	74.7%	3.27
2	128×128	91.7%	4.23
3	64×64	95.1%	4.54
4	32×32	95.6%	4.61
5	16×16	95.5%	4.63

FIGURE 8.44

The impact of dead zone interval selection on wavelet coding.



there is little gain. This is due to the fact that the histogram of the detail coefficients is highly peaked around zero.

The rms reconstruction errors corresponding to the dead zone thresholds in Fig. 8.44 increase from 0 to 1.94 intensity levels at a threshold of 5, and to 3.83 intensity levels for a threshold of 18, where the number of zeroes reaches 93.85%. If every detail coefficient were eliminated, that percentage would increase to about 97.92% (by about 4%), but the reconstruction error would grow to 12.3 intensity levels.

JPEG-2000

JPEG-2000 extends the popular JPEG standard to provide increased flexibility in both the compression of continuous-tone still images and access to the compressed data. For example, portions of a JPEG-2000 compressed image can be extracted for retransmission, storage, display, and/or editing. The standard is based on the wavelet coding techniques just described. Coefficient quantization is adapted to individual scales and subbands, and the quantized coefficients are arithmetically coded on a bit-plane basis (see Sections 8.4 and 8.8). Using the notation of the standard, an image is encoded as follows (ISO/IEC [2000]).

The first step of the encoding process is to DC level shift the samples of the $Ssiz$ -bit unsigned image to be coded by subtracting 2^{Ssiz-1} . If the image has more than one component, such as the red, green, and blue planes of a color image, each component is shifted individually. If there are exactly three components, they may be optionally decorrelated using a reversible or nonreversible linear combination of the components. The *irreversible component transform* of the standard, for example, is

$$\begin{aligned} Y_0(x, y) &= 0.299I_0(x, y) + 0.587I_1(x, y) + 0.114I_2(x, y) \\ Y_1(x, y) &= -0.16875I_0(x, y) - 0.33126I_1(x, y) + 0.5I_2(x, y) \\ Y_2(x, y) &= 0.5I_0(x, y) - 0.41869I_1(x, y) - 0.08131I_2(x, y) \end{aligned} \quad (8-60)$$

$Ssiz$ is used in the standard to denote intensity resolution.

The irreversible component transform is the component transform used for lossy compression. The component transform itself is not irreversible. A different component transform is used for reversible compression.

where I_0 , I_1 , and I_2 are the level-shifted input components, and Y_0 , Y_1 , and Y_2 are the corresponding decorrelated components. If the input components are the red, green, and blue planes of a color image, Eq. (8-60) approximates the $R'G'B'$ to $Y'C_bC_r$ color video transform (Poynton [1996]).[†] The goal of the transformation is to improve compression efficiency; transformed components Y_1 and Y_2 are difference images whose histograms are highly peaked around zero.

After the image has been level-shifted and optionally decorrelated, its components can be divided into *tiles*. Tiles are rectangular arrays of pixels that are processed independently. Because an image can have more than one component (e.g., it could be made up of three color components), the tiling process creates *tile components*. Each tile component can be reconstructed independently, providing a simple mechanism for accessing and/or manipulating a limited region of a coded image. For example, an image having a 16:9 aspect ratio could be subdivided into tiles so one of its tiles is a subimage with a 4:3 aspect ratio. That tile then could be reconstructed without accessing the other tiles in the compressed image. If the image is not subdivided into tiles, it is a single tile.

The 1-D discrete wavelet transform of the rows and columns of each tile component is then computed. For error-free compression, the transform is based on a biorthogonal, 5/3 coefficient scaling and wavelet vector (Le Gall and Tabatabai [1988]). A rounding procedure is defined for non-integer-valued transform coefficients. In lossy applications, a 9/7 coefficient scaling-wavelet vector is employed (Antonini, Barlaud, Mathieu, and Daubechies [1992]). In either case, the transform is computed using the fast wavelet transform of Section 7.10 or via a complementary *lifting-based* approach (Mallat [1999]). For example, in lossy applications, the coefficients used to construct the 9/7 FWT analysis filter bank are given in Table 7.1. The complementary lifting-based implementation involves six sequential “lifting” and “scaling” operations:

$$\begin{aligned}
 Y(2n+1) &= X(2n+1) + \alpha[X(2n) + X(2n+2)] & i_0 - 3 \leq 2n+1 < i_1 + 3 \\
 Y(2n) &= X(2n) + \beta[Y(2n-1) + Y(2n+1)] & i_0 - 2 \leq 2n < i_1 + 2 \\
 Y(2n+1) &= Y(2n+1) + \gamma[Y(2n) + Y(2n+2)] & i_0 - 1 \leq 2n+1 < i_1 + 1 \\
 Y(2n) &= Y(2n) + \delta[Y(2n-1) + Y(2n+1)] & i_0 \leq 2n < i_1 \\
 Y(2n+1) &= -KY(2n+1) & i_0 \leq 2n+1 < i_1 \\
 Y(2n) &= Y(2n)/K & i_0 \leq 2n < i_1
 \end{aligned}$$

(8-61)

Here, X is the tile component being transformed, Y is the resulting transform, and i_0 and i_1 define the position of the tile component within a component. That is, they are the indices of the first sample of the tile-component row or column being transformed and the one immediately following the last sample. Variable

Lifting-based implementations are another way to compute wavelet transforms. The coefficients used in the approach are directly related to the FWT filter bank coefficients.

[†] $R'G'B'$ is a gamma-corrected, nonlinear version of a linear CIE (International Commission on Illumination) RGB colorimetry value. Y' is luminance and C_b and C_r are color differences (i.e., scaled $B' - Y'$ and $R' - Y'$ values).

These lifting-based coefficients are specified in the standard.

Recall from Chapter 7 that the DWT decomposes an image into a set of band-limited components called subbands.

n assumes values based on i_0 , i_1 , and determines which of the six operations is being performed. If $n < i_0$ or $n > i_1$, $X(n)$ is obtained by symmetrically extending X . For example, $X(i_0 - 1) = X(i_0 + 1)$, $X(i_0 - 2) = X(i_0 + 2)$, $X(i_1) = X(i_1 - 2)$, and $X(i_1 + 1) = X(i_1 - 3)$. At the conclusion of the lifting and scaling operations, the even-indexed values of Y are equivalent to the FWT lowpass filtered output; the odd-indexed values of Y correspond to the highpass FWT filtered result. Lifting parameters α , β , γ , and δ are -1.586134342 , -0.052980118 , 0.882911075 , and 0.433506852 , respectively, and scaling factor K is 1.230174105 .

The transformation just described produces four subbands; a low-resolution approximation of the tile component and the component's horizontal, vertical, and diagonal frequency characteristics. Repeating the transformation N_L times, with subsequent iterations restricted to the previous decomposition's approximation coefficients, produces an N_L -scale wavelet transform. Adjacent scales are related spatially by powers of 2, and the lowest scale contains the only explicitly defined approximation of the original tile component. As can be surmised from Fig. 8.45, where the notation of the JPEG-2000 standard is summarized for the case of $N_L = 2$, a general N_L -scale transform contains $3N_L + 1$ subbands whose coefficients are denoted a_b for $b = N_L HL, \dots, 1HL, 1LH, 1HH$. The standard does not specify the number of scales to be computed.

When each of the tile components has been processed, the total number of transform coefficients is equal to the number of samples in the original image, but the important visual information is concentrated in a few coefficients. To reduce the number of bits needed to represent the transform, coefficient $a_b(u, v)$ of subband b is quantized to value $q_b(u, v)$ using

$$q_b(u, v) = \text{sign}[a_b(u, v)] \cdot \text{floor} \left[\frac{|a_b(u, v)|}{\Delta_b} \right] \quad (8-62)$$

where the *quantization step size* Δ_b is

$$\Delta_b = 2^{R_b - \varepsilon_b} \left(1 + \frac{\mu_b}{2^{11}} \right) \quad (8-63)$$

Do not confuse the standard's definition of nominal dynamic range with the closely related definition in Chapter 2.

R_b is the *nominal dynamic range* of subband b , while ε_b and μ_b are the number of bits allotted to the *exponent* and *mantissa* of the subband's coefficients. The nominal dynamic range of subband b is the sum of the number of bits used to represent the original image and the *analysis gain* bits for subband b . Subband analysis gain bits follow the simple pattern shown in Fig. 8.45. For example, there are two analysis gain bits for subband $b = 1HH$.

For error-free compression, $\mu_b = 0$, $R_b = \varepsilon_b$, and $\Delta_b = 1$. For irreversible compression, no particular quantization step size is specified in the standard. Instead, the number of exponent and mantissa bits must be provided to the decoder on a subband basis, called *expounded quantization*, or for the $N_L LL$ subband only, called *derived quantization*. In the latter case, the remaining subbands are quantized using

extrapolated N_{LL} subband parameters. Letting ε_0 and μ_0 be the number of bits allocated to the N_{LL} subband, the extrapolated parameters for subband b are

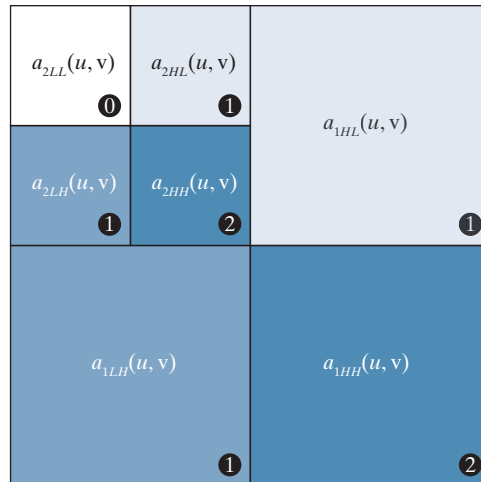
$$\begin{aligned}\mu_b &= \mu_0 \\ \varepsilon_b &= \varepsilon_0 + n_b - N_L\end{aligned}\tag{8-64}$$

where n_b denotes the number of subband decomposition levels from the original image tile component to subband b .

In the final steps of the encoding process, the coefficients of each transformed tile-component's subbands are arranged into rectangular blocks called *code blocks*, which are coded individually, one bit plane at a time. Starting from the most significant bit plane with a nonzero element, each bit plane is processed in three passes. Each bit (in a bit plane) is coded in only one of the three passes, which are called *significance propagation*, *magnitude refinement*, and *cleanup*. The outputs are then arithmetically coded and grouped with similar passes from other code blocks to form *layers*. A layer is an arbitrary number of groupings of coding passes from each code block. The resulting layers finally are partitioned into *packets*, providing an additional method of extracting a spatial region of interest from the total code stream. Packets are the fundamental unit of the encoded code stream.

JPEG-2000 decoders simply invert the operations previously described. After reconstructing the subbands of the tile-components from the arithmetically coded JPEG-2000 packets, a user-selected number of the subbands is decoded. Although the encoder may have encoded M_b bit planes for a particular subband, the user, due to the embedded nature of the code stream, may choose to decode only N_b bit planes. This amounts to quantizing the coefficients of the code block using a step

FIGURE 8.45
JPEG 2000 two-scale wavelet transform tile-component coefficient notation and analysis gain.



Quantization as defined earlier in the chapter is irreversible. The term “inverse quantized” does not mean that there is no information loss. This process is lossy except for the case of reversible JPEG-2000 compression, where $\mu_b = 0$, $R_b = \epsilon_b$, and $\Delta_b = 1$.

size of $2^{M_b - N_b} \cdot \Delta_b$. Any nondecoded bits are set to zero and the resulting coefficients, denoted $\bar{q}_b(u, v)$, are inverse quantized using

$$R_{q_b}(u, v) = \begin{cases} \left(\bar{q}_b(u, v) + r \cdot 2^{M_b - N_b(u, v)} \right) \cdot \Delta_b & \bar{q}_b(u, v) > 0 \\ \left(\bar{q}_b(u, v) - r \cdot 2^{M_b - N_b(u, v)} \right) \cdot \Delta_b & \bar{q}_b(u, v) < 0 \\ 0 & \bar{q}_b(u, v) = 0 \end{cases} \quad (8-65)$$

where $R_{q_b}(u, v)$ denotes an inverse-quantized transform coefficient, and $N_b(u, v)$ is the number of decoded bit planes for $\bar{q}_b(u, v)$. *Reconstruction parameter* r is chosen by the decoder to produce the best visual or objective quality of reconstruction. Generally, $0 \leq r < 1$, with a common value being $r = 1/2$. The inverse-quantized coefficients then are inverse-transformed by column and by row using an FWT⁻¹ filter bank whose coefficients are obtained from Table 7.1, or via the following lifting-based operations:

$$\begin{aligned} X(2n) &= K \cdot Y(2n) & i_0 - 3 \leq 2n < i_1 + 3 \\ X(2n+1) &= (-1/K)Y(2n+1) & i_0 - 2 \leq 2n - 1 < i_1 + 2 \\ X(2n) &= X(2n) - \delta[X(2n-1) + X(2n+1)] & i_0 - 3 \leq 2n < i_1 + 3 \\ X(2n+1) &= X(2n+1) - \gamma[X(2n) + X(2n+2)] & i_0 - 2 \leq 2n + 1 < i_1 + 2 \\ X(2n) &= X(2n) - \beta[X(2n-1) + X(2n+1)] & i_0 - 1 \leq 2n < i_1 + 1 \\ X(2n+1) &= X(2n+1) - \alpha[X(2n) + X(2n+2)] & i_0 \leq 2n + 1 < i_1 \end{aligned} \quad (8-66)$$

where parameters α , β , γ , δ , and K are as defined for Eq. (8-61). Inverse-quantized coefficient row or column element $Y(n)$ is symmetrically extended when necessary. The final decoding steps are the assembly of the component tiles, inverse component transformation (if required), and DC level shifting. For irreversible coding, the inverse component transformation is

$$\begin{aligned} I_0(x, y) &= Y_0(x, y) + 1.402Y_2(x, y) \\ I_1(x, y) &= Y_0(x, y) - 0.34413Y_1(x, y) - 0.71414Y_2(x, y) \\ I_2(x, y) &= Y_0(x, y) + 1.772Y_1(x, y) \end{aligned} \quad (8-67)$$

and the transformed pixels are shifted by $+2^{S_{siz}-1}$.

EXAMPLE 8.28: A comparison of JPEG-2000 wavelet-based coding and JPEG DCT-based compression.

Figure 8.46 shows four JPEG-2000 approximations of the monochrome image in Figure 8.9(a). Successive rows of the figure illustrate increasing levels of compression, including $C = 25, 52, 75$, and 105. The images in column 1 are decompressed JPEG-2000 encodings. The differences between these images and the original image [see Fig. 8.9(a)] are shown in the second column, and the third column contains a zoomed portion of the reconstructions in column 1. Because the compression ratios for the first two rows are virtually identical to the compression ratios in Example 8.18, these results can be compared (both qualitatively and quantitatively) to the JPEG transform-based results in Figs. 8.29(a) through (f).



FIGURE 8.46 Four JPEG-2000 approximations of Fig. 8.9(a). Each row contains a result after compression and reconstruction, the scaled difference between the result and the original image, and a zoomed portion of the reconstructed image. (Compare the results in rows 1 and 2 with the JPEG results in Fig. 8.29.).

A visual comparison of the error images in rows 1 and 2 of Fig. 8.46 with the corresponding images in Figs. 8.29(b) and (e) reveals a noticeable decrease of error in the JPEG-2000 results—3.86 and 5.77 intensity levels, as opposed to 5.4 and 10.7 intensity levels for the JPEG results. The computed errors favor the wavelet-based results at both compression levels. Besides decreasing reconstruction error, wavelet coding dramatically increases (in a subjective sense) image quality. Note that the blocking artifact that dominated the JPEG results [see Figs. 8.29(c) and (f)] is not present in Fig. 8.46. Finally, we note that the compression achieved in rows 3 and 4 of Fig. 8.46 is not practical with JPEG. JPEG-2000 provides useable images that are compressed by more than 100:1, with the most objectionable degradation being increased image blur.

8.12 DIGITAL IMAGE WATERMARKING

The methods and standards of Sections 8.2 through 8.11 make the distribution of images (in photographs or videos) on digital media and over the Internet practical. Unfortunately, the images so distributed can be copied repeatedly and without error, putting the rights of their owners at risk. Even when encrypted for distribution, images are unprotected after decryption. One way to discourage illegal duplication is to insert one or more items of information, collectively called a *watermark*, into potentially vulnerable images in such a way that the watermarks are inseparable from the images themselves. As integral parts of the watermarked images, they protect the rights of their owners in a variety of ways, including:

1. *Copyright identification.* Watermarks can provide information that serves as proof of ownership when the rights of the owner have been infringed.
2. *User identification or fingerprinting.* The identity of legal users can be encoded in watermarks and used to identify sources of illegal copies.
3. *Authenticity determination.* The presence of a watermark can guarantee that an image has not been altered, assuming the watermark is designed to be destroyed by any modification of the image.
4. *Automated monitoring.* Watermarks can be monitored by systems that track when and where images are used (e.g., programs that search the Web for images placed on Web pages). Monitoring is useful for royalty collection and/or the location of illegal users.
5. *Copy protection.* Watermarks can specify rules of image usage and copying (e.g., to DVD players).

In this section, we provide a brief overview of *digital image watermarking*, which is the process of inserting data into an image in such a way that it can be used to make an assertion about the image. The methods described have little in common with the compression techniques presented in the previous sections (although they do involve the coding of information). In fact, watermarking and compression are in some ways opposites. While the objective in compression is to reduce the amount of data used to represent images, the goal in watermarking is to add information and data (i.e., watermarks) to them. As will be seen in the remainder of the section, the watermarks themselves can be either visible or invisible.

A *visible watermark* is an opaque or semi-transparent subimage or image that is placed on top of another image (i.e., the image being watermarked) so that it is obvious to the viewer. Television networks often place visible watermarks (fashioned after their logos) in the upper or lower right-hand corner of the television screen. As the following example illustrates, visible watermarking typically is performed in the spatial domain.

EXAMPLE 8.29: A simple visible watermark.

The image in Fig. 8.47(b) is the lower right-hand quadrant of the image in Fig. 8.9(a) with a scaled version of the watermark in Fig. 8.47(a) overlaid on top of it. Letting f_w denote the watermarked image, we can express it as a linear combination of the unmarked image f and watermark w using

$$f_w = (1 - \alpha)f + \alpha w \quad (8-68)$$

where constant α controls the relative visibility of the watermark and the underlying image. If α is 1, the watermark is opaque and the underlying image is completely obscured. As α approaches 0, more of the underlying image and less of the watermark is seen. In general, $0 < \alpha \leq 1$; in Fig. 8.47(b), $\alpha = 0.3$. Figure 8.47(c) is the computed difference (scaled in intensity) between the watermarked image in (b) and the unmarked image in Fig. 8.9(a). Intensity 128 represents a difference of 0. Note that the underlying image is clearly visible through the “semi-transparent” watermark. This is evident in both Fig. 8.47(b) and the difference image in Fig. 8.47(c).

Unlike the visible watermark of the previous example, *invisible watermarks* cannot be seen with the naked eye. They are imperceptible but can be recovered with an appropriate decoding algorithm. Invisibility is assured by inserting them as visually redundant information [information that the human visual system ignores or cannot

a
b c

FIGURE 8.47

A simple visible watermark:
(a) watermark;
(b) the watermarked image;
and
(c) the difference between the watermarked image and the original (non-watermarked) image.



perceive (see Section 8.1)]. Figure 8.48(a) provides a simple example. Because the least significant bits of an 8-bit image have virtually no effect on our perception of the image, the watermark from Fig. 8.47(a) was inserted or “hidden” in its two least significant bits. Using the notation introduced above, we let

$$f_w = 4\left(\frac{f}{4}\right) + \frac{w}{64} \quad (8-69)$$

and use unsigned integer arithmetic to perform the calculations. Dividing and multiplying by 4 sets the two least significant bits of f to 0, dividing w by 64 shifts its two most significant bits into the two least significant bit positions, and adding the two results generates the *LSB watermarked image*. Note that the embedded watermark is not visible in Fig. 8.48(a). By zeroing the most significant 6 bits of this image and scaling the remaining values to the full intensity range, however, the watermark can be extracted as in Fig. 8.48(b).

An important property of invisible watermarks is their resistance to both accidental and intentional attempts to remove them. *Fragile invisible watermarks* are destroyed by any modification of the images in which they are embedded. In some applications, like image authentication, this is a desirable characteristic. As Figs. 8.48(c) and (d) show, the LSB watermarked image in Fig. 8.48(a) contains a fragile invisible watermark. If the image in (a) is compressed and decompressed using lossy JPEG, the watermark is destroyed. Figure 8.48(c) is the result after com-

a b
c d

FIGURE 8.48

A simple invisible watermark: (a) watermarked image; (b) the extracted watermark; (c) the watermarked image after high quality JPEG compression and decompression; and (d) the extracted watermark from (c).



Digital Image
Processing

pressing and decompressing Fig. 8.48(a); the rms error is 2.1 bits. If we try to extract the watermark from this image using the same method as in (b), the result is unintelligible [see Fig. 8.48(d)]. Although lossy compression and decompression preserved the important visual information in the image, the fragile watermark was destroyed.

Robust invisible watermarks are designed to survive image modification, whether the so-called *attacks* are inadvertent or intentional. Common inadvertent attacks include lossy compression, linear and nonlinear filtering, cropping, rotation, resampling, and the like. Intentional attacks range from printing and rescanning to adding additional watermarks and/or noise. Of course, it is unnecessary to withstand attacks that leave the image itself unusable.

Figure 8.49 shows the basic components of a typical image watermarking system. The encoder in Fig. 8.49(a) inserts watermark w_i into image f_i producing watermarked image f_{w_i} ; the complementary decoder in (b) extracts and validates the presence of w_i in watermarked input f_{w_i} or unmarked input f_j . If w_i is visible, the decoder is not needed. If it is invisible, the decoder may or may not require a copy of f_i and w_i [shown in blue in Fig. 8.49(b)] to do its job. If f_i and/or w_i are used, the watermarking system is known as a *private* or *restricted-key* system; if not, it is a *public* or *unrestricted-key* system. Because the decoder must process both marked and unmarked images, $w_\emptyset$ is used in Fig. 8.49(b) to denote the absence of a mark. Finally, we note that to determine the presence of w_i in an image, the decoder must correlate extracted watermark w_j with w_i and compare the result to a predefined threshold. The threshold sets the degree of similarity that is acceptable for a “match.”

EXAMPLE 8.30: A DCT-based invisible robust watermark.

Mark insertion and *extraction* can be performed in the spatial domain, as in the previous examples, or in the transform domain. Figures 8.50(a) and (c) show two watermarked versions of the image in Fig. 8.9(a) using the DCT-based watermarking approach outlined here (Cox et al. [1997]):

1. Compute the 2-D DCT of the image to be watermarked.
2. Locate its K largest coefficients, $c_1, c_2, \dots, c_K$, by magnitude.
3. Create a watermark by generating a K -element pseudo-random sequence of numbers, $\omega_1, \omega_2, \dots, \omega_K$, taken from a Gaussian distribution with mean $\mu = 0$ and variance $\sigma^2 = 1$. (Note: A pseudo-random number sequence approximates the properties of random numbers. It is not truly random because it depends on a predetermined initial value.)
4. Embed the watermark from Step 3 into the K largest DCT coefficients from Step 2 using the following equation

$$c'_i = c_i \cdot (1 + \alpha \omega_i) \quad 1 \leq i \leq K \quad (8-70)$$

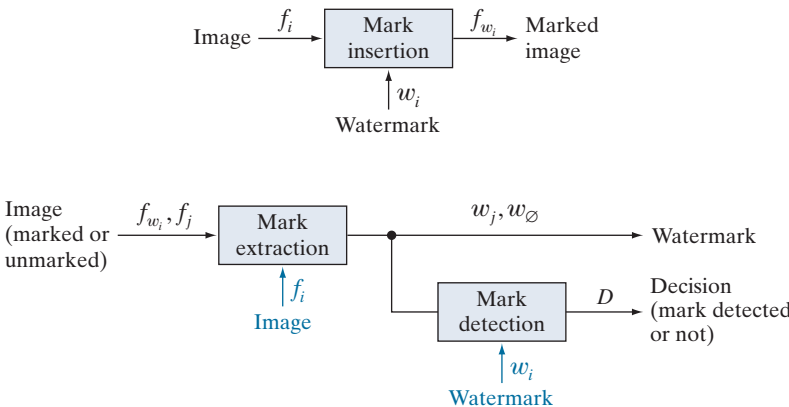
for a specified constant $\alpha > 0$ (that controls the extent to which ω_i alters c_i). Replace the original c_i with the computed c'_i from Eq. (8-70). (For the images in Fig. 8.50, $\alpha = 0.1$ and $K = 1000$.)

5. Compute the inverse DCT of the result from Step 4.

By employing watermarks made from pseudo-random numbers and spreading them across an image's perceptually significant frequency components, α can be made small, reducing watermark visibility. At

a
b

FIGURE 8.49
A typical image watermarking system:
(a) encoder;
(b) decoder.



a b
c d

FIGURE 8.50
(a) and (c) Two watermarked versions of Fig. 8.9(a);
(b) and (d) the differences (scaled in intensity) between the watermarked versions and the unmarked image. These two images show the intensity contribution (although scaled dramatically) of the pseudo-random watermarks on the original image.



the same time, watermark security is kept high because (1) the watermarks are composed of pseudo-random numbers with no obvious structure, (2) the watermarks are embedded in multiple frequency components with spatial impact over the entire 2-D image (so their location is not obvious) and (3) attacks against them tend to degrade the image as well (i.e., the image's most important frequency components must be altered to affect the watermarks).

Figures 8.50(b) and (d) make the changes in image intensity that result from the pseudo-random numbers that are embedded in the DCT coefficients of the watermarked images in Figs. 8.50(a) and (c)

visible. Obviously, the pseudo-random numbers must have an effect (even if too small to see) on the watermarked images. To display the effect, the images in Figs. 8.50(a) and (c) were subtracted from the unmarked image in Fig. 8.9(a) and scaled in intensity to the range $[0, 255]$. Figures 8.50(b) and (d) are the resulting images; they show the 2-D spatial contributions of the pseudo-random numbers. Because they have been scaled, however, you cannot simply add these images to the image in Fig. 8.9(a) and get the watermarked images in Figs. 8.50(a) and (c). As can be seen in Figs. 8.50(a) and (c), their actual intensity perturbations are small to negligible.

To determine whether a particular image is a copy of a previously watermarked image with watermark $\omega_1, \omega_2, \dots, \omega_K$ and DCT coefficients $c_1, c_2, \dots, c_K$, we use the following procedure:

1. Compute the 2-D DCT of the image in question.
2. Extract the K DCT coefficients (in the positions corresponding to $c_1, c_2, \dots, c_K$ of Step 2 in the watermarking procedure) and denote the coefficients as $\hat{c}_1, \hat{c}_2, \dots, \hat{c}_K$. If the image in question is the previously watermarked image (without modification), $\hat{c}_i = c_i$ for $1 \leq i \leq K$. If it is a modified copy of the watermarked image (i.e., it has undergone some sort of attack), $\hat{c}_i \approx c_i$ for $1 \leq i \leq K$ (the $\hat{c}_i$ will be approximations of the c_i). Otherwise, the image in question will be an unmarked image or an image with a completely different watermark, and the $\hat{c}_i$ will bear no resemblance to the original c_i .
3. Compute watermark $\hat{\omega}_1, \hat{\omega}_2, \dots, \hat{\omega}_K$ using

$$\hat{\omega}_i = \frac{\hat{c}_i - c_i}{\alpha c_i} \quad \text{for } 1 \leq i \leq K \quad (8-71)$$

Recall that watermarks are sequences of pseudo-random numbers.

4. Measure the similarity of $\hat{\omega}_1, \hat{\omega}_2, \dots, \hat{\omega}_K$ (from Step 3) and $\omega_1, \omega_2, \dots, \omega_K$ (from Step 3 of the watermarking procedure) using a metric such as the correlation coefficient

$$\gamma = \frac{\sum_{i=1}^K (\hat{\omega}_i - \bar{\hat{\omega}})(\omega_i - \bar{\omega})}{\sqrt{\sum_{i=1}^K (\hat{\omega}_i - \bar{\hat{\omega}})^2 \cdot \sum_{i=1}^K (\omega_i - \bar{\omega})^2}} \quad 1 \leq i \leq K \quad (8-72)$$

where $\bar{\omega}$ and $\bar{\hat{\omega}}$ are the means of the two K -element watermarks. (Note: Correlation coefficients are discussed in detail in Section 12.3.)

5. Compare the measured similarity, γ , to a predefined threshold, T , and make a binary detection decision:

$$D = \begin{cases} 1 & \text{if } \gamma \geq T \\ 0 & \text{otherwise} \end{cases} \quad (8-73)$$

In other words, $D = 1$ indicates that watermark $\omega_1, \omega_2, \dots, \omega_K$ is present (with respect to the specified threshold, T); $D = 0$ indicates that it was not.

Using this procedure, the original watermarked image in Fig. 8.50(a), measured against itself, yields a correlation coefficient of 0.9999, i.e., $\gamma = 0.9999$. It is an unmistakable match. In a similar manner, the image in Fig. 8.50(b), when measured against the image in Fig. 8.50(a), results in a γ of 0.0417. It could not be mistaken for the watermarked image in Fig. 8.50(a) because the correlation coefficient is so low.

To conclude the section, we note that the DCT-based watermarking approach of the previous example is fairly resistant to watermark attacks, partly because it is a private or restricted-key method. Restricted-key methods are always more resilient than their unrestricted-key counterparts. Using the watermarked image in Fig. 8.50(a), Fig. 8.51 illustrates the ability of the method to withstand a variety of common attacks. As can be seen in the figure, watermark detection is quite good over the range of attacks that were implemented; the resulting correlation coefficients (shown under each image in the figure) vary from 0.3113 to 0.9945. When subjected to a high quality but lossy (resulting in an rms error of 7 intensities) JPEG compression and decompression, $\gamma = 0.9945$. Even when the compression and reconstruction yields an rms error of 10 intensity levels, $\gamma = 0.7395$; and the usability of this image has been significantly degraded. Significant smoothing by spatial filtering and the addition of Gaussian noise do not reduce the correlation coefficient below 0.8230. However, histogram equalization reduces γ to 0.5210; and rotation has the largest effect; reducing γ to 0.3313. All attacks, except for the lossy JPEG compression and reconstruction in Fig. 8.51(a), have significantly reduced the usability of the original watermarked image.

Summary, References, and Further Reading

The principal objectives of this chapter were to present the theoretic foundation of digital image compression, to describe the most commonly used compression methods, and to introduce the related area of digital image watermarking. Although the level of the presentation is introductory in nature, the references provide an entry into the extensive body of literature dealing with the topics discussed. As evidenced by the international standards listed in Tables 8.3 through 8.5, compression plays a key role in document image storage and transmission, the Internet, and commercial video distribution (e.g., DVDs). It is one of the few areas of image processing that has received a sufficiently broad commercial appeal to warrant the adoption of widely accepted standards. Image watermarking is becoming increasingly important as more and more images are distributed in compressed digital form.

The introductory material of the chapter, which is generally confined to Section 8.1, is basic to image compression, and may be found in one form or another in most of the general image processing books cited at the end of Chapter 1. For additional information on the human visual system, see Netravali and Limb [1980], as well as Huang [1966], Schreiber and Knapp [1958], and the references cited at the end of Chapter 2. For more on information theory, see the book website or Abramson [1963], Blahut [1987], and Berger [1971]. Shannon's classic paper, "A Mathematical Theory of Communication" [1948], lays the foundation for the area and is another excellent reference. Subjective fidelity criteria are discussed in Frendendall and Behrend [1960]. Throughout the chapter, a variety of compression standards are used in examples. Most of them were implemented using Adobe Photoshop (with freely available compression plug-ins) and/or MATLAB, which is described in Gonzalez et al. [2004]. Compression standards, as a rule, are lengthy and complex; we have not attempted to cover any of them in their entirety. For more information on a particular standard, see the published documents of the appropriate standards organization—the International Standards Organization, International Electrotechnical Commission, and/or the International Telecommunications Union.

The lossy and error-free compression techniques described in Sections 8.2 through 8.11 and watermarking techniques in Section 8.12 are, for the most part, based on the original papers cited in the text. The algorithms covered are representative of the work in this area, but are by no means exhaustive. The material on LZW coding has its origins in the work of Ziv and Lempel [1977, 1978]. The material on arithmetic coding follows the development in Witten, Neal, and Cleary [1987]. One of the more important implementations of arithmetic coding is summarized in Pennebaker et al. [1988]. For a good discussion of lossless predictive coding, see the tutorial by Rabbani and Jones [1991]. The adaptive predictor of Eq. (8-55) is from Graham [1958]. For more on motion compensation, see S. Solari [1997], which also contains an introduction to general video compression and compression standards, and Mitchell et al. [1997]. The DCT-based watermarking technique in Section 8.12 is based on the paper by Cox et al. [1997]. For more on watermarking, see the books by Cox et al. [2001] and Parhi and Nishitani [1999]. See also the paper by S. Mohanty [1999].



a	b	c
d	e	f

FIGURE 8.51 Attacks on the watermarked image in Fig. 8.50(a): (a) lossy JPEG compression and decompression with an rms error of seven intensity levels; (b) lossy JPEG compression and decompression with an rms error of 10 intensity levels (note the blocking artifact); (c) smoothing by spatial filtering; (d) the addition of Gaussian noise; (e) histogram equalization; and (f) rotation. Each image is a modified version of the watermarked image in Fig. 8.50(a). After modification, they retain their watermarks to varying degrees, as indicated by the correlation coefficients below each image.

Many survey articles have been devoted to the field of image compression. Noteworthy are Netravali and Limb [1980], A. K. Jain [1981], a special issue on picture communication systems in the *IEEE Transactions on Communications* [1981], a special issue on the encoding of graphics in the *Proceedings of IEEE* [1980], a special issue on visual communication systems in the *Proceedings of the IEEE* [1985], a special issue on image sequence compression in the *IEEE Transactions on Image Processing* [1994], and a special issue on vector quantization in the *IEEE Transactions on Image Processing* [1996]. In addition, most issues of the *IEEE Transactions on Image Processing*, *IEEE Transactions on Circuits and Systems for Video Technology*, and *IEEE Transactions on Multimedia* include articles on video and still image compression, motion compensation, and watermarking.

Problems

Solutions to the problems marked with an asterisk (*) are in the DIP4E Student Support Package (consult the book website: www.ImageProcessingPlace.com).

8.1 Answer the following.

- (a) Can variable-length coding procedures be used to compress a histogram equalized image with 2^n intensity levels? Explain.
- (b) Can such an image contain spatial or temporal redundancies that could be exploited for data compression?

8.2 One variation of run-length coding involves (1) coding only the runs of 0's or 1's (not both) and (2) assigning a special code to the start of each line to reduce the effect of transmission errors. One possible code pair is (x_k, r_k) , where x_k and r_k represent the k th run's starting coordinate and run length, respectively. The code (0, 0) is used to signal each new line.

- (a) Derive a general expression for the maximum average runs per scan line required to guarantee data compression when run-length coding a $2^n \times 2^n$ binary image.
- (b) Compute the maximum allowable value for $n = 10$.

8.3 Consider an 8-pixel line of intensity data, {108, 139, 135, 244, 172, 173, 56, 99}. If it is uniformly quantized with 4-bit accuracy, compute the rms error and rms signal-to-noise ratios for the quantized data.

8.4* Although quantization results in information loss, it is sometimes invisible to the eye. For example, when 8-bit pixels are uniformly quantized to fewer bits/pixel, false contouring often occurs. It can be reduced or eliminated using *improved gray-scale (IGS) quantization*. A sum (initially set to zero) is formed from the current 8-bit intensity value and the four least significant bits of the previously generated sum. If the four most significant bits of the intensity value are 1111_2 , however, 0000_2 is added instead. The four most significant bits of the resulting sum are used as the coded pixel value.

- (a) Construct the IGS code for the intensity data in Problem 8.3.

- (b) Compute the rms error and rms signal-to-noise ratios for the IGS data.

8.5 A 1024×1024 8-bit image with 5.3 bits/pixel entropy [computed from its histogram using Eq. (8-7)] is to be Huffman coded.

- (a) What is the maximum compression that can be expected?
- (b) Will it be obtained?
- (c) If a greater level of lossless compression is required, what else can be done?

8.6* The base e unit of information is commonly called a *nat*, and the base-10 information unit is called a *Hartley*. Compute the conversion factors needed to relate these units to the base-2 unit of information (the bit).

8.7* Prove that, for a zero-memory source with q symbols, the maximum value of the entropy is $\log q$, which is achieved if and only if all source symbols are equiprobable. [Hint: Consider the quantity $\log q - H(z)$ and note the inequality $\ln x \leq x - 1$.]

8.8 Answer the following.

- (a) How many unique Huffman codes are there for a three-symbol source?
- (b) Construct them.

8.9 Consider the simple 4×8 , 8-bit image:

```

21 21 21 95 169 243 243 243
21 21 21 95 169 243 243 243
21 21 21 95 169 243 243 243
21 21 21 95 169 243 243 243

```

- (a) Compute the entropy of the image.
- (b) Compress the image using Huffman coding.
- (c) Compute the compression achieved and the effectiveness of the Huffman coding.
- (d)* Consider Huffman encoding pairs of pixels rather than individual pixels. That is, consider the image to be produced by the second extension of the zero-memory source

that produced the original image. What is the entropy of the image when looked at as pairs of pixels?

- (e) Consider coding the differences between adjacent pixels. What is the entropy of the new difference image? What does this tell us about compressing the image?
 - (f) Explain the entropy differences in (a), (d) and (e).
- 8.10** Using the Huffman code in Fig. 8.8, decode the encoded string 010100000101011110100.
- 8.11** Compute Golomb code $G_3(n)$ for $0 \leq n \leq 15$.
- 8.12** Write a general procedure for decoding Golomb code $G_m(n)$.
- 8.13** Why is it not possible to compute the Huffman code of the nonnegative integers, $n \geq 0$, with the probability mass function of Eq. (8-13)?
- 8.14** Compute exponential Golomb code $G_{\text{exp}}^2(n)$ for $0 \leq n \leq 15$.
- 8.15*** Write a general procedure for decoding exponential Golomb code $G_{\text{exp}}^k(n)$.
- 8.16** Plot the optimal Golomb coding parameter m as a function of ρ for $0 < \rho < 1$ in Eq. (8-14).
- 8.17** Given a four-symbol source $\{a, b, c, d\}$ with source probabilities $\{0.1, 0.4, 0.3, 0.2\}$, arithmetically encode the sequence $bbadc$.
- 8.18*** The arithmetic decoding process is the reverse of the encoding procedure. Decode the message 0.23355 given the coding model

Symbol	Probability
a	0.2
e	0.3
i	0.1
o	0.2
u	0.1
!	0.1

- 8.19** Use the LZW coding algorithm to encode the 7-bit ASCII string “aaaaaaaaa”.
- 8.20*** Devise an algorithm for decoding the LZW encoded output of Example 8.7. Since the dictionary that was used during the encoding is not available, the code book must be reproduced as the output is decoded.

- 8.21** Decode the BMP encoded sequence $\{3, 4, 5, 6, 0, 3, 103, 125, 67, 0, 2, 47\}$.

- 8.22** Do the following:

- (a) Construct the entire 4-bit Gray code.
- (b) Create a general procedure for converting a Gray-coded number to its binary equivalent and use it to decode 0111010100111.

- 8.23** Use the CCITT Group 4 compression algorithm to code the second line of the following two-line segment:

01100111001111111100001

11111110001110000111111

Assume that the initial reference element a_0 is located on the first pixel of the second line segment. (Note: Employ the CCITT 2-D code table from the book website.)

- 8.24*** Do the following.

- (a) List all the members of JPEG DC coefficient difference category 3.
- (b) Compute their default Huffman codes using the appropriate Huffman code table from the book website.

- 8.25** How many computations are required to find the optimal motion vector of a macroblock of size 8×8 using the MAD optimality criterion, single pixel precision, and a maximum allowable displacement of 8 pixels? What would it become for $\frac{1}{4}$ pixel precision?

- 8.26** What are the advantages of using B-frames for motion compensation?

- 8.27*** Draw the block diagram of the companion motion compensated video decoder for the encoder in Fig. 8.36.

- 8.28** An image whose autocorrelation function is of the form of Eq. (8-48) with $\rho_h = 0$ is to be DPCM coded using a second-order predictor.

- (a) Form the autocorrelation matrix $\mathbf{R}$ and vector $\mathbf{r}$.
- (b) Find the optimal prediction coefficients.
- (c) Compute the variance of the prediction error that would result from using the optimal coefficients.

- 8.29*** Derive the Lloyd-Max decision and reconstruction levels for $L = 4$ and the uniform probability density function

$$p(s) = \begin{cases} \frac{1}{2A} & -A \leq s \leq A \\ 0 & \text{otherwise} \end{cases}$$

- 8.30** A radiologist from a well-known research hospital recently attended a medical conference at which a system that could transmit 4096×4096 12-bit digitized X-ray images over standard T1 (1.544 Mb/s) phone lines was exhibited. The system transmitted the images in a compressed format using a progressive technique in which a reasonably good approximation of the X-ray was first reconstructed at the viewing station, then refined gradually to produce an error-free display. The transmission of the data needed to generate the first approximation took approximately 5 or 6 s. Refinements were made every 5 or 6 s (on the average) for the next 1 min, with the first and last refinements having the most and least significant impact on the reconstructed X-ray, respectively. The physician was favorably impressed with the system, because she could begin her diagnosis by using the first approximation of the X-ray and complete it as the error-free reconstruction of the X-ray was being generated. Upon returning to her office, she submitted a purchase request to the hospital administrator. Unfortunately, the hospital was on a relatively tight budget, which recently had been stretched by the hiring of an aspiring young electrical engineering graduate. To

appease the radiologist, the administrator gave the young engineer the task of designing such a system. (He thought it might be cheaper to design and build a similar system in-house. The hospital currently owned some of the elements of such a system, but the transmission of the raw X-ray data took more than 2 min.) The administrator asked the engineer to have an initial block diagram by the afternoon staff meeting. With little time and only a copy of *Digital Image Processing* from his recent school days in hand, the engineer was able to devise a system conceptually to satisfy the transmission and associated compression requirements. Construct a conceptual block diagram of such a system, specifying the compression techniques you would recommend.

- 8.31** Show that the lifting-based wavelet transform defined by Eq. (8-61) is equivalent to the traditional FWT filter bank implementation using the coefficients in Table 7.1. Define the filter coefficients in terms of α , β , γ , δ , and K .
- 8.32** Compute the quantization step sizes of the subbands for a JPEG-2000 encoded image in which derived quantization is used and 8 bits are allotted to the mantissa and exponent of the $2LL$ subband.
- 8.33** How would you add a visible watermark to an image in the frequency domain?
- 8.34*** Design an invisible watermarking system based on the discrete Fourier transform.
- 8.35** Design an invisible watermarking system based on the discrete wavelet transform.